Meta-RoCE Protocol for AI Accelerators v 1.0

October 12, 2026

Contents

I Introduction and Overview 15

1 Introduction 17
- 1.1 Motivation . . . 17
- 1.2 Fabric Features . . . 18
- 1.3 Design Overview . . . 19
- 1.4 Congestion Control and Loss Recovery . . . 19
- 1.5 Supported Operations . . . 20
- 1.6 Document Organization . . . 21

II Semantics and Ordering 23

2 Sequence Numbers 25
- 2.1 Overview . . . 25
- 2.2 Message Sequence Number . . . 26
- 2.3 Packet Offset Sequence Number . . . 28
  - 2.3.1 Endpoints must agree on MTU . . . 29
- 2.4 Completion Sequence Number . . . 30
- 2.5 Flow Sequence Number . . . 31

3 Ordering 33
- 3.1 Terminology . . . 33
- 3.2 Supported Message Sequences . . . 35
  - 3.2.1 Writes with Write/immediate . . . 35
  - 3.2.2 Writes with Atomics . . . 35
  - 3.2.3 Concurrent Reads . . . 36
  - 3.2.4 Pipelined Sends . . . 36
  - 3.2.5 Specific Undefined Message Sequences . . . 37
- 3.3 Completion Ordering Modes . . . 38
  - 3.3.1 Ordered Completion Mode . . . 38
  - 3.3.2 Unordered Completion Mode . . . 39
  - 3.3.3 Mixing Ordered and Unordered WQE Completions (optional) . . . 39
- 3.4 In-Protocol Message Sequencing . . . 39
  - 3.4.1 Shared receive queues . . . 40
  - 3.4.2 Applied examples . . . 41

3.4.3 Atomic ordering . . . 43
3.4.4 State required . . . 43

4 Reliable Delivery **45**
4.1 Bit Error Detection (mCRC) . . . 45
4.2 Acknowledgment Generation . . . 46
4.2.1 Transmitter sets the ack request bit . . . 46
4.2.2 Receiver sends acknowledgment . . . 47
4.2.3 Acknowledgment contents: FSN fields . . . 48
4.2.4 Acknowledgment contents: MSN field . . . 50
4.2.5 MSN Fields and Delayed Update . . . 50
4.3 Acknowledgment contents: Additional . . . 51
4.4 Packet Trackers . . . 54
4.4.1 Transmitter Packet Tracker, per flow . . . 54
4.4.2 Receiver Packet Tracker, per flow . . . 54
4.5 Packet loss detection by per-flow timeout . . . 54
4.5.1 RTT Measurement . . . 54
4.5.2 RTO computation . . . 56
4.5.3 RTO retransmission count . . . 57
4.6 Packet Loss Detection by Out of Order Receipt / SACK . . . 57
4.6.1 In-network packet reordering . . . 58
4.7 Eager Tail Retransmission (optimization) . . . 59
4.8 Alternate Flow Retransmission . . . 60
4.9 FSN UNA (Clear) field . . . 60
4.10 RNR Data Recovery . . . 62
4.10.1 RNR-Cancel . . . 64
4.10.2 Acknowledgment within a BRNR'd message . . . 65

5 Error Handling **67**
5.1 Retriable Errors . . . 67
5.2 Recoverable error examples . . . 68
5.2.1 Invalid Flow ID . . . 68
5.2.2 Invalid Tag . . . 68
5.2.3 Invalid or not cached QP . . . 69
5.2.4 Source QP mismatch . . . 69
5.2.5 Undisclosed padding . . . 69
5.2.6 QP not in a valid state . . . 69
5.3 Message-fatal errors . . . 70
5.3.1 Reliability layer or connectivity failure . . . 70
5.3.2 End-to-end message failure . . . 70
5.3.3 Work queue error . . . 70
5.3.4 QP error . . . 71
5.3.5 CQ error . . . 71
5.4 Requester side errors . . . 71
5.5 Responder side errors . . . 73
5.5.1 Memory region validation . . . 76

5.5.2 Unrecognized opcodes . . . 76

III Scheduling of Data Transmission: Flows, Congestion, Limits 79

6 Congestion Control 81
6.1 Baseline Congestion Control . . . 81
6.1.1 QP and Flow Windows . . . 82
6.1.2 Baseline CC Theory of Operation . . . 83
6.1.3 Fast Start . . . 85
6.2 Advanced/Programmable Congestion Control . . . 89
6.2.1 Constraints . . . 89
6.2.2 Available Signals . . . 90
6.2.3 Algorithm state storage . . . 92
6.3 Receiver Driven Rate Limiting . . . 92
6.3.1 Signals Available to Calculate `TargetRate` . . . 92
6.3.2 Converting `TargetRate` to a `QWND` cap . . . 93
6.3.3 Reduction to target rate . . . 94
6.3.4 Interaction with ECN Marking . . . 95

7 Multipath and Flow Selection 97
7.1 Selection of Flow Entropy Values . . . 97
7.1.1 Acknowledgment Entropy Values . . . 98
7.2 Transmit Packet Scheduling . . . 98
7.3 Number of Flows per QP . . . 99
7.3.1 Flow context . . . 99
7.4 Adding (activating) and Removing (in-activating) Flows . . . 100
7.4.1 Inactive transmit flows . . . 100
7.4.2 Activating transmit flows . . . 100
7.5 Multi-port Interfaces . . . 102
7.5.1 Requirements . . . 102
7.5.2 State . . . 102
7.5.3 Initial Flow to Port Assignment . . . 103
7.5.4 Failover Algorithm . . . 103
7.5.5 Port status maintenance . . . 105
7.5.6 Port status reporting in acknowledgment . . . 105
7.5.7 Hysteresis / debouncing . . . 106

IV Headers 107

8 Packet Headers 109
8.1 Base Header . . . 109
8.1.1 Opcodes . . . 109
8.1.2 Version . . . 110
8.1.3 Flow ID . . . 110
8.1.4 Destination and Source QP . . . 110

8.1.5 Packet validation . . . 110
8.2 Data Packet Header (METH) . . . 113
8.2.1 Rate hints / Desired Rate . . . 114
8.2.2 Transmit Timestamp . . . 114
8.2.3 Zero-byte opcodes . . . 114
8.3 Read Request and Write Headers . . . 116
8.4 Immediate Data (ImmDt) . . . 116
8.5 Ack Packet . . . 118
8.5.1 CNP Congestion notification . . . 118
8.5.2 Tag . . . 119
8.5.3 Cumulative FSN . . . 119
8.5.4 Max FSN . . . 120
8.5.5 Cumulative Responder Delivered MSN (CDMSN) . . . 120
8.5.6 Ack Status . . . 120
8.5.7 Active Port Set . . . 123
8.5.8 Rate Hints / Permitted Gbps . . . 123
8.5.9 Ack Packet FSN Bit Vector . . . 123
8.5.10 Ack Packet RNR Bit Vector . . . 124
8.6 RNR-Cancel / Receiver Is Ready Packet . . . 125
8.7 Control Packet Header . . . 125
8.7.1 Ping (Echo) Control Packet Headers . . . 126
8.8 Atomic Headers . . . 128
8.8.1 Atomic Extended Transport Header (AtomicETH) . . . 128
8.8.2 Atomic Ack Extended Transport Header (AtomicAckETH) . . . 128
8.8.3 Send Last Opcode . . . 129
8.8.4 Invalidate Extended Transport Header (IETH) . . . 129

V Implementation 131

9 Implementation Conformance Tables 133
9.1 Required and Optional Implementation Features . . . 133
9.2 Packet validity . . . 148

10 Verbs Library Integration 153
10.1 QP Type . . . 153
10.2 Profile Selection . . . 154
10.3 Initial CSN/MSN . . . 154
10.4 Retransmission Timeout and Persistent Retry . . . 154
10.5 QP MTU . . . 155
10.6 Tags . . . 155
10.6.1 Creating a Tag . . . 156
10.6.2 Use of the returned QP structure . . . 156
10.6.3 Timing restrictions . . . 156
10.6.4 Tags on both endpoints . . . 156
10.6.5 Miscellaneous . . . 156

10.7 Read with Immediate 157
10.8 Multi-port API 157
10.8.1 Extending the semantics of `ibv_modify_qp()` 157

List of Figures

1.1 Protocol layer overview . . . 20

2.1 Sequence numbers overview . . . 26
2.2 Message Sequence Numbers . . . 27
2.3 Packet Offset Sequence Numbers . . . 28
2.4 Completion Sequence Numbers . . . 30
2.5 Flow Sequence Numbers . . . 31

3.1 Supported message sequence: Writes with Write/Immediate . . . 35
3.2 Supported message sequence: Writes with Send . . . 36
3.3 Supported message sequence: Writes with Atomics . . . 36
3.4 Supported message sequence: Concurrent Reads . . . 37
3.5 Supported message sequence: Receives completed out of order in unordered completion mode. . . . 37
3.6 Completion sequence: Writes with Write/Immediate . . . 41
3.7 Completion sequence: Writes with Write/Immediate, with in-network reordering . . . 41
3.8 Completion sequence: Writes with Atomics . . . 42
3.9 Completion sequence: Receives completed out of order . . . 42

4.1 mCRC coverage . . . 46
4.2 Selective acknowledgment fields . . . 49
4.3 Flow sequence number fields packet sequence . . . 51
4.4 Message sequence number fields packet sequence . . . 52
4.5 Message sequence number fields if acknowledgments are generated before message completion is checked. . . . 53
4.6 Round-trip-time measurement using echoed timestamps . . . 55
4.7 Round-trip-time measurement with out-of-order packets . . . 55
4.8 Sender retransmission cursor . . . 57
4.9 Sender window state . . . 58
4.10 Partial RTO burst with ACK-driven continuation . . . 59
4.11 Basic example of sending UNA in data packets . . . 61
4.12 RNR initial state . . . 63
4.13 RNR resolved state . . . 64

6.1 QP and flow window hierarchy . . . 82

6.2 Transmission order in fast start . . . 87
6.3 Multi-port transmission order in fast start . . . 88
6.4 Interleaved transmission order in fast start . . . 88
6.5 Rate reduction zones . . . 95

7.1 Acknowledgment generation from data headers . . . 98
7.2 Flow state machine, sender side. . . . 101
7.3 Flow-to-port assignment and failover direction . . . 104

8.1 Base header format . . . 110
8.2 Packet header format, Operation Type == Send || Read Response . . . 113
8.3 Packet header format, Operation Type == Read Request || Write . . . 116
8.4 Immediate Data header . . . 117
8.5 Ack packet format . . . 119
8.6 RNR-Cancel packet format . . . 125
8.7 Control packet format . . . 126
8.8 Echo request and echo response packet format. . . . 127
8.9 Atomic Extended Transport Header (AtomicETH) . . . 128
8.10 Atomic ACK Extended Transport Header (AtomicAckETH) . . . 129
8.11 Invalidate Extended Transport Header (IETH). The R_Key identifies the memory region to be invalidated at the responder. This header follows the METH and precedes the payload. . . 130

List of Tables

3.1 Completion rules in ordered mode 39

6.1 Baseline Congestion Control Parameters 84

7.1 Multiplane topology wiring 105
7.2 Full multiport topology wiring 105
7.3 Partial multiport topology wiring 106

8.1 Opcodes 111
8.2 Opcodes that support zero byte payloads 115
8.3 Ack status codes 121
8.4 RNR backoff time encoding. 121
8.5 ACK/NAK Header Fields 122
8.6 Echo response error state codes 126

10.1 Configuration of a queue pair with Tags 155

Contributors

Meta Arvind Srinivasan, Neil Spring, William Chenkai Zhang, Zeeshan Atlaf Lokhandwala, Ashkan Nikravesh, Georgios Nikolaidis, Pankaj Kansal, Ved Kushwaha, Jae Won Soo, Anand Jayaraman, Rajiv Krishnamurthy, Stig Arne Olsen, Chidambaram Muthu, Kirtesh Patil, Wei Lin Guay, Magne Sandven, Brian Manula, Sandeep Nagaraja, Christopher Canel, Dag Moxnes, Evangelos Tasoulas, Prankur Gupta, Kaushik Ghosh

AMD Balakrishnan Raman, Pradeep Thangaraju, Gaurav Borker, Krishna Doddapaneni, Nataraj Batchu, Vijay Sampath, Vishwas Danivas

Part I

Introduction and Overview

Chapter 1

Introduction

Meta-RoCE is an RDMA transport protocol designed for hardware-offloaded operation over best-effort Ethernet networks with shallow-buffer switches. It targets large-scale AI training clusters, where thousands of accelerators exchange data across networks that span racks, buildings, and sometimes geographic locations. The protocol is designed to scale to hundreds of thousands of nodes and millions of network endpoints without compromising tail latency.

The goals of the Meta-RoCE transport are:

- High network efficiency at scale, sustaining throughput without head-of-line blocking.
- Resilience to network degradation, recovering from packet loss and partial failures without visible impact on application performance.
- Adaptability to evolving workloads, link speeds, and network topologies, with programmable congestion control parameters that can be tuned without hardware changes.
- Practical implementation on-chip alongside special-purpose accelerators, where power and area constrain NIC memory and logic resources.

1.1 Motivation

Traditional RDMA transports rely on a lossless network fabric maintained by Priority Flow Control (PFC). PFC prevents packet loss by pausing a sender when a downstream switch buffer fills; however, at scale, PFC creates several problems. Switch buffer space reserved for PFC headroom grows with link speed and distance, consuming memory that could otherwise absorb transient bursts. A single congested port can propagate pause frames upstream, stalling unrelated traffic in a phenomenon known as a pause storm. Because PFC operates per traffic class rather than per flow, a misbehaving or slow receiver can block all senders sharing that class.

Single-flow connections compound these issues. ECMP (Equal-Cost Multi-Path) routing hashes each connection to one network flow; when multiple connections hash to the same link, that link becomes a bottleneck while parallel links sit idle. The resulting hash collisions are unavoidable in large fabrics and worsen as the number of endpoints grows. Spreading a single large transfer across multiple network paths requires either switch-level packet spraying with switch support for reorder buffers, or endpoint-directed

multiflow, which traditional RDMA transports do not provide. Multi-port NICs connected to parallel network planes must also manage how traffic is distributed across ports, regardless of in-network spraying.

One approach to spreading traffic is QP scaling: creating many parallel connections between the same pair of endpoints. Each connection hashes to a potentially different ECMP flow, increasing path diversity and aggregate packets in flight. However, QP scaling trades one set of problems for another. Connection state multiplies with the number of QPs, consuming on-NIC memory that is constrained by power and area on accelerator-attached NICs. Congestion control fragments across many small windows, each learning the network state independently rather than sharing a consolidated view. The application or collective library must partition messages across connections to balance load, adding software complexity. Modern training workloads that combine data parallelism, pipeline parallelism, and tensor parallelism may generate dozens of concurrent communication streams between the same endpoint pair, further multiplying these costs.

Meta-RoCE addresses these limitations by consolidating multipath load balancing, congestion control, and stream multiplexing into a single reliable connection managed by the NIC transport layer.

1.2 Fabric Features

Meta-RoCE is designed to operate over standard Ethernet fabrics. The primary expectation from the network is Explicit Congestion Notification (ECN): switches mark packets using a WRED-like (Weighted Random Early Detection) ramp as queue depth increases, and the transport reacts to these marks to reduce its sending rate. ECN marking is widely available and is the foundation of Meta-RoCE congestion control.

Beyond ECN, Meta-RoCE relies on standard Ethernet features that are universally available: ECMP for flow diversity, and traffic classes for separating control and data traffic when desired. The protocol is compatible with a range of fabric designs, including scheduled fabrics, disaggregated deep-buffer fabrics, and shallow-buffer fabrics.

Several network features that other transport designs depend upon are optional or unnecessary for Meta-RoCE:

PFC is not required. Meta-RoCE treats the network as lossy and recovers from packet loss through selective acknowledgment and retransmission. If the fabric supports PFC, Meta-RoCE can use the PFC traffic class to provide a more reliable channel for acknowledgments, but does not depend on it for data traffic.

Switch-based packet spraying is not required. Some fabrics offer switch-level packet spraying across equal-cost links. Meta-RoCE can benefit from this when available, but does not require it; the NIC manages its own multipath load balancing by directing packets across flows with different entropy values.

Packet trimming is not used. Some switch designs trim data from packets that would otherwise be dropped, forwarding only the header so that loss is detected quickly. Meta-RoCE does not rely on this capability.

In-network telemetry is not used. Fine-grained queue depth or path latency information from switches can inform transport decisions, but Meta-RoCE operates without it, relying instead on end-to-end measurements and ECN signals.

Credit-based flow control is not required. Link-level credit schemes prevent loss by requiring a receiver to grant credits before a sender transmits. Meta-RoCE does not use link-level credits; its receiver-driven rate limiting operates at the transport layer.

1.3 Design Overview

Following end-to-end design, Meta-RoCE places transport functions—reliability, ordering, congestion response, and flow management—in the endpoints rather than depending on per-hop network mechanisms. The transport is organized around four mechanisms, illustrated in Figure 1.1.

Connection multiplexing through Tags. A single Meta-RoCE connection (QP) can carry multiple independent message streams, called Tags. Each Tag has its own sequence number space and completion ordering, so a stall on one stream—due to packet loss or a receiver that is not ready—does not block progress on others. Tags correspond to the concurrent communicators or collective operations that modern training workloads generate between the same endpoint pair.

Explicit multipath transport. The sender maintains multiple flows within a connection, each with its own sliding window and selective acknowledgment state. By controlling the entropy value in each packet's encapsulation header, the NIC directs traffic across different ECMP routes without relying on switch-level spraying. Flow state is tracked with fixed-size bitmaps, bounding memory requirements regardless of how many flows are active. Because each flow is tracked independently, silence on a flow is distinguishable from silence on the connection as a whole: a flow that stops producing acknowledgments can be declared failed and its traffic moved to surviving flows, whereas oblivious packet spraying cannot distinguish a failed flow from transient congestion. When the bandwidth-delay product of the network grows—due to higher link speeds or longer distances—the sender scales by adding flows rather than enlarging per-flow windows. This keeps each flow a fixed-size resource and spreads traffic across more of the available fabric, improving load balance.

Relaxed-order data delivery. Data packets can arrive and be written to memory out of order; completions are delivered to the application in order. Every data packet carries its offset within the message, so data can be placed at its final memory location as it arrives. For RDMA Write operations, each packet additionally carries the destination memory address, length, and remote key; for Send operations, the matching receive work request provides the address and length. This eliminates the need for large on-NIC reorder buffers whose size would otherwise grow with the bandwidth-delay product of the network.

Congestion control with receiver-driven rate limiting. A baseline AIMD (Additive Increase, Multiplicative Decrease) algorithm adjusts per-flow windows in response to ECN marks and packet loss. In addition, the receiver communicates a target rate to each sender, allowing fast convergence during incast without relying on PFC. A programmable interface allows the congestion control parameters to be tuned or replaced to suit specific network conditions.

1.4 Congestion Control and Loss Recovery

Meta-RoCE congestion control and loss recovery are designed to work together to keep network queues small and avoid reliance on PFC.

The sender adjusts its sending rate using a window-based algorithm that responds to ECN marks from the network. Because the sender spreads packets across multiple flows with different entropy values, what would otherwise be a single large flow (an "elephant flow" with low entropy) becomes many smaller flows that load-balance across the fabric. Per-flow windows allow the sender to react to congestion on individual network paths without throttling the entire connection. In addition, the receiver communicates a target rate to each sender based on how much inbound bandwidth is available, allowing senders to converge quickly during incast rather than discovering the limit through repeated ECN marks or packet loss.

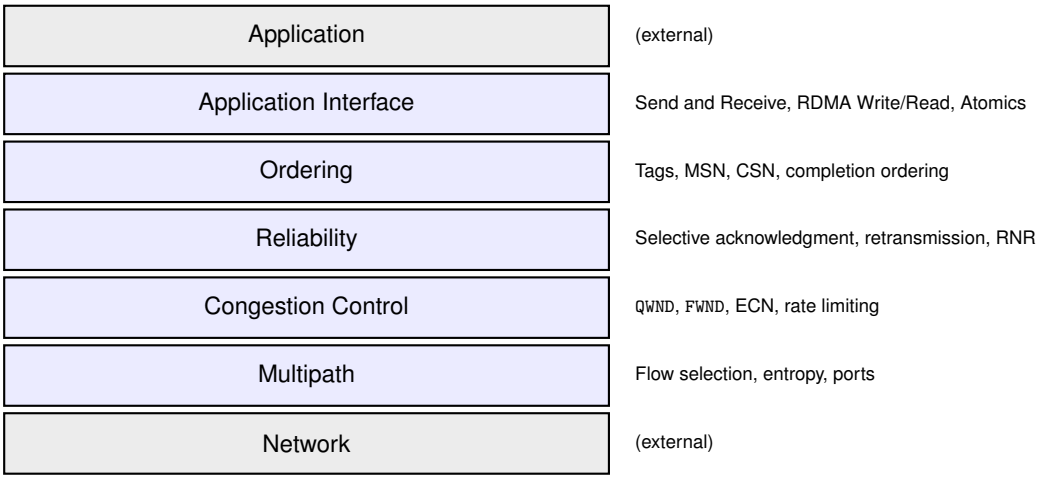


Figure 1.1: Meta-RoCE protocol layers. The application interface exposes operations to the application. Ordering assigns sequence numbers and tracks completion. Reliability provides acknowledgment and retransmission. Congestion control manages per-QP and per-flow windows. Multipath distributes packets across flows with different entropy values. The network layer supports encapsulation of Meta-RoCE packets to traverse a network.

Meta-RoCE treats the network as lossy and recovers from packet loss through selective acknowledgment. The receiver reports which packets have arrived using a per-flow bitmap, and the sender retransmits only the missing packets. Because data can be delivered out of order and written directly to its final memory location, the receiver does not need to stall waiting for earlier packets before processing later ones. This avoids head-of-line blocking at the NIC and eliminates the need for PFC to prevent loss in the first place.

1.5 Supported Operations

The Meta-RoCE transport supports the following operations:

1. SEND and corresponding RECV, with and without immediate, with and without Tags.

2. RDMA Write, with and without immediate

3. RDMA Read, with and without immediate.

4. Atomics: compare-and-swap and fetch-and-add, with opcodes reserved for bitwise and more complex operations (Table 8.1).

RDMA Read and Write may support low-latency memory operations; although this may use a different WQE format, no alternate message types are expected.

1.6 Document Organization

The remainder of this document is organized as follows. Chapter 2 defines the sequence number hierarchy that enables out-of-order data delivery with in-order completion semantics. Chapter 3 describes how message completions are ordered within and across tags. Chapter 4 covers acknowledgment, selective retransmission, and the mCRC integrity check. Chapter 5 specifies error detection and reporting. Chapter 6 details congestion control, including the baseline AIMD algorithm, receiver-driven rate limiting, and the programmable extension interface. Chapter 7 describes multipath operation and multi-port failover. Chapter 8 defines packet header formats. Chapter 9 summarizes features and configurability requirements, and Chapter 10 specifies integration with the verbs library.

Part II

Semantics and Ordering

Chapter 2

Sequence Numbers

2.1 Overview

Sequence numbers in Meta-RoCE include:

Message Sequence Number (MSN) - incrementing on every message initiated within a QP+Tag, providing a unique identifier to each message and enabling related messages to be ordered. The MSN of a read response or atomic response (AtomicAck) is the MSN of the request. The MSN can wrap, and is negotiated at QP setup. (Section 2.2)

Packet Offset Sequence Number (POSN) - incrementing on every packet within a message, indicating the offset into a buffer where data should be delivered. The POSN starts at zero and cannot wrap, leading to a maximum message size. (Section 2.3)

Completion Sequence Number (CSN) - incrementing on every message within a QP+Tag that requires a completion to notify the responder, permits matching send messages with the in-posted-order corresponding receive buffer. The CSN can wrap, and is negotiated at QP setup. (Section 2.4)

Flow Sequence Number (FSN) - incrementing on every packet within a flow, and is the unit that is acknowledged and retransmitted if necessary. It is named for the per-flow space it occupies rather than the packet it counts, as each flow has an independent sequence. The FSN starts at zero and can wrap. Until it wraps, a *(Flow,FSN)* identifies one *(Tag,Opcode,MSN,POSN)*, so a retransmission repeats the FSN it was first sent with. Data is outstanding under exactly one *(Flow,FSN)* at a time. After an RNR rejects it, the same data is sent under a new *(Flow,FSN)*, and the original FSN is not reassigned. (Section 2.5)

Together, these sequence numbers allow data to be delivered even when prior packets have not yet been received. Meta-RoCE can therefore send messages across multiple flows and use networks configured for best-effort delivery where packets may be discarded.

Figure 2.1 describes the relationship between sequence numbers.

The unidirectional scope of flow sequence numbers is a *flow*, identified by the Flow ID. The scope of message sequence numbers and completion sequence numbers is the *QP+Tag*, which represents the

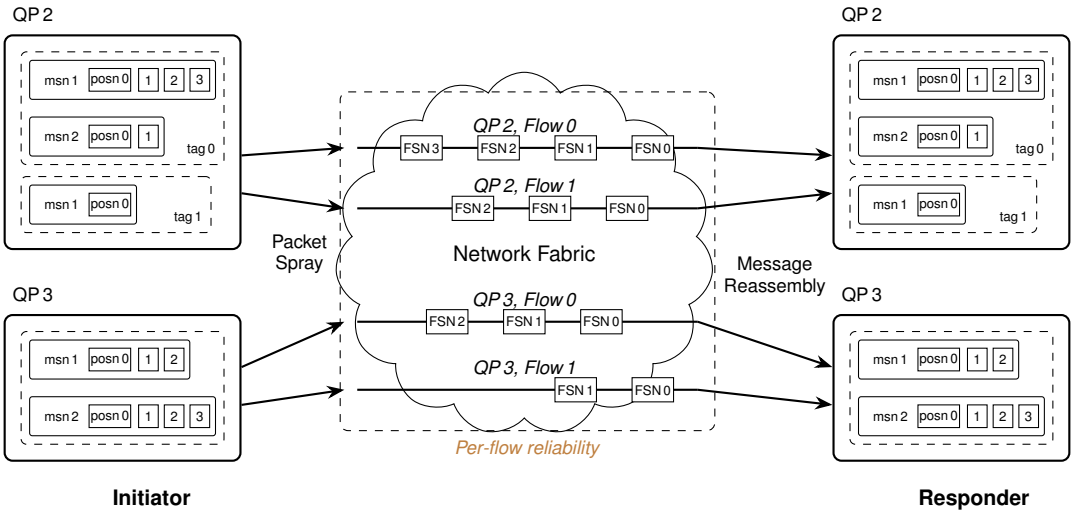


Figure 2.1: Sequence numbers overview: POSNs are scoped within messages (MSNs), which are scoped within a QP and Tag. FSNs are scoped per flow; each QP has its own independent set of flows.

communicator within a queue pair. Where tags are not supported, the scope of message sequence numbers and completion sequence numbers is the same.

In contrast to RoCE, there is no QP-level packet sequence number, and packets within messages are not identified as first/middle/last.

2.2 Message Sequence Number

The message sequence number (MSN) provides a unique identifier for each message that also exposes the order in which messages were posted for this QP and, where supported, Tag. The message sequence number of a read request or an atomic operation is used in the response. The MSN echoed in a read response is the MSN from the read request, and may numerically coincide with an MSN assigned by the responder to its own outbound messages; these are independent sequences.

An initial MSN for the QP and Tag may be negotiated; examples in this document assume zero is used as the initial MSN. After initialization of the QP, MSNs are assigned sequentially as described here.

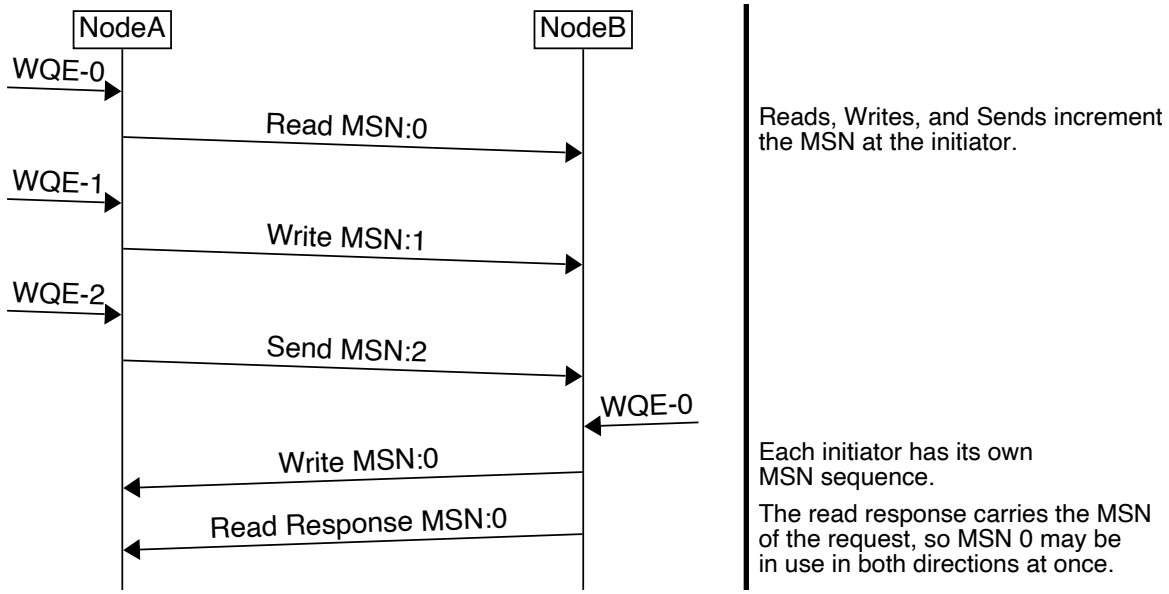


Figure 2.2: Message Sequence Numbers

2.3 Packet Offset Sequence Number

The packet offset sequence number (POSN) is a sequence number within the message. It starts from zero in every message, and indexes in payload-sized increments. Each packet within a message must be full sized, except the last when a message has a size that is not a multiple of the payload size. With a 16-bit POSN and 4096-byte payload, a message of 2^{28} bytes, 256 MB, may be sent. Larger messages are not supported and must be split.

The following diagram shows the transmission of two 16KB messages, each sent as four packets having 4KB payload each.

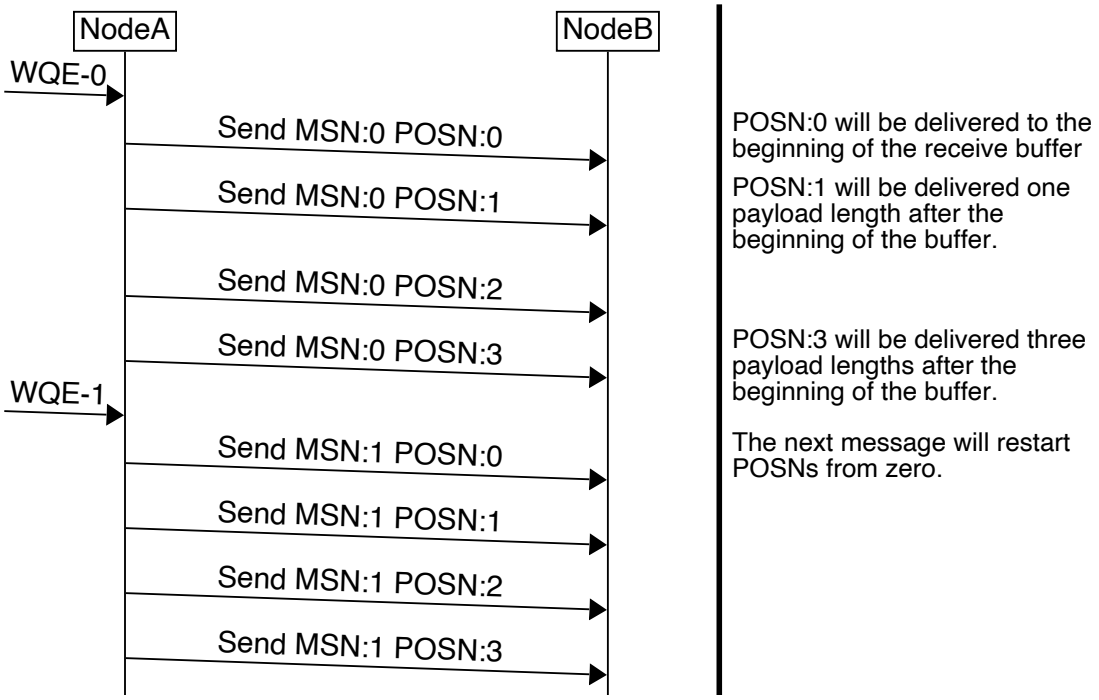


Figure 2.3: Packet Offset Sequence Numbers

Once transmitted, a data packet identified by a POSN within the context of a *(QP, Tag, MSN)* is bound to the FSN scoped within the context of a *(QP, Flow)*. (This binding can be removed via B-RNR processing, but is otherwise immutable.) A receiver must prevent duplicate delivery of the same data by preventing delivery of spurious retransmissions at the per-flow FSN level, and may additionally prevent delivery of duplicate data at the per-message POSN level. A receiver is not expected to identify an incorrectly implemented sender that uses the same *(QP, Tag, MSN, POSN)* on more than one *(Flow, FSN)* without explicit B-RNR. No explicit error reporting for duplicate data is expected, though it is likely that an inconsistency (e.g., too many POSNs received for a given message) would eventually be discovered.

2.3.1 Endpoints must agree on MTU

Meta-RoCE depends on agreement between both endpoints on the MTU to be used for a connection; the receiver must be able to use the MTU to interpret the Packet Offset Sequence Number as an offset into the message. If the transmitter were to send with a smaller MTU than the receiver expects, the receiver could place data incorrectly (for POSN values that would be valid at larger MTU) and need to validate that POSN values are in range to avoid over-running a receive buffer. It is expected that a NIC will provide a link MTU of 4200 bytes or greater as default, which is sufficient for a QP MTU of 4096 bytes plus protocol headers. Both endpoints must agree before posting WQEs, and the MTU for a QP cannot be changed after posting the first WQE on that QP.

2.4 Completion Sequence Number

The completion sequence number is defined in greater detail in Section 3.4, and represents the expected sequence of the receive WQE (RWQE) at the responder, for operations that require an RWQE such as write-with-immediate and send. The CSN is defined for such operations and should be ignored for messages of other types. Figure 2.4 shows how the CSN advances independently of the MSN, tracking only those messages that consume a receive WQE.

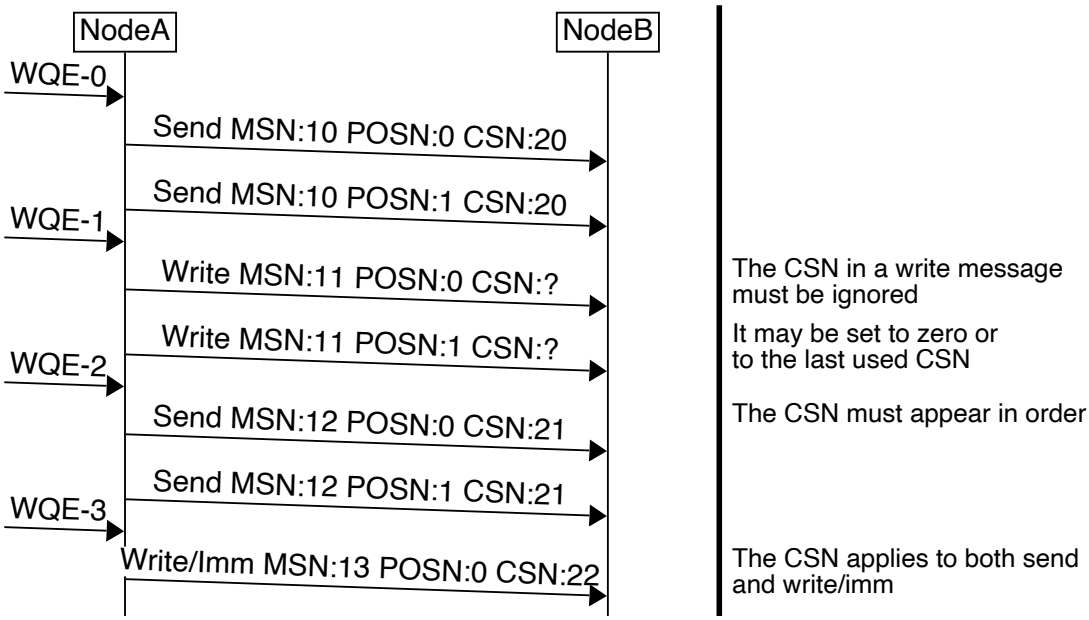


Figure 2.4: Completion Sequence Numbers

2.5 Flow Sequence Number

The flow sequence number provides a sequential number of each packet sent on a flow, independent of which message or packet within the message is being carried.

The use of the flow sequence number is defined in greater detail in Section 4. Within a flow, the flow sequence number allows identification of out-of-order receipt leading to per-flow retransmission.

In the example of Figure 2.5, two flows send the two 16KB messages from Figure 2.3. Per-flow acknowledgments identify the FSNs that have been delivered in order.

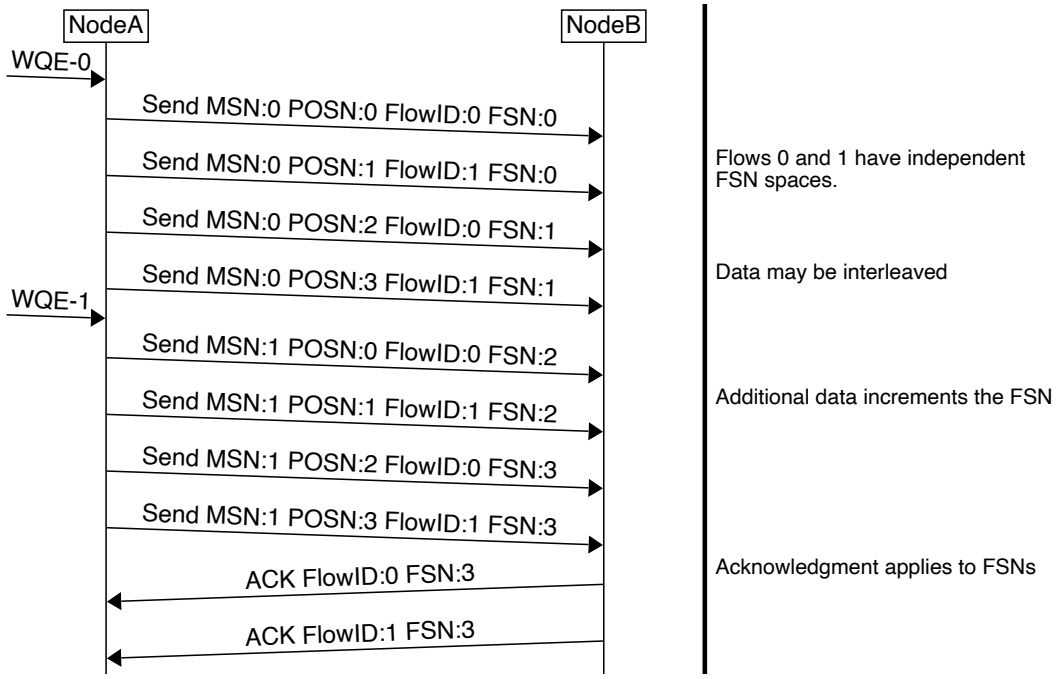


Figure 2.5: Flow Sequence Numbers

The flow sequence number, within its scope of flow id and QP number, uniquely identifies data, until the FSN wraps. Data, identified by *(QP,Tag,Opcode,MSN,POSN)*, must be outstanding under only one *(Flow,FSN)* at a time, so a retransmission repeats the FSN it was first sent with. By checking that each FSN is delivered exactly once, additional protection against duplicate delivery at the message level is not required. After an RNR, the same data is sent under a new *(Flow,FSN)*. Because the receiver rejected the data without delivering it, releasing its claim on the original FSN allows the FSN space to advance for other Tags. The original FSN is not reassigned.

Chapter 3

Ordering

This chapter specifies the ordering constraints on data delivery to memory, atomic execution, and completion posting at a Meta-RoCE endpoint. To enable Meta-RoCE network interfaces that have no reorder buffer, it is expected that data, once sent, can always be delivered directly to memory. To support the strategies used by collective communication libraries to reduce the load of processing completion events, atomic execution and receive completions must be ordered relative to the delivery of prior messages in the same message stream.

The initiator can always delay starting to send a message into the network, waiting until all prior messages have been completely delivered. However, enforcing ordering in this manner sacrifices performance. In contrast, this section deals with ordering the delivery of message completions where multiple messages are in progress at the same time, potentially with data being delivered out of order.

3.1 Terminology

The following terminology applies to ordering of messages and data delivery.

Message - The operation (write, send, etc.) that may comprise several packets, spread across multiple flows, where the contents of those packets may be delivered into memory in any order.

Tag - Operations in different tags within a queue pair are ordered independently; the ordered, first-in-first-out "Queue" aspect of the traditional queue pair applies only within the scope of a specific tag. Where tags are supported, the work request and completion queues are determined by the tag. Where tags are not supported, assume the tag is zero.

Flow - A per-QP transport resource identified by a Flow ID, with its own FSN space, congestion window (`FWND`), and RTT state. Tags and flows are orthogonal: a tag's packets scatter across flows.

Delivered Message - A message is "delivered" from the perspective of the receiver when all data within the message has been written to memory. A message is "delivered" from the perspective of the sender when all data packets within the message have been acknowledged. The receiver must not acknowledge a packet unless the data is guaranteed to reach memory before the acknowledgment arrives at the sender; therefore, delivery at the receiver always precedes delivery at the sender. Being "delivered" is a precondition of being "complete."

Complete Message - A delivered message is also "complete" depending on the ordering configuration of the queue pair; when the queue pair is ordered, the message is complete only when all prior messages have been completed.

Completion - Used interchangeably with CQE (Completion Queue Entry) to indicate informing the job that a message finished, when those operations require notification. A CQE is posted only when it matches a posted WQE (Work Queue Element). This section addresses when a CQE should be posted; necessarily after all packets have been received and their data delivered to memory, but optionally also after other, "prior," messages have been completed.

Virtual completion - Internally to the Meta-RoCE implementation, the completion state of an incoming message must be maintained even for those message types that do not post a completion to the job. A virtual completion exists when the message is complete. The message becomes complete differently for reads and writes. For the sender in a read, all data must be acknowledged. For the receiver of a write, all data must be received. For an atomic, the message is complete when executed, which does not occur until after all prior messages are complete.

Receive-WQE (RWQE) - A receive WQE that is posted in preparation for receiving a type of message that will generate a completion. The RWQE specifies the buffer to be used for incoming data. An RWQE indicates whether the received message will include immediate data to be included in the completion.

Virtual Receive-WQE (vRWQE) - The Meta-RoCE internal state that is complementary to a RWQE, but allocated upon receipt of message data from the remote rather than upon posting of a WQE.

Receive-WQE-Consuming Operation - Write-with-immediate and Send operations require that the receiver have posted a RWQE to receive the data. All operations post completions at the initiator; these operations also post completions at the responder.

non-Receive-WQE-Consuming Operation - Write, Read, and atomics expect to operate without a corresponding WQE (or CQE) at the other side. These operations generate virtual completions at the responder and completions at the initiator.

Unordered mode - A mode in which completions do not wait for prior messages to complete.

Fence, or Execution Fence - A method at the initiator that prevents processing of a WQE and those that follow until prior WQEs in the same queue have completed. This enforces an ordering over operations at the job level, at the likely cost of a round-trip-time to ensure that prior work is complete. The fence indicator can be set on an atomic operation that follows a read request to ensure that the atomic operation is not issued until the read is complete.

Completion Fence - A per-WQE bit ensures that this specific WQE is completed only when all prior WQEs in the queue have completed. The completion fence allows a QP to mix ordered and unordered operations. The completion fence can apply at both initiator (WQE) and responder (RWQE). Unlike the execution fence, it holds only the WQE with the bit set; WQEs posted after it may still complete out of order.

3.2 Supported Message Sequences

Meta-RoCE must address the mismatch between potentially reordered packet delivery and the RoCEv2 programming model where completions are delivered in order and imply that prior messages have been delivered. The goal is to gain maximum performance and flexibility from out-of-order delivery, while remaining easy to use with predictable behavior even when delivery is out of order. This involves supporting the following modes.

3.2.1 Writes with Write/immediate

The following diagram shows the ordering pattern for RDMA writes mixed with RDMA writes-with-immediate. The two spaces of message sequence number, MSN and CSN, are defined in Sections 2.2 and 2.4. The responder must post an RWQE for the writes with immediate, but not for the bare writes. However, the write-with-immediate is not expected to complete until all of the writes that have been issued by the initiator have been delivered.

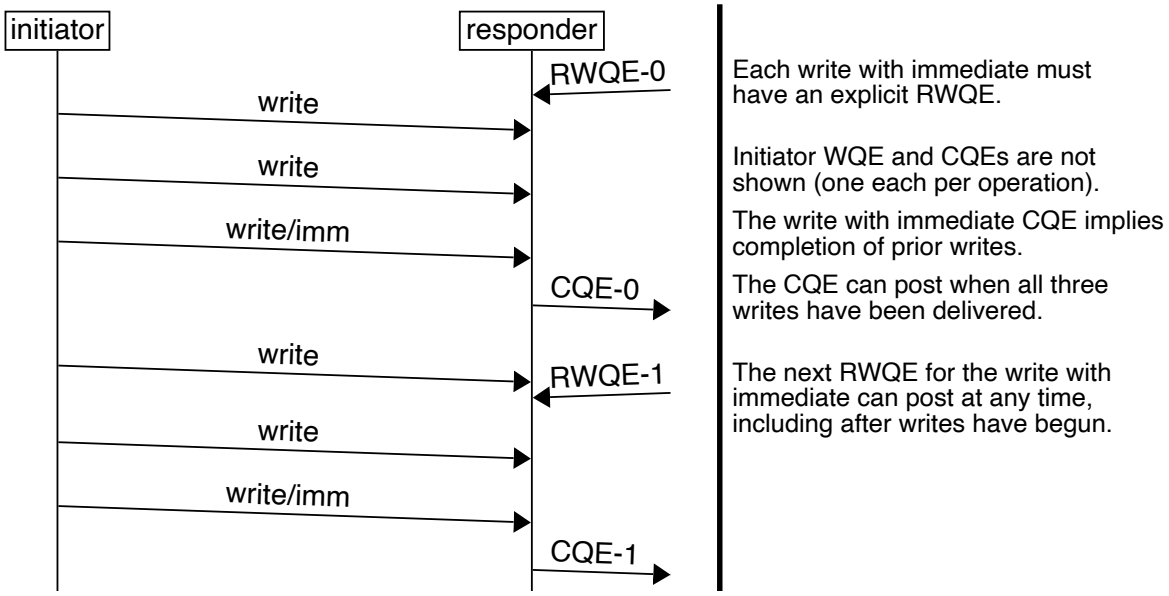


Figure 3.1: Supported message sequence: Writes with Write/Immediate

Mixed writes and sends are equivalent; the completion of a send indicates complete delivery of prior writes, as shown in Figure 3.2.

3.2.2 Writes with Atomics

In cases where the recipient is configured to check the count of received writes, where each writer increments the count atomically, it is necessary to execute the atomic increment only after all prior writes.

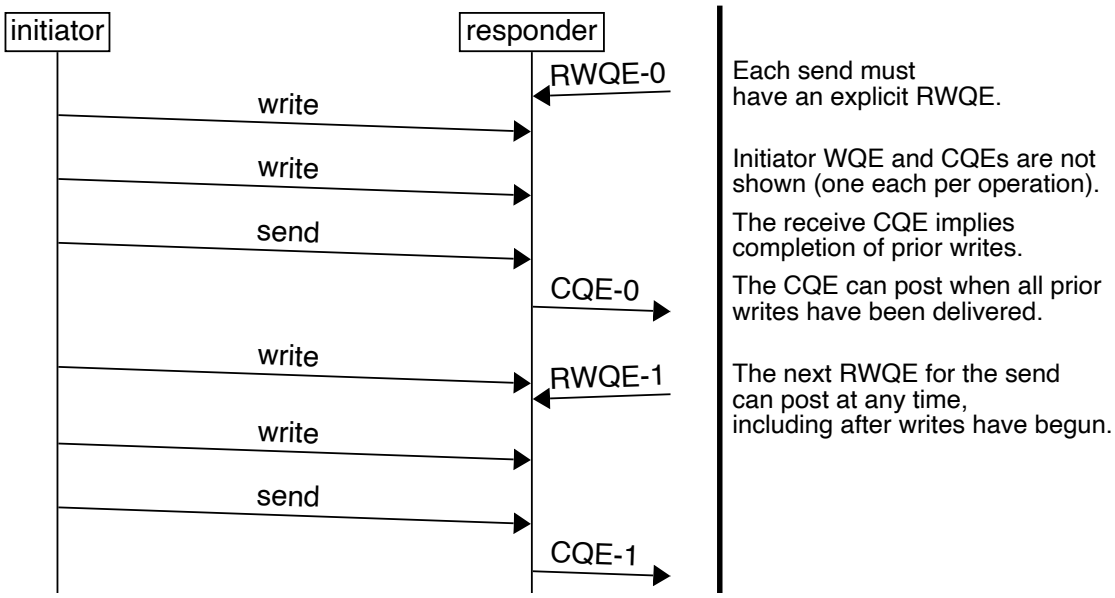


Figure 3.2: Supported message sequence: Writes with Send

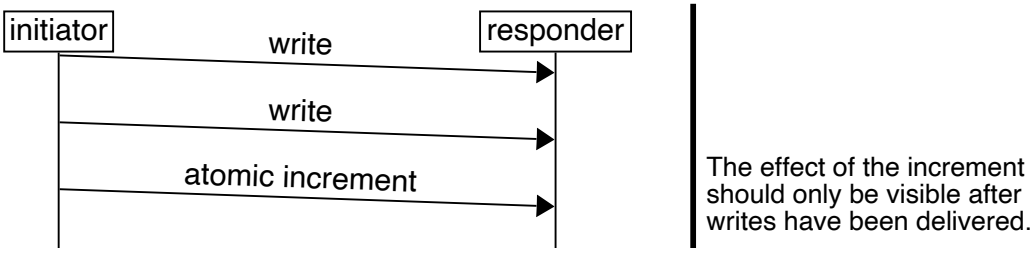


Figure 3.3: Supported message sequence: Writes with Atomics

3.2.3 Concurrent Reads

Two reads may be issued concurrently and their requests handled in the order in which they arrive. Read completions should be delivered in order unless the QP is in unordered mode. Figure 3.4 shows two concurrent reads whose responses arrive out of order but are completed in the original request sequence.

3.2.4 Pipelined Sends

Meta-RoCE allows receive messages to complete out of order when the QP is in an unordered mode, but will ensure that the first send matches the first receive, as shown in the following diagram. That is, on a private receive queue, the first message sent will be delivered into the first receive buffer posted, but in unordered mode, if the second message is delivered first, it may also be completed first. Where the receive

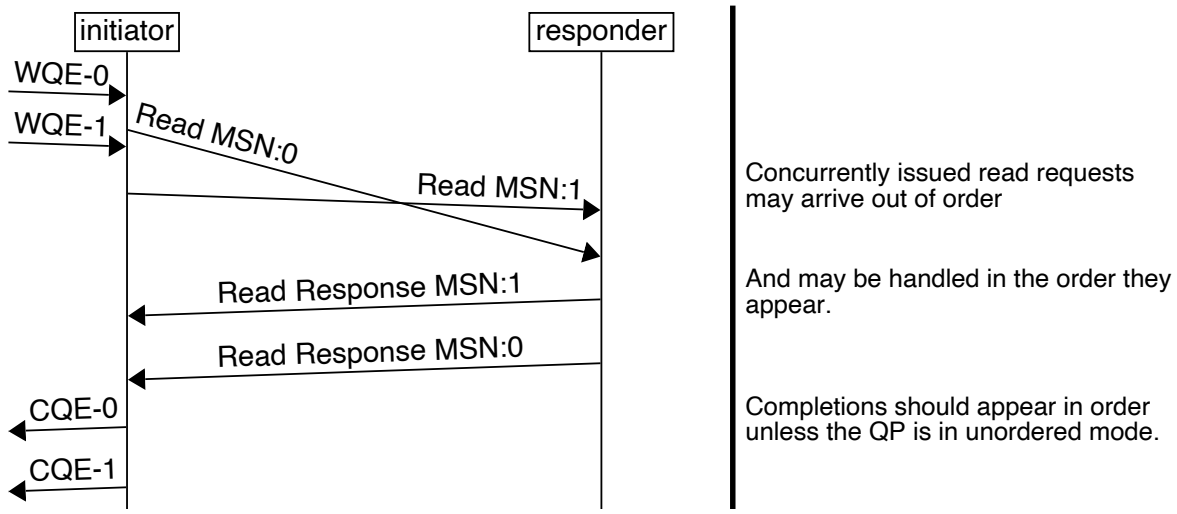


Figure 3.4: Supported message sequence: Concurrent Reads

queue is shared, buffers are taken in arrival order instead (Section 3.4.1).

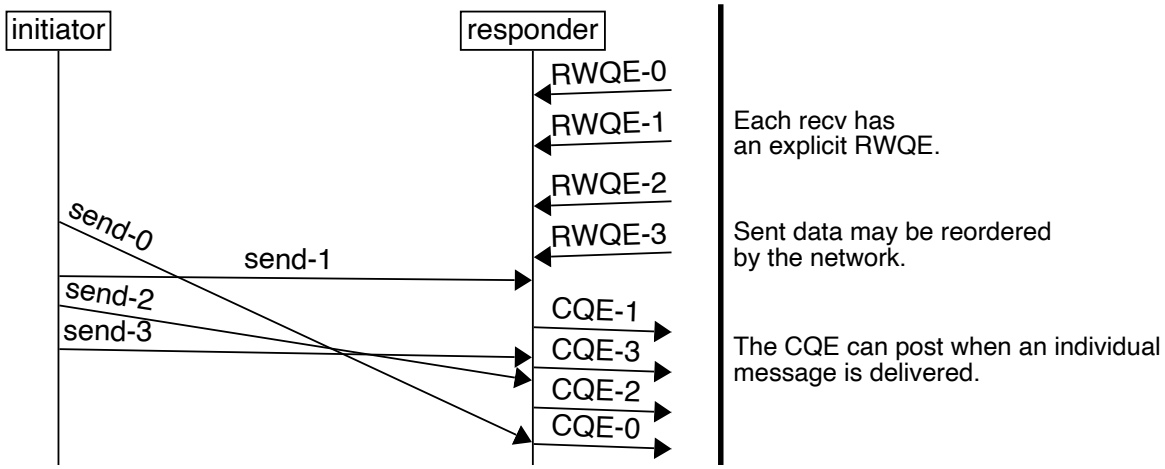


Figure 3.5: Supported message sequence: Receives completed out of order in unordered completion mode.

3.2.5 Specific Undefined Message Sequences

Some sequences of operations issued by the sender, if multiple operations are “in flight” at a time, will have results that depend on the order in which packets are received and thus delivered into memory.

The following sequences are not expected in practice and do not need well-defined ordering; they can be addressed with an execution fence that ensures operations are not concurrent.

- *Concurrently issued writes to the same address in memory will have undefined behavior.* Where in RoCE, the ordering of writes and in-order delivery of packets would likely leave the result of the last write, in Meta-RoCE, this is not defined. The delivery order of packets in different messages may be interleaved, leaving the destination memory buffer with some data from one message and some data from the other.
- *Concurrently issued writes and reads to the same address in memory will have undefined behavior.* If issued simultaneously, it is likely that the small read that required no initiator-side DMA could reach the responder first, but Meta-RoCE provides no guarantee that the read would be processed before or after the write.
- *The use of an atomic that should execute before concurrently issued data access (read or write).* The atomic will execute after prior operations complete, but the read or write will begin as soon as it is received. Atomic-before-access seems unlikely, as the result of the atomic operation did not reach the initiator first so that it could determine whether it executed successfully. Atomic-after-completion is a more likely pattern and is supported.

3.3 Completion Ordering Modes

Meta-RoCE supports two modes for completion ordering at both sender (initiator) and receiver (responder). Within a tag, completions may be either ordered or unordered. Unordered completions permit eager-as-possible pipelining of chunked operations. At the initiator, ensuring completions are posted in a defined order is trivial, as the initiator knows the order in which WQEs are posted and there are no virtual completions. At the responder, the mapping between completions and the preceding virtual completion must be learned from the sender. Restated, that a write/immediate must follow a prior write is explicit in the sequence number in the protocol messages.

It is not advised to mix virtual completion operations (e.g., write) with explicit completion operations (e.g., send) in unordered mode, as it is likely the case that the job expects the explicit completion to indicate that the other operation is complete. A Meta-RoCE implementation may reconfigure an unordered completion QP+Tag into ordered mode upon observing non-Receive-WQE-Consuming Operations.

3.3.1 Ordered Completion Mode

Ordered completion mode maintains the “queue pair” ordering, in which all operations complete in order: the first message to have a WQE posted is the first message to be sent, and will be the first message to have a CQE posted. This ordering must not depend on the order in which message packets are delivered (at the recipient) or acknowledged (at the sender).

A read consumes no receive work queue entry, so it does not appear in the responder’s completion order and does not hold later completions. Read with immediate does consume one, and its completion tells the responder’s application that the read was served, so it waits for the initiator’s acknowledgment.

Ordered *completion* mode does not order *writes* to memory; polling the last address of a write is *never* permitted as a means to infer that any other information has also been written or operations have been completed.

Operation	Has RWQE	Completes at the responder when
Send	Yes	All data delivered; all prior messages complete.
Write	No	All data delivered
Write/imm	Yes	All data delivered; all prior messages complete.
Read	No	All request packets received
Read/imm	Yes	All data acknowledged by initiator; all prior messages complete
Atomic	No	Executed and is complete once all prior messages are complete

Table 3.1: Operations complete at the responder in ordered mode when prior operations have completed and the message payload is completely delivered. Atomics are special in that they are executed (delivered) only when prior messages are complete, permitting atomic operations to be used as an unlock or message-complete signal.

Atomics in ordered completion mode are queued until they can be completed. That is, an atomic operation (read-modify-write) will not be executed until it can be completed (all prior operations have been completed). Non-atomic operations (e.g., writes) that might occur after the atomic operation will not be held until the atomic executes. Meta-RoCE is always allowed to deliver any data that it receives, only completions are ordered.

3.3.2 Unordered Completion Mode

In unordered completion mode, sends, receives, writes with immediate, and atomics may be completed in the order the data are received or acknowledged. Should an initiator wish to create an order (e.g., for an atomic) where one operation may only complete after another, an explicit fence is required.

In unordered completion mode, operations that depend on a specific execution order—such as using an atomic to indicate that a prior write has completed—require an explicit execution fence between them. Even though neither of these operations (write, atomic) consumes a WQE at the responder or generates a completion for the job to read, the atomic must be fenced until the write is finished when seeking ordered-mode behavior on an unordered-mode queue pair.

3.3.3 Mixing Ordered and Unordered WQE Completions (optional)

To implement a model in which individual WQEs require in-order completions while others on the same queue pair (plus tag) do not involves storing information about each out-of-order message. Tracking which messages have been completed in order is comparable in design to the same tracking of out-of-order packet delivery.

To accomplish mixed ordered and unordered completions, the “completion fence” bit is used, where the RWQE may be configured to prevent its completion until all prior RWQEs (both virtual for an incoming write or read request, and actual with a corresponding WQE) have completed, or may be configured to complete out of sequence.

3.4 In-Protocol Message Sequencing

To best explain the semantics being provided, this section describes the underlying protocol-level implementation.

Each data packet carries two message-level sequence numbers:

1. The message sequence number (MSN) is the unique identifier of the message, starting from zero unless otherwise negotiated, referring to the order in which WQEs were posted at the initiator.
2. The completion sequence number (CSN) is similar, but reflects the order in which WQEs should have been posted at the receiver. The CSN is meaningful only for WQEs that have a corresponding receive WQE.

On a private receive queue, the CSN ensures that the receiver can match sent data to the correct, matching receive buffer (Section 3.4.1 covers a shared receive queue), while the MSN ensures that the receiver knows whether there were additional messages to deliver completely before posting the completion or executing an atomic. Matching sends to receives ensures that, if a sender breaks a message into four chunks, the first chunk to be sent will match the first posted receive.

For example, in a QP that exclusively uses SEND/RECV, the MSN will always equal the CSN. Conversely, in a QP that exclusively uses writes and atomics, which do not have a corresponding WQE at the receiver, the CSN is undefined and should be ignored.

3.4.1 Shared receive queues

The matching described so far is between a message and a buffer on a private receive queue. Support for shared receive queues is optional, and the choice is made per receive queue: the tags of one queue pair may draw from different shared receive queues, or from private ones, in any combination.

Where a receive queue is shared, a buffer is taken when a message arrives, so two messages that arrive out of order take their buffers in the order they arrived. Reserving a buffer against each tag's next CSN would defeat the sharing. The CSN continues to bind every packet of a message to a single buffer, rather than to select which posted buffer that is.

Completions are posted in the order of Section 3.3.1, so a message that took a later-posted buffer still completes in its turn. A write, read, or atomic takes no buffer and advances no CSN, so where sends are mixed with them only the sends draw from the shared queue. An empty shared receive queue is a missing RWQE for every receive queue drawing from it, and is answered as Section 5.5 describes.

An implementation must not deadlock a shared receive queue where messages arrive out of order and the application keeps a bounded pool of receives, replenishing them as they complete. It may avoid the deadlock, admitting a message only where its CSN is within the queue's length of the next expected, so that with three receives posted and CSN 7 next expected, CSN 9 is admitted and CSN 10 is not. It may instead reserve against it, taking the receives for the skipped CSNs from the queue as it matches the message and holding them privately for those still to arrive.

3.4.2 Applied examples

The three examples from Section 3.2 are reprised below, annotated with MSN and CSN.

Writes with Write/immediate

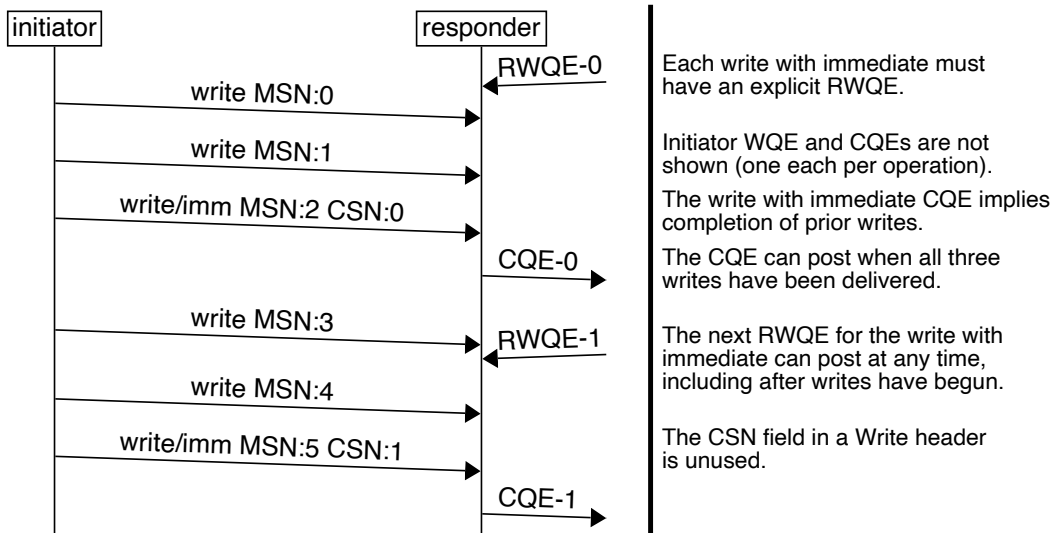


Figure 3.6: Completion sequence: Writes with Write/Immediate

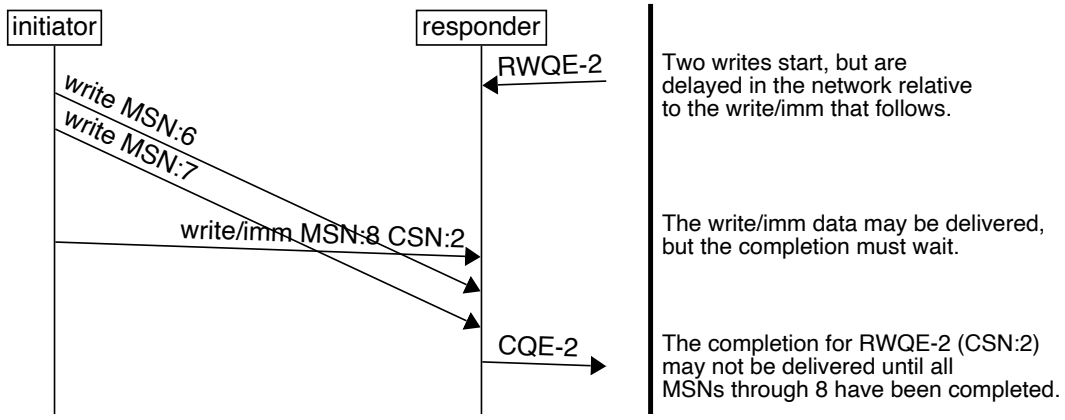


Figure 3.7: Completion sequence: Writes with Write/Immediate, with in-network reordering

Writes with Atomics

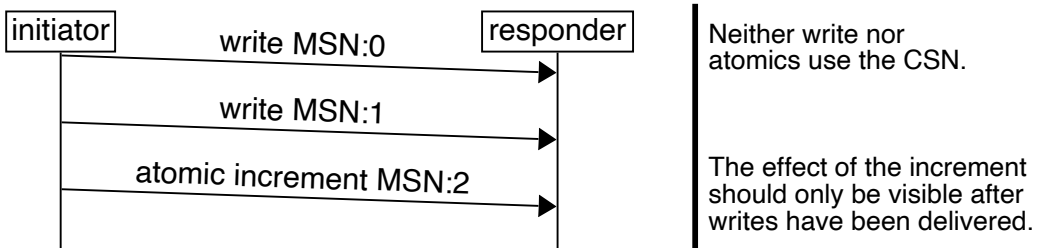


Figure 3.8: Completion sequence: Writes with Atomics

Pipelined Sends

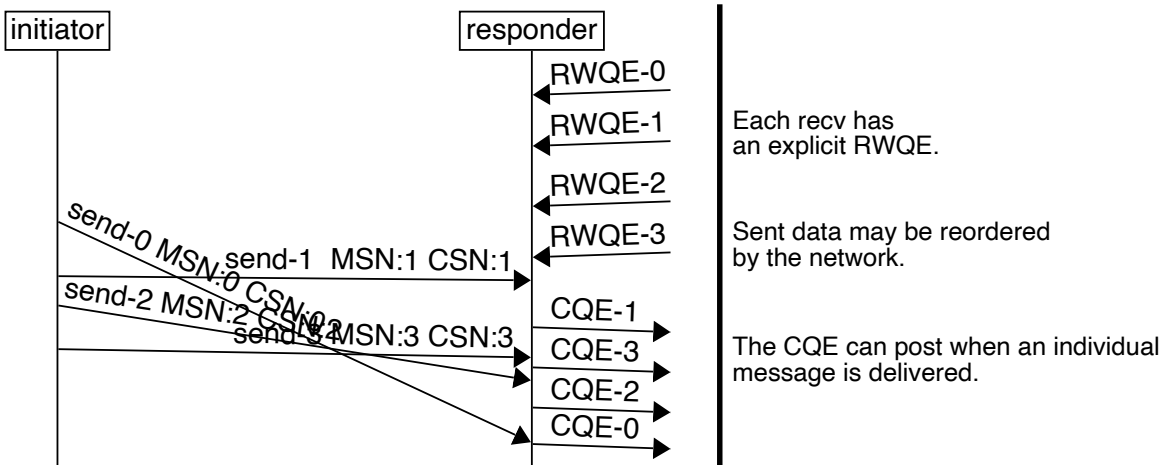


Figure 3.9: Completion sequence: Receives completed out of order

3.4.3 Atomic ordering

The execution of the atomic operation is its delivery; since this execution implies the completion of prior operations, it must not execute until prior vRWQEs are complete. An application that needs a write or read to wait for a preceding atomic sets the fence indicator on that work request. Where an atomic precedes another operation, it is expected either to decide whether that operation executes, as when acquiring a lock, or to be unrelated. An application that needs the decision waits for the result before posting the operation, and unrelated operations need no ordering. A fence behind an atomic therefore has little use, and the protocol does not optimize for it.

Loads and stores are ordered as atomics; executed only when all prior messages have been completed or executed.

3.4.4 State required

Each QP and Tag maintains the following state, separated into two groups by when the state is updated.

State altered on WQE posting

Initiator, per QP+Tag: Next Transmit MSN - Each WQE requires an MSN; this stores the message sequence number for the next WQE on this QP+Tag, and is incremented with each WQE. Initial value: negotiated initial SQ MSN.

Initiator, per QP+Tag: Next Transmit CSN - Completion-generating WQEs also require a CSN; this stores the next completion sequence number for the next completion-generating WQE on this QP+Tag. It is incremented with each completion generating WQE. Initial value: negotiated initial SQ CSN.

Responder, per QP+Tag: Next Receive CSN - RWQEs require a CSN; this stores the next completion sequence number that will be assigned to the next completion-generating WQE on this QP+Tag. Incremented with each new RWQE posted. Initial value: negotiated initial RQ CSN.

State altered on WQE and vWQE completion

Initiator and Responder, per QP+Tag: Cumulative-Responder-Delivered-MSN (CDMSN) - This MSN is the last to have completed in order at the responder. It is updated as in-sequence messages are delivered and optionally also completed. Initial value: one less than the negotiated initial MSN.

Initiator and Responder, per QP+Tag: Out-of-sequence completed MSNs - May be maintained as a bitset of MSNs that were completed out of order; one set per direction. It is updated as out-of-sequence messages are delivered or completed, and allows minimizing the state required to track out-of-sequence completion. Initial value: empty.

Initiator and Responder, per QP+Tag: Ordered-mode, out-of-sequence CQEs - CQEs that cannot be posted wait until the next in-sequence completed MSN equals the MSN of this message. An implementation may store them or defer generating them.

Each WQE, RWQE, and vRWQE includes the associated MSN and (except for vRWQE) CSN.

Chapter 4

Reliable Delivery

This chapter addresses the reliable delivery of Meta-RoCE RDMA data on a network that supports best-effort delivery, including round trip time estimation, acknowledgment generation, and recovery when the receiver has not posted a receive buffer.

The Meta-RoCE transport reliability layer uses a fungible packet tracking resource. Sender and receiver track packets that may be delivered and acknowledged out of order by the network, and the number of packets that are allowed in the network at a time is governed by a set of per-flow windows, the size of which is controlled as described in Chapter 6.

Where this window is small and resources are limited, sending endpoints need not use as many flows, and receiving endpoints need not retain unacknowledged out-of-sequence packets.

4.1 Bit Error Detection (mCRC)

Meta-RoCE Transport protocol follows a similar structure as RoCEv2. The protocol is encapsulated as a UDP packet whose destination port, 2766 by default, identifies the Meta-RoCE Transport payload. The transport headers, payload, and padding use an end-to-end data protection through Meta-RoCE mCRC present at the end of UDP payload. L2FCS is the standard Ethernet CRC at the end of the frame and updated at each hop. Figure 4.1 describes the framing used for Meta-RoCE Transport.

The Meta-RoCE mCRC is a CRC-32 that is modeled on the Ethernet CRC: it is pre-inverted (starts with a value of -1) and post inverted (the result is bitwise inverted before being stored), and it uses the same polynomial, and thus the same check value. As shown in Figure 4.1, the data protected by the Meta-RoCE mCRC starts from the beginning of the Meta-RoCE header and extends to the end of the data. The Meta-RoCE mCRC does not include the LRH/GRH of a RoCEv2 iCRC, and does not include any components of the outer IP and UDP headers.

As with RoCEv2, a Meta-RoCE sender is not expected to correctly set the UDP checksum. The UDP checksum must be either zero or correct for the payload. A Meta-RoCE receiver is not expected to validate the UDP checksum. It may, if desired, validate a received packet that carries a nonzero UDP checksum before accepting the packet, but must not reject a packet for having a UDP checksum value of zero.

A frame received with an invalid mCRC must be discarded and counted.

When carried directly over Ethernet, a frame must reach the 60-byte minimum, excluding the frame check sequence. The base and data headers occupy 32 bytes, and the 14-byte Ethernet header leaves 46

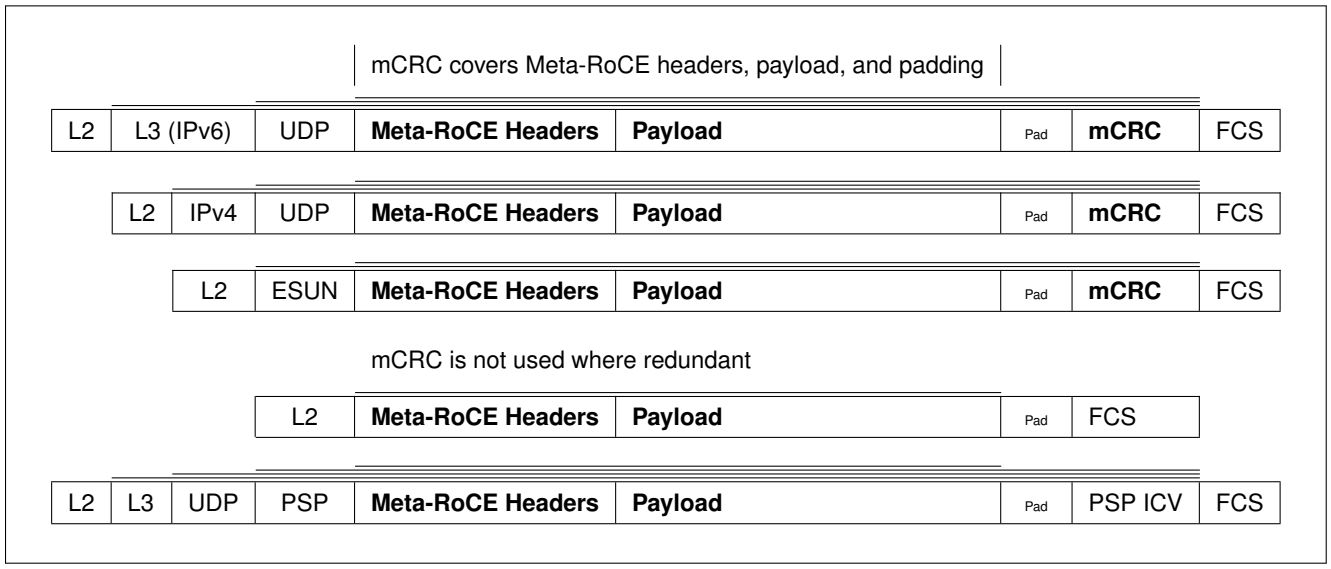


Figure 4.1: Frame Meta-RoCE mCRC coverage. Stacked lines indicate L2, L3, and UDP payload. Where the end-to-end mCRC would be redundant, it is omitted, e.g., when carried by PSP encapsulation where an integrity check value is present, or when carried directly by Ethernet where the FCS is unaltered end-to-end. When carried by ESUN encapsulation, the mCRC is required because the frame is not end-to-end protected by an unchanging CRC.

bytes for Meta-RoCE, so a payload shorter than 14 bytes is padded up to that length. The transmitter must record the number of padding bytes in the Padding field of the data header (Section 8.2). Fourteen is the largest count this requires, so four bits suffice.

4.2 Acknowledgment Generation

This section describes three components: when the transmitter should request an acknowledgment, when the receiver should send an acknowledgment even when not explicitly requested, and finally the contents of this acknowledgment packet.

4.2.1 Transmitter sets the ack request bit

A bit in the data packet header signals the receiver to send an immediate acknowledgment for the packet being received. The receiver may send acknowledgments at other times (Section 4.2.2), but must send an acknowledgment for data packets received with the ack request bit set.

The transmitter should set the ack request bit on every packet that is:

- At the end of a window of transmitted packets, i.e., whenever sending this packet leaves no available space for sending additional data into the network based on the congestion control algorithm in use.
- At the end of a batch of transmitted packets. A batch may be defined as what is sent in a burst on the same flow (Section 6.1.3), as what is sent when the queue pair takes its turn, perhaps four packets

at a time at a 1024-byte MTU, or as what a large enough flow window and a large enough backlog of data allow.

- Whenever sending a smaller than MTU sized packet, as the immediate acknowledgment may allow eager completion of this message, even if additional messages are pending to be sent.
- Whenever the per-flow window is less than a configurable number of (typically 4) packets.
- On all retransmissions. In addition to requesting an ack, retransmissions are also marked within the data header and in the encapsulating IPv6 header. Retransmissions that do not change the flow entropy (typically SACK-based retransmissions on the same flow) may also be sent with the high bit of the IPv6 hop limit field set to enable in-network monitoring. Where the typical hop limit is 64, the retransmission would be sent with 192. Retransmissions by timeout that involve an entropy change should not set the high bit, as the packet will traverse a new network path where the marking would be misleading.
- On the first packet sent on each flow during fast-start. (Section 6.1.3)
- When there are no more packets to send.

The transmitter may set the ack request bit on other packets as well, for example to increase acknowledgment frequency for high priority traffic, or to provide a clearer picture of delivered packets at the end ("tail") of a message.

As an optimization, a read or atomic request that solicits a response may be implicitly acknowledged by that response. A sender with this capability need not set the ack request bit on such a request.

4.2.2 Receiver sends acknowledgment

The receiver must send an acknowledgment in the following situations. In this context, "send" means preparing an acknowledgment that will either be transmitted promptly or superseded by a subsequent acknowledgment; it does not require an immediate or dedicated transmission for each condition. Under load or with small packets, multiple conditions may occur before an acknowledgment can be transmitted; it is permitted for a single acknowledgment to satisfy multiple pending conditions. An acknowledgment may wait its turn to transmit, including behind higher priority traffic, but should be transmitted or superseded within 2μs of the condition that requires it.

- After receipt and delivery to memory of a packet carrying the ack request bit (Section 4.2.1).
- After receipt and delivery of out-of-order data, i.e., if flow sequence number (FSN) 2 is received when flow sequence number 1 has not been. An eager acknowledgment expressing the "hole" in the sequence space allows immediate retransmission.
- On receipt of out-of-window data, e.g., if flow sequence number 2 has been cumulatively acknowledged, but flow sequence number 1 is received, or if flow sequence number 400 is received when sequence number 3 is expected. The acknowledgment sent in response to out of window data restates the receiver's state. In these examples, the acknowledgment would only state a cumulative FSN of 2, with no additional information.
- Before the count of CE-marked packets would exceed 15, the largest value the CNP field can express (Section 8.5.1); acknowledging at any lower count also satisfies this condition. Sending the acknowledgment reports and clears the count.

An HRNR or BRNR backoff withholds the transmission of data for messages this endpoint initiated. Acknowledgments, read responses, and atomic acknowledgments continue to be generated and sent.

It is expected that acknowledged data will be delivered to memory in reasonable time once acknowledged. Data need not be delivered to memory before sending the acknowledgment, enabling quick response, if successful and timely delivery can be guaranteed. Data that cannot be delivered to memory (e.g., because the sent data does not have a matching receive to identify where in memory it should be delivered) must not be acknowledged. Data that has previously been delivered should be acknowledged.

The acknowledgment is flow-specific; it must echo the flow identifier from the packet that caused the ack to be sent.

The traffic class for data and the traffic class for acknowledgments are configured separately, per device and per queue pair. An acknowledgment should be placed in a class the network schedules ahead of the data class. Where no such class is available, both use the same one.

4.2.3 Acknowledgment contents: FSN fields

Three fields appear in the packet header as described below. These are defined for supporting selective acknowledgments.

- **CumulativeFSN**: This represents a FSN from which the bit vector operations will be applied as an offset. The number is circularly non-decreasing and hence any information loss due to drop of an acknowledgment packet will be recovered by the next one. This should equal rcv.nxt-1, but a receiver may set this to be less than this value if the first selective acknowledgment bits are set. (The rcv.nxt variable is defined to be the next in-order sequence number expected in IETF RFC 793's description of a well known sliding window transport protocol.)

- **SelAckBitVector**[256 bits]: A value of 1 represents a received packet that has been received and will (barring fatal hardware) be delivered to memory or otherwise processed appropriately. A value of 0 represents other packets; a 0 before a 1 indicates a packet that is being negatively acknowledged. Bit 0 represents the packet at CumulativeFSN+1 and has the same meaning as every other bit. It is normally zero, since CumulativeFSN would otherwise have advanced, and an implementation may set it to one where CumulativeFSN has, for implementation specific reasons, not advanced.

  That the bit vector is 256 bits long does not imply that the state associated with retransmission necessarily tracks exactly 256 packets; implementations may be designed to track more or fewer to adapt to a specific deployment scenario. Where more are tracked, only the 256 packets after CumulativeFSN are reported in the bit vector; where fewer are tracked, the trailing bits will be zero.

- **MaxFSN**: This represents the highest FSN that a receiver has fully acknowledged (received and will be delivered to memory). MaxFSN is primarily an optimization: in the common case, MaxFSN == CumulativeFSN and it is not necessary to consult the bit vector. Otherwise, at most $\min(256, \mathrm{MaxFSN} - \mathrm{CumulativeFSN})$ bits need to be checked for out-of-sequence delivery.

  MaxFSN may be set to a value beyond the range expressible by the 256-bit selective acknowledgment bit vector, if the receiver has fully received and will deliver the identified frame. A receiver that tracks more than 256 out-of-order packets may advance MaxFSN past the bit vector range, advancing CumulativeFSN as intervening packets arrive. This permits a sender with an effective flow window larger than 256 (e.g., during loss recovery) to continue making progress.

  MaxFSN must not be set to the FSN of a packet that is not being fully acknowledged. In particular, packets that were received but discarded (out of window) must not be reflected in MaxFSN.

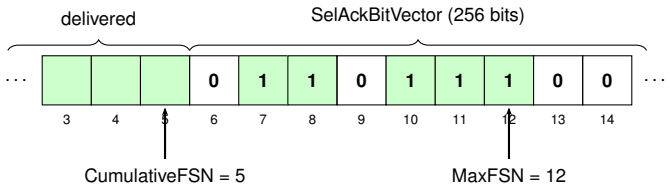


Figure 4.2: Acknowledgment fields on an FSN number line. Packets at or before CumulativeFSN (shaded) are contiguously delivered. The SelAckBitVector begins at CumulativeFSN$+1$; a 1 indicates a received packet and a 0 indicates a gap. MaxFSN marks the highest acknowledged FSN. In the common case, MaxFSN equals CumulativeFSN and no bits need to be consulted.

Figure 4.2 shows these fields on an FSN number line, illustrating how the bit vector encodes gaps between CumulativeFSN and MaxFSN.

An acknowledgment that has passed packet validation (Section 8.1.5) must be discarded without further processing under six conditions, and these are the only grounds for discarding it. In this explanation, `before()` is a less-than operation over sequence numbers that can wrap; `!before()` is equivalent to greater-than-or-equal-to. An acknowledgment must be discarded if any of the following conditions are true:

- `before(MaxFSN, snd.una)`, i.e., this ack at best has old information, perhaps it was delayed. This condition may occur in normal operation. The sender has already accounted for everything the ack identifies; the ack must be discarded as stale. No counter is expected.

- `!before(CumulativeFSN, snd.nxt)`, i.e., this sender has not sent the packet being cumulatively acknowledged. This error should never occur in normal operation. The ack must be discarded and counted.

- `!before(MaxFSN, snd.nxt)`, i.e., the ack reports receiving a packet this sender has not sent. This error should never occur in normal operation. The ack must be discarded, and may be counted.

- In implementations that use the CDMSN to complete messages, the CDMSN is at or beyond the Next Transmit MSN for the ack's Tag, where that Tag is one the sender holds, i.e., the responder reports delivering a message this sender has not sent. This error should never occur in normal operation. The ack must be discarded and counted.

- `before(MaxFSN, CumulativeFSN)`, i.e., the receiver sent an ack as if the window is greater than half the sequence space for FSNs, which should not be possible. This error should never occur in normal operation. This is a malfunction at the peer. The ack must be discarded, and may be counted.

- $RTT_{meas} > \min(2 \times MaxRTO, MaxAckAge \times (\texttt{RTT[f]} + \text{mdev}))$, i.e., the ack was generated in response to a packet the sender transmitted more than *MaxAckAge* round trip times ago. An acknowledgment held in the network this long cannot be trusted to identify data in the current sequence space; it must be discarded and counted. The measured round trip time (RTT_{meas}) is the difference between the current time (now) minus the echo timestamp in the ack, described in more detail below in Section 4.5.1. The mean deviation (mdev) term adjusts for cases of high uncertainty in round trip time, for both initial conditions and cases where the forwarding delay is varying. It is described more below in Section 4.5.2. The MaxRTO term is a configurable upper bound so that this limit does not

prevent valid communication; 200ms is likely suitable for any network and may be reduced as an optimization for smaller networks. MaxAckAge is configurable and defaults to 64; its ceiling is derived after this list. The per-flow estimated RTT term, `RTT[f]`, may be replaced with the per-QP weighted RTT, `RateWeightedRTT`, depending on which is more readily available.

No field of a discarded acknowledgment is applied, including the timestamp, congestion count, rate hints, and active port set.

The MaxAckAge condition protects the sequence space. A flow sequence number identifies data uniquely only until the FSN wraps (Section 2.5), so an acknowledgment delayed far enough can arrive with an FSN from before the sequence space wrapped. Applying it would cumulatively acknowledge a frame that was never delivered: the sender would retire the frame and never retransmit it, while the receiver, whose own CumulativeFSN remains behind the gap, could only repeat an acknowledgment that the first condition above discards as stale. No part of such an acknowledgment, including its cumulative FSN, may be applied.

The bound is expressed in round trip times because a sender advances its FSN by at most `FWND[f]` per round trip time. Aliasing requires the FSN to advance more than half the 16-bit flow sequence space while the acknowledgment is held, which takes $32768/\texttt{FWND[f]}$ round trip times. MaxAckAge must therefore be configured below $32768/\texttt{FWND}_{max}$: 128 at the 256-packet scope of the selective acknowledgment bit vector, twice the default. The 32-bit transmit timestamp (Section 8.2.2) spans approximately four seconds at nanosecond precision, so a hold longer than that wraps the difference; such a hold exceeds the buffering available in the networks this specification targets.

4.2.4 Acknowledgment contents: MSN field

The CDMSN field describes the progress of the QP through message sequence numbers. These fields may be used to confirm progress while debugging or to determine that a message has been completed at the remote endpoint.

- **Cumulative-Responder-Delivered-MSN (CDMSN)** The cumulative responder delivered MSN represents the highest MSN of in-order received and delivered messages. For example, after having received all packets of write messages having MSN 0 and 1, this field will contain 1. An initiator may use this field to identify send and write WQEs that can be completed in order, in cases where a sequence number is preferable to confirming that each FSN on each flow has been delivered. The transmission of this sequence number in an acknowledgment is not sent reliably, and the FSN retransmission layer itself will not retransmit an arbitrary packet to solicit an update. The requirement is that the packets in this message be delivered; an implementation may also determine that the message is complete before updating this field.

Figure 4.4 shows how the CDMSN field advances as complete messages are delivered in order, independently of per-packet FSN acknowledgments.

4.2.5 MSN Fields and Delayed Update

The data sender should use the acknowledgment of all FSNs within the message to determine that a message has been delivered and need not be retransmitted further. The CDMSN is a cheaper route to the same knowledge. The responder already tracks which messages it has delivered (Section 3.4.4) and reports the highest delivered in order, so an initiator can complete messages from one cumulative number rather than from every FSN on every flow.

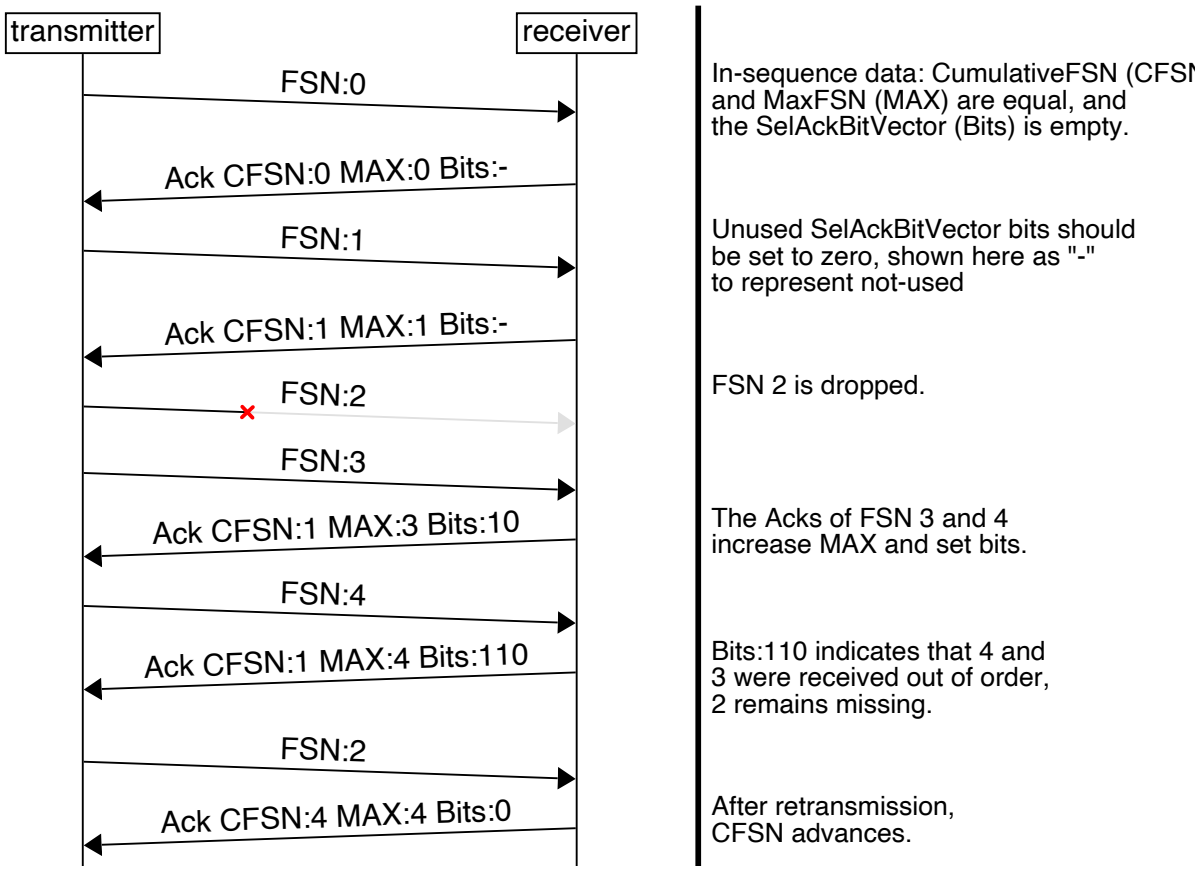


Figure 4.3: Flow sequence number fields packet sequence

It is expected that CDMSN may not be updated on the critical processing path before sending an acknowledgment, and thus may lag behind the FSNs that have been delivered. That is, on receipt of a packet, checking whether this packet completes an in-sequence message may occur in a stage of processing separate from acknowledgment generation.

A delayed-update CDMSN sequence is shown in Figure 4.5.

If the MSN fields are being used to determine that a message has been completely delivered, a mechanism to solicit an acknowledgment is needed. This mechanism may be a retransmission of prior data if there are no packets from later messages to send.

4.3 Acknowledgment contents: Additional

Additional acknowledgment formats may be defined, e.g., to reduce acknowledgment size or consolidate acknowledgment across multiple flows.

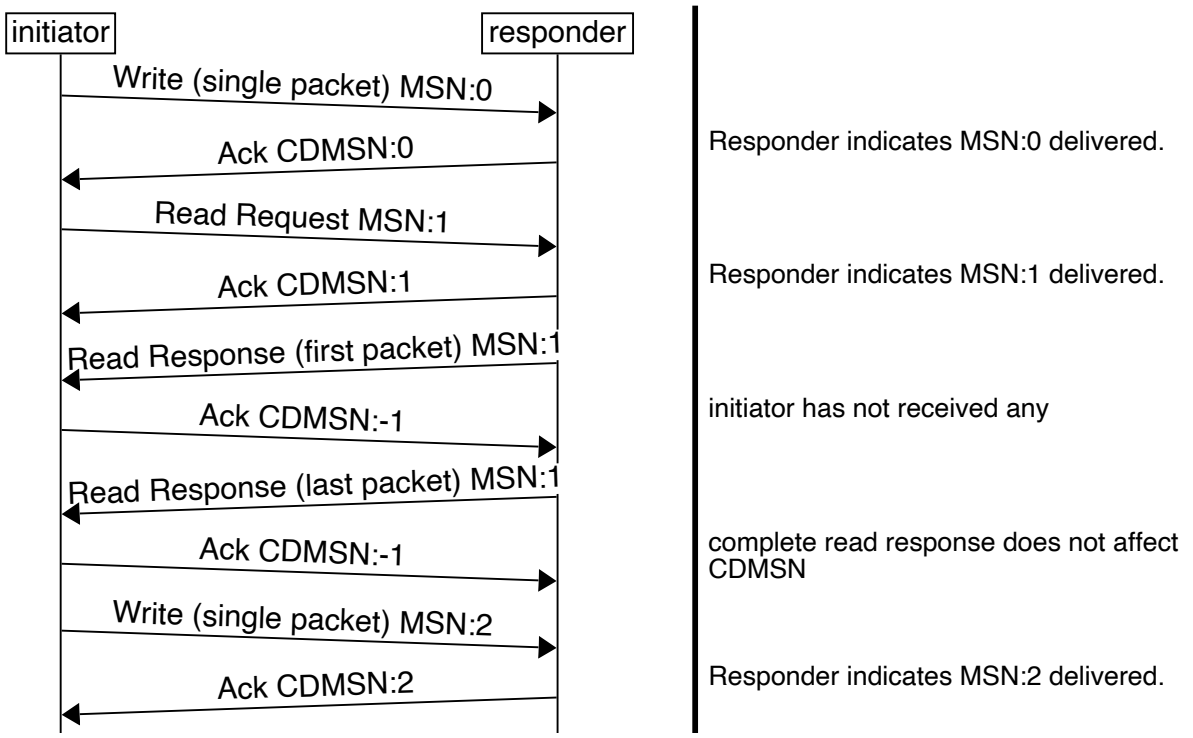


Figure 4.4: Message sequence number fields packet sequence

The RNR bitset in the ack is defined below in Section 4.10, and is present only when one of the RNR states is active in the Ack Status field (Section 8.5.6).

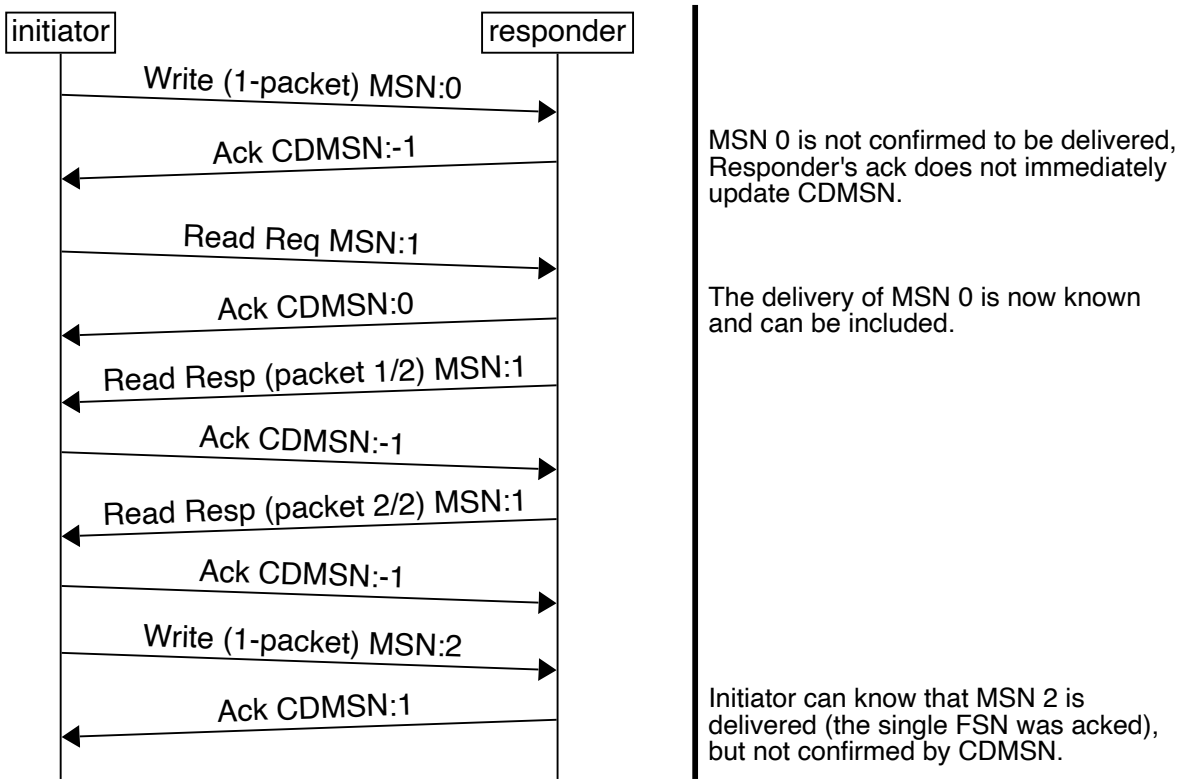


Figure 4.5: Message sequence number fields if acknowledgments are generated before message completion is checked.

4.4 Packet Trackers

This section describes the state required at transmitter and receiver to remember what has been transmitted (and may need retransmission) and to remember what has already been received and passed to the hardware for delivery to memory.

4.4.1 Transmitter Packet Tracker, per flow

The transmitter must track variables equivalent to snd.una and snd.nxt, as defined below and in IETF RFC 793, as well as an array that maps not yet acknowledged flow sequence numbers to the message contents that they carry.

The snd.nxt variable is the *next scheduled FSN*: As new packets are transmitted, this number monotonically increases and the tracker entry binding this FSN to the QP/Tag/MSN/POSN would be updated accordingly. The tracker must have space to hold this binding before a packet may be sent.

The snd.una variable is the *next contiguously acknowledged FSN*: This pointer represents the FSN until which the data has been contiguously received at the receiver and is advanced based on the acknowledgments received. Due to packet drops, some of the packets may have been retransmitted and packets delivered in an out-of-order manner. The “una” part reflects that this FSN is “un”-acknowledged.

In specific cases, snd.una may be advanced by the sender without having received contiguous acknowledgment, described below in Section 4.10 regarding RNR handling.

The difference between snd.nxt and snd.una is the “window.” The sender must track the delivery status of each FSN within this window so that unacknowledged packets can be retransmitted.

The size of the window may be constrained both by available resources and by the congestion control algorithm (Chapter 6). For maximum throughput performance, resources should be allocated so that the window may reach twice the bandwidth delay product (BDP) of the flow, counted in packets.

4.4.2 Receiver Packet Tracker, per flow

The receiver must track values described in the acknowledgment contents section (Section 4.2.3).

4.5 Packet loss detection by per-flow timeout

Determining that an acknowledgment is overdue by enough time that a new transmission should be sent requires instrumenting the round trip time and computing an expected upper bound to that round trip time.

4.5.1 RTT Measurement

Figure 4.6 shows the RTT measurement algorithm used to estimate the RTT of each Meta-RoCE flow.

Every data packet header contains a timestamp T_{send}. When a data packet triggers an ACK, the timestamp of that data packet is echoed in the ACK. This is shown in Figure 4.6.

As shown in Figure 4.7, the timestamp most recently received on a flow is echoed; if packets are received out of order, the echoed timestamp of the second packet sent (the first packet received) will appear in the first ACK, and the timestamp of the first packet sent will appear in the second ACK.

Upon receiving an ACK that passes the acceptance criteria in Section 4.2.3, the sender can then calculate the measured round trip time as the difference, $RTT_{meas} = T_{curr} - T_{send}$. This RTT includes both

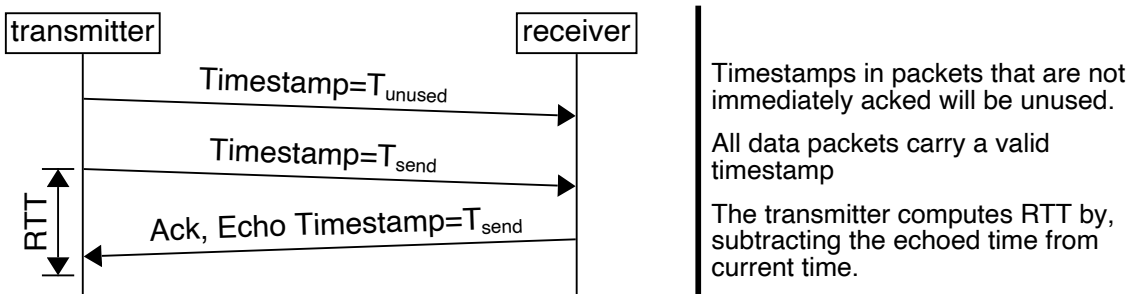


Figure 4.6: Round-trip-time measurement using echoed timestamps

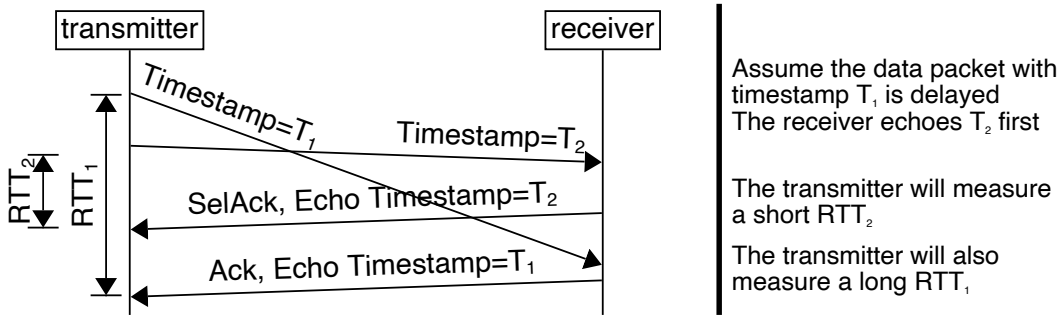


Figure 4.7: Round-trip-time measurement with out-of-order packets

the time spent in the network, and the time taken to process the data packet and generate the ACK at the receiver, which is expected to be consistent.

There are two smoothed estimates of the round trip time to update: for the flow and for the queue pair overall. The smoothed round trip time estimate for the flow is used in retransmission timing and may be used when managing congestion. The receiver-driven rate limiting algorithm (Section 6.3.3) combines the smoothed round trip time estimate for the overall queue pair with rate hints to compute a queue-pair level window limit.

Upon ACK receipt, both `RTT[f]` and `RateWeightedRTT` are updated with an exponential moving average filter:

$$\texttt{RTT[f]} = (1 - \alpha_f) \times \texttt{RTT[f]} + \alpha_f \times RTT_{meas}$$
$$\texttt{RateWeightedRTT} = (1 - \alpha_{qp}) \times \texttt{RateWeightedRTT} + \alpha_{qp} \times RTT_{meas}$$

where the α parameters for per-flow smoothing (α_f) and per-QP smoothing (α_{qp}) may be configured by device. Implementations may restrict α_f and α_{qp} to be negative powers of two for simplicity of implementation.

Following RFC6298 Item 2.2, on the first RTT measurement for a flow the smoothed estimate is initialized to that measurement ($\texttt{RTT[f]} = RTT_{meas}$) and the mean deviation to half of it ($\texttt{mdev} = RTT_{meas}/2$), rather than filtered from a starting value.

In deployments where all flows have comparable RTT, the RateWeightedRTT may be approximated by the smoothed RTT of a single flow (`RTT[f]`).

To aid in setting latency parameters for retransmission and for congestion control, an implementation should be capable of tracking a histogram of measured round trip time latencies. The bucket boundaries and their number are left to the implementation.

4.5.2 RTO computation

A retransmission timeout value is configured in the conventional way as a function of smoothed round trip time and a multiple of mean deviations, with a configurable lower bound (do not retransmit-by-timeout before this time), upper bound (ensure retransmit-by-timeout occurs at least this fast), and device-configurable offset ρ. The most recent measured round trip time is an additional lower bound.

The lower bound should accept values from 10 microseconds through 5 milliseconds, and the upper bound values from 1 millisecond through 100 milliseconds.

In the notation below, `RTT[f]` and `mdev` refer to per-flow estimates; implementations that do not maintain per-flow RTT tracking (which is recommended but not required) may substitute the per-QP `RateWeightedRTT` and per-QP mean deviation. The timer applies per flow regardless of whether the RTT estimate is per-flow or per-QP. There is one timer per flow, not one per packet; the timer is reset every time a new packet is sent on that flow.

$$\texttt{mdev} = (1-\beta) \times \texttt{mdev} + \beta \times |RTT_{meas} - \texttt{RTT[f]}|$$
$$\texttt{RTO} = MIN(upper, MAX(lower, \texttt{RTT[f]} + 4 \times \texttt{mdev} + \rho, RTT_{meas}))$$

When there is unacknowledged data (snd.nxt $>$ snd.una), and RTO time has elapsed since the last transmission, retransmission should occur.

This estimator adopts the basic structure of RFC6298: initialization on the first measurement (Item 2.2), the same update rules for the smoothed round trip time and its variation (Item 2.3), and configurable lower and upper bounds (Items 2.4 and 2.5) scaled appropriately for in-datacenter network topologies rather than the RFC's one-second floor and sixty-second ceiling. Item 2.3 guards against the variation term becoming very small by flooring it at the clock granularity; this design achieves a similar effect with the additive offset ρ described below. Additional background can be found in Peterson and Davie.

Karn's algorithm is not necessary because round trip time is measured exclusively by echoed timestamps, so there is no retransmission ambiguity. This design also differs from RFC6298's timer management: the timer is not reset on acknowledgment receipt (unlike Item 5.3) unless the new expiration time would be sooner; multiple packets may be retransmitted at timer expiration (unlike Item 5.4, see Section 4.5.3); and the timer is not expected to be backed off exponentially after expiration, though an implementation may apply exponential back-off (as in Item 5.5) after a configurable number of linear retries. (See Linux `tcp_syn_linear_timeouts` for a comparable parameter.) The clock may be coarse; the retransmission need not happen at precisely the time of expiration of the timer.

The ρ (rho) parameter is required. It primarily accounts for the expected difference in time between (a) when the retransmission timer is set and (b) when the timestamp is placed on the outgoing packet. These times may differ significantly if the decision to transmit may be made and the retransmission timer reset before the contents of the packet are fetched via DMA, while the timestamp is set just as the fully assembled packet is sent. In this design, the measured `RTT[f]` would be too small to predict when an ack is overdue. Small increases in ρ may also be useful in protocol tuning to discourage extra retransmissions by timeout.

Acknowledgment Fields			New Sender State Variables			Action
CumulativeFSN	SelAckBitVector	MaxFSN	snd.una	snd.cur	snd.nxt	
			0	0	10	Initial State
1	-	1	2	2	12	Transmit 10, 11
1	10	3	2	3	12	Retransmit 2
1	110	4	2	3	13	Transmit 12
1	100110	7	2	6	14	Retransmit 5, Transmit 13
1	1100110	8	2	7	14	Retransmit 6
4	11100	9	5	7	16	Transmit 14,15

Figure 4.8: Sender retransmission cursor advancement by acknowledgment sequence. The cursor advances with snd.una or as packets are retransmitted.

4.5.3 RTO retransmission count

The number of packets retransmitted by timeout must be configurable. Configuration must allow extremes of a single packet (either the first, snd.una, or last, snd.nxt-1) and all unacknowledged packets. The appropriate logic depends on the topology and other parameters of congestion. With a high timeout interval, the state of the network has likely changed and the connection is behind schedule, justifying a complete retransmission of all pending packets. With a low timeout interval, eagerly probing the network with a single retransmission may avoid spurious retransmissions.

Retransmissions by timeout should not be prevented by congestion window checks.

4.6 Packet Loss Detection by Out of Order Receipt / SACK

In Figure 4.3, the sender could identify that a packet was lost, observing a hole in the acknowledged sequence space. The snd.una variable can advance to this hole, but will need to retransmit the missing packet just once in this round trip time, regardless of how often it receives a similar selective acknowledgment that was generated by subsequent packets.

In addition to snd.una and snd.nxt, the sender maintains the “next-retransmission-cursor” pointing to the next packet that may be retransmitted due to a selective ack. This cursor is abbreviated snd.cur. The purpose of snd.cur is to prevent repeated retransmission in response to selective acknowledgment. Its update is illustrated in Figure 4.8. The cursor is updated to be at least snd.una as in-order data are acknowledged (snd.una advances). The cursor is incremented to cover data retransmitted to fill a gap in the selective acknowledgment. Receiving additional duplicate selective acknowledgments with the same gap will not solicit additional retransmissions, because the missing packets precede the cursor. During SACK-driven retransmission (without a preceding RTO), snd.cur does not advance beyond MaxFSN; packets between MaxFSN and snd.nxt are not considered lost until a subsequent acknowledgment reports them or the RTO expires. Like snd.una, snd.cur advances past an FSN only when that FSN is delivered, marked RNR, or has been retransmitted.

The RTO timer is reset by any transmission, including the retransmission of data to fill a hole in the selective acknowledgment.

On expiration of the RTO timer, the sender resets snd.cur to snd.una, changes the flow’s entropy value (Section 4.8), and then retransmits. Resetting snd.cur permits the use of selective acknowledgment to re-retransmit if necessary. When retransmissions are sent starting at snd.una for the RTO, snd.cur advances.

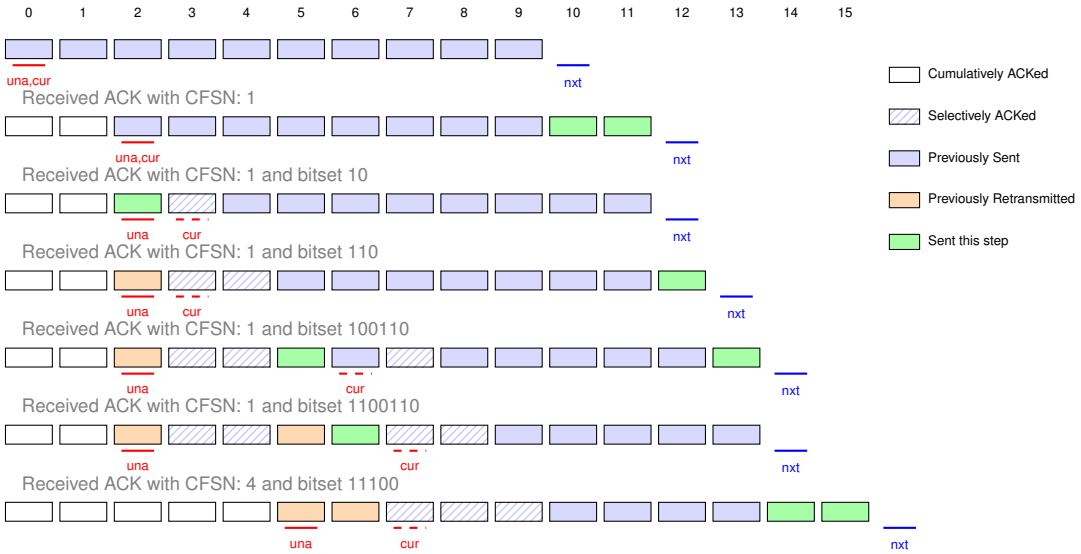


Figure 4.9: Sender window state corresponding to Figure 4.8. Underlines mark cursor positions: solid red for `snd.una`, dashed red for `snd.cur`, blue for `snd.nxt`. Between rows, the received acknowledgment that triggers the state transition is shown.

The implementation can choose how many packets to retransmit when the RTO fires, which may be fewer than the total number of missing packets. After a partial RTO burst, snd.cur remains at the burst endpoint (it is not reset back to snd.una). Subsequent acknowledgments, whether cumulative or selective, drive continued retransmission from snd.cur. Figure 4.10 illustrates this sequence.

All (RTO and SACK based) retransmissions must be sent with the retransmission bit in the data header set. This bit is meant to aid in estimating network error rates.

Data sent again because the receiver was not ready is a separate case. After a BRNR-NAK the data is sent under a new *(Flow,FSN)* (Section 2.5), so it is not a retransmission and neither the “R” nor the “T” bit is set. After an HRNR both bits are undefined: resumption draws from the retransmit queue on a backoff timer, much as the timeout path does, so an implementation need not distinguish these packets from a retransmission. In an environment where HRNR occurs, the retransmission counters can therefore include packets that were not lost. Retransmissions in which the expected network path does not change, typically SACK-based, may also be marked with the high bit of the IPv6 hop limit field set. Using the IPv6 header marking scheme, where the typical hop limit is 64, the retransmission would be sent with hop limit 192. Marking in the IPv6 header may be more convenient for switches to count without parsing the Meta-RoCE header.

4.6.1 In-network packet reordering

Loss detection by out-of-order receipt expects that packets on a flow will not be reordered by the network. In practice this expectation is that the network does not “spray” packets that are sent with the same “5-tuple” flow information. To permit the protocol to operate in an environment where network switches make

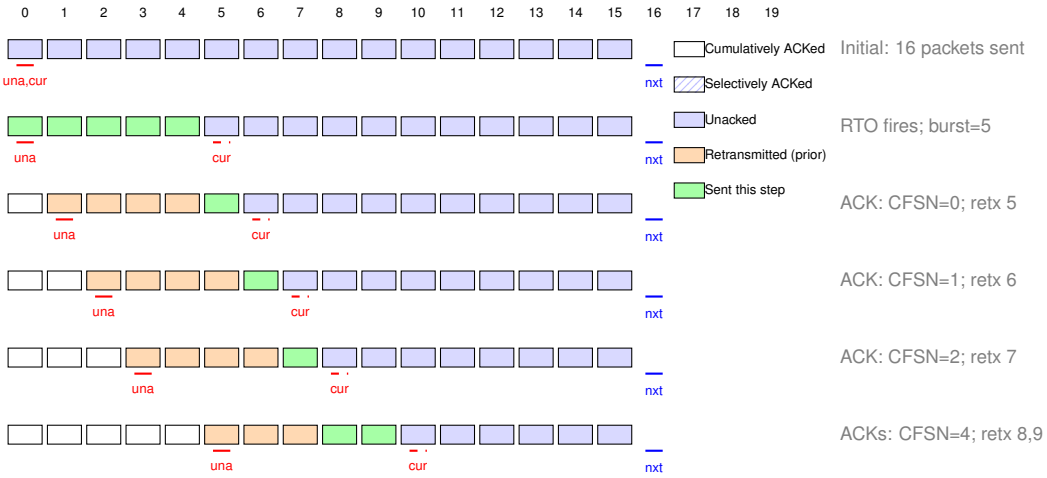


Figure 4.10: Partial RTO retransmission followed by ACK-driven continuation. In this example, no new data is available (snd.nxt does not advance); all transmissions after the RTO burst are retransmissions of previously sent packets. The RTO fires and retransmits 5 of 16 unacknowledged packets; snd.cur advances to 5. Each subsequently acknowledged packet allows one retransmission from snd.cur, which skips already-acknowledged FSNs.

per-packet load-balancing decisions, implementations must support at least two modes:

1. disabling packet loss detection by selective ack, using the ack only to prevent retransmission of received data by timeout, and

2. permitting sack-based retransmission and advancement of the next-retransmission-cursor (snd.cur) only when the cursor has a value less than the maximum received FSN (MaxFSN) minus a configurable fraction of the flow window (defined below in Section 6.1.1).

Additional retransmission modes are permitted to achieve the objective of retransmission soon after the acknowledgment for a packet is overdue.

4.7 Eager Tail Retransmission (optimization)

As an optimization in situations in which tail loss is suspected, a transmitter with no additional data to send may eagerly retransmit the last packet sent when the connection has been idle for an interval shorter than RTO but longer than RTT. Such an optimization may be appropriate in cases where the minimum RTO is configured to be high (because SACK-based recovery proves effective and RTO triggers falsely due to highly variable queuing delay) and tail completion time is an important performance constraint that occasional tail retransmissions are beneficial.

4.8 Alternate Flow Retransmission

An implementation may avoid repeatedly transmitting down bad physical flows. Since the transmitter determined that there was packet loss on the flow whose timer fired, it is reasonable to immediately try an alternative physical path. The retransmission must repeat the same flow ID and FSN, but may be sent with different information in the encapsulating header (a changed UDP port, addresses associated with different physical network ports, or a different IPv6 flow ID) depending on the implementation.

- **Change entropy on RTO**: After an RTO, it is expected that no packets from this flow are active in the network. The transmitter must change the entropy value (source port) of the current flow before retransmitting, so that retransmitted packets are directed across the network differently. A configurable number of timeouts may be retried on the original entropy value before the change. Where the current entropy value appears to lead to a failed, persistently congested, or longer-than-expected physical path, changing the entropy value to get a different physical path through the network may recover the virtual flow. This is comparable to the autoflowlabel technique.
- **Change output port on subsequent RTO**: On network interface devices with multiple physical ports (Section 7.5), the transmitter may reassign a flow to a different output port if the current port pair appears unresponsive after a second or subsequent RTO. This is a local decision that does not require coordination with the receiver; the flow ID and FSN are unchanged, but the source and destination addresses in the encapsulating header reflect the new port pair. The port failover algorithm (Section 7.5.4) provides a mechanism for selecting an alternate port pair from the available topology.

4.9 FSN UNA (Clear) field

Data packets contain the snd.una field. As a reminder, snd.una represents the sequence number that the sender believes is not yet acknowledged. Unacknowledged FSNs can therefore be skipped, which is used when a message was stopped due to RNR and will be transmitted later with different FSNs. Figure 4.11 illustrates how UNA advances in data packets as the sender learns which FSNs have been acknowledged.

The UNA field tells a receiver how much per-FSN state it may free, and no more than that about the packet carrying it. A receiver that reads it as a bound on what it may accept discards good data, and multipath reordering makes a lagging UNA ordinary rather than exotic.

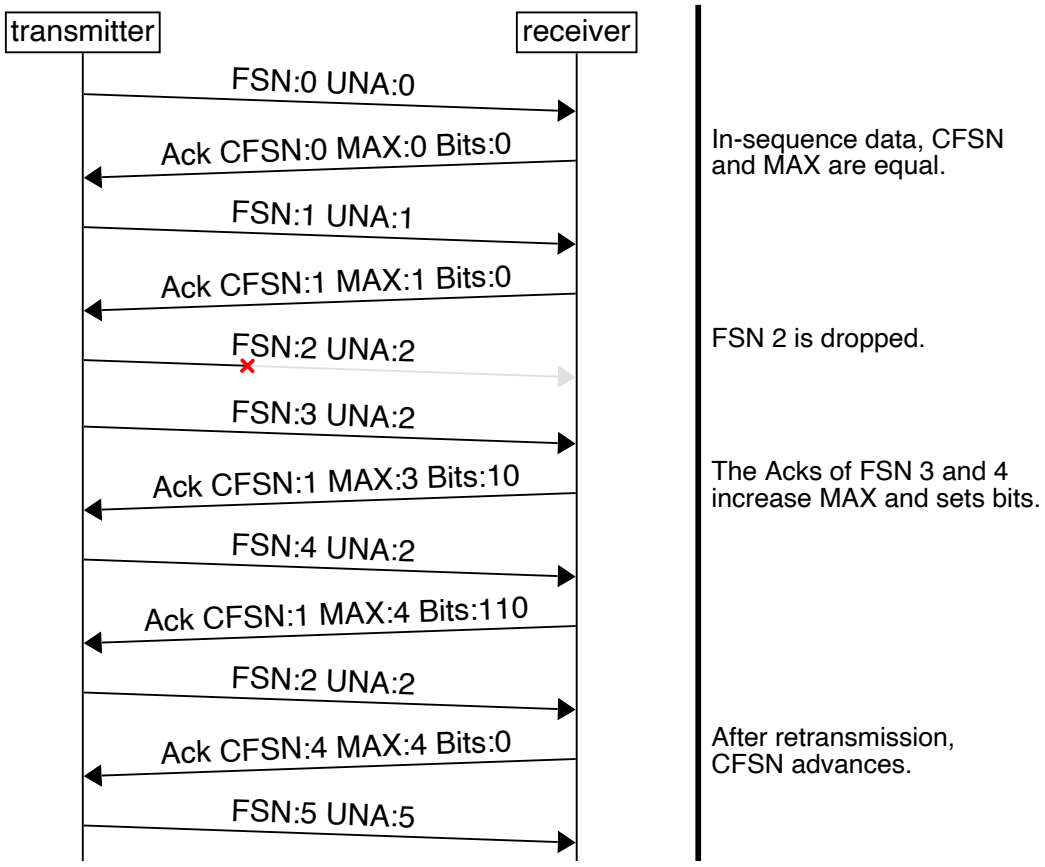


Figure 4.11: Basic example of sending UNA in data packets

4.10 RNR Data Recovery

RNR messages are sent to the initiator when the receiver is not ready to receive data. The receiver may lack necessary resources to accept and process received data packets. Primary objectives of data recovery when RNR occurs are:

- Once the resources (including hardware, buffers, etc.) are available on the receiver side, the goal is for recovery to happen within 1 round-trip time and resume forward progress.
- Recovery from RNR aims to minimize retransmission of packets that are in flight. For instance, if receive buffers are posted in the middle of the first flight of packets and some packets could not be delivered, ideally only the lost packets would be retransmitted.
- Implementation state management trades off tracking granularity against network efficiency under error conditions; less state may lead to more retransmissions than necessary.
- The detection and recovery of RNR is not expected to slow down the fast flow. Normal delivery remains unaffected, across other QPs and other unaffected tags within the same QP. Traffic for alternative communicators (tags) on the same QP is not blocked.
- After several retry attempts, the implementation eventually fails the work request (this is a software-defined configurable timeout that does not exist in other systems) and flags errors to the software/application for further actions.

Once transmitted, a packet is in one of three logical states at the receiver's reliability layer:

1. The packet is reliably delivered on the network and also to the application. This is the normal mode of operation, where acknowledgments and completions are delivered.
2. The packet is reliably delivered on the network, but not to the application. An example of this is when packets are dropped at the receiver due to lack of resources, but the network delivery was successful (potentially after retransmission due to packet loss).
3. The packet is lost on the network due to errors, congestion, or other faults.

The detection of Receiver Not Ready (RNR) on the receiver logically belongs to state #2. When RNR occurs, the acknowledgment packet carries a bit set to indicate to the transmitter which packets resulted in state #2. In addition, the cumulative acknowledgment pointer (CumulativeFSN) is "frozen" until the transmitter acknowledges that those sequence numbers will not be delivered by setting the "Flow Sequence Number Clear (UNA)" field in subsequent data packets. The cumulative acknowledgment pointer cannot move past, thereby "blocking the window" and gives a chance for the clean up to be initiated.

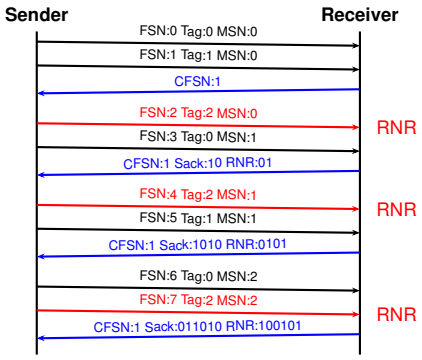
The FSNs that resulted in RNR are consumed and will not be assigned to new packets until the sequence numbers wrap.

A receive work queue entry is assigned a completion sequence number when it is posted, and the responder's highest posted CSN only advances (Section 2.4). A receive buffer matched to a message remains matched to that message. A packet refused because its message had no matching receive is delivered on retry once a receive is posted for the message's CSN.

Actions taken by the transmitters:

Sender Tracker, Flow 0

FSN	Tag	MSN	POSN
0	0	0	0
1	1	0	0
2	2	0	0
3	0	1	0
4	2	1	0
5	1	1	0
6	0	2	0
7	2	2	0



Receiver State, Flow 0

FSN	State
0	delivered
1	delivered
2	rnr
3	delivered
4	rnr
5	delivered
6	delivered
7	rnr
8	*not-seen*
9	*not-seen*
10	*not-seen*

Figure 4.12: Sender tracker, message sequence, and receiver per-FSN state after packets 0–7 are received. Tag 2 packets (FSN 2, 4, 7) are network-delivered but RNR'd (red). Acknowledgments carry Sack and RNR bitsets whose least significant bit corresponds to CFSN+1. The sender has not yet processed the acknowledgments shown.

1. When the RNR is received with the rnr-status-bitmap set, it will walk through the transmit packet state and identify the affected {Tag, MSN}.
2. The transmitter will stop the scheduling of new traffic on the affected {Tag, MSN} and, at least, any subsequent messages on the same Tag that are identified by a CSN. Packets from messages prior to the identified MSN and retransmissions may be sent. Packets from messages after the identified MSN may be attempted, but this option should be restricted to messages that require minimal receiver state (writes without immediate).
3. The transmitter will start the BRNR timer, which determines when the affected packets are retried. It can be configured either in exponential back off mode or in a linear back off mode. A retry the timer triggers is a BRNR probe, which may solicit a further BRNR.
4. The transmitter will keep a copy of the rnr_bit_map received in the ack and when a packet is either acknowledged as delivered or in the rnr bit map, advance the unacknowledged pointer (Flow Sequence Number Clear / UNA) to the next FSN for which an ack is expected.
5. The receiver acknowledges this by sending the acknowledgment packet with the updated cumulative acknowledgment pointer (**CumulativeFSN**). It should use the Flow Sequence Number Clear (UNA) value to advance its next expected FSN and reclaim space in the selective ack and RNR bitsets.

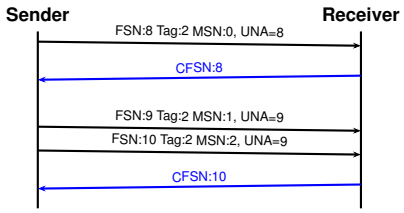
Figure 4.12 shows the sender and receiver per-FSN state after an initial burst of eight packets. When a packet is sent, the sender's tracker creates a binding from the flow and FSN to the tag, MSN, and packet offset (POSN). The receiver maintains per-FSN state with three meaningful values: *not-seen* (no packet received for this FSN), *delivered* (received and delivered to the application), and *rnr* (received on the network but not deliverable due to a missing receive buffer).

In this example, tag 2 lacks receive buffers. FSN 2, 4, and 7 are network-delivered but trigger RNR. The CumulativeFSN freezes at 1, the highest contiguously delivered FSN before the first RNR'd packet.

Acknowledgments carry both the sack and RNR bitsets with CumulativeFSN frozen at 1. The CumulativeFSN in acknowledgments must not advance until a data packet with a UNA field greater than CumulativeFSN is received. Until then, acknowledgments may include increased values of MaxFSN and updated

Sender Tracker, Flow 0

FSN	Tag	MSN	POSN
0–7		(freed)	
8	2	0	0
9	2	1	0
10	2	2	0



Receiver State, Flow 0

FSN	State
0–7	(freed)
8	delivered
9	delivered
10	delivered

Figure 4.13: After the sender processes the RNR acknowledgments and the BRNR timer expires, it sends FSN 8 with UNA=8, allowing the receiver to free per-FSN state for FSN 0–7. The acknowledgment of FSN 8 confirms that the receiver is ready, and the sender exits the RNR state to send the remaining two frames.

bitsets as additional packets are received.

Upon receiving the RNR acknowledgment, the sender:

1. Identifies the RNR'd FSNs (2, 4, 7) from the bitset and looks up the corresponding tag and MSN in the tracker: tag 2 / MSN 0, tag 2 / MSN 1, tag 2 / MSN 2.
2. Halts scheduling of new packets for tag 2.
3. Because all packets before FSN 8 have been accounted for (delivered or RNR'd), the sender advances UNA to 8. When the acknowledgment of FSN 8 is received (CFSN=8), the sender advances UNA to 9 and carries this value on the next data packets sent.

When the receiver processes a data packet carrying UNA=8, it frees the per-FSN state for FSN 0–7 and advances CumulativeFSN to 7. Because the data packet itself (FSN 8) is delivered, CumulativeFSN advances further to 8. If instead FSN 8 triggered another RNR, CumulativeFSN would remain at 7 with an RNR bit set for FSN 8. Tag 2 messages are rescheduled with new FSNs, as shown in Figure 4.13.

4.10.1 RNR-Cancel

An RNR-Cancel (Section 8.6) tells an initiator that what an RNR reported missing is present again: a receive WQE for a BRNR, or the queue pair itself for an HRNR. It allows the default backoff interval to be longer, reducing the traffic spent probing, while still restarting within a round trip once the responder is ready. A responder should support RNR-Cancel. It is not acknowledged or retransmitted: where the network is congested enough to lose it, recovery by probing is appropriate.

An initiator must drop an RNR-Cancel that finds no transmission held.

When canceling a BRNR, which holds transmission on one tag, the responder must send a cancel for each tag that becomes ready, and its Tag field identifies that tag. The Posted CSN is then the highest CSN with a receive WQE posted on that tag, so the initiator learns how many receive-WQE-consuming messages it may send before risking another RNR (Section 2.4).

When canceling an HRNR, which holds transmission on every tag (Section 8.5.6), the responder must send a single cancel, with the Tag zero and the Posted CSN that of tag zero. The initiator must resume every tag, and having learned nothing about buffer availability on the others may then meet a per-Tag BRNR.

Where the tag draws from a shared receive queue (Section 3.4.1) and RNR-Cancel is supported on shared receive queues, the responder must reduce the Posted CSN to the next CSN expected. No buffer is reserved. One is enough to restart the sender, and a larger count would claim knowledge of the queue the responder does not have. An RNR-Cancel may be sent to only a subset of the senders a BRNR holds, avoiding a thundering herd.

4.10.2 Acknowledgment within a BRNR'd message

A single acknowledgment can have both a SACK bitset and an RNR bitset, and both may identify packets of the same message, refusing one packet and acknowledging another.

A responder may refuse a packet with a BRNR only until it has acknowledged another packet of the same message. A BRNR reports that no matching receive was posted, and a matching receive does not disappear once it is found.

In an acknowledgment that refuses one packet of a message and acknowledges another, the refusal preceded the acknowledgment, so the buffer was posted between the receipt of the two packets. The alternative reading, that the buffer was present for one packet and then absent for the other, is not credible. In-network reordering may still deliver acknowledgments and BRNR-NAKs to the initiator in either order.

The RNR state of the earlier packet is not revocable. That packet was not delivered and is sent again under a new *(Flow,FSN)* (Section 2.5). The acknowledgment of the later packet shows that a backoff would serve no purpose. An initiator may treat such an acknowledgment as a BRNR of zero duration, set and immediately cleared, and resume transmission on the tag without waiting.

Chapter 5

Error Handling

Transport errors can occur when the underlying network experiences a failure that interrupts connectivity or when initiator and responder have inconsistent state about a QP (e.g., flow count) or its messages (e.g., what the next message type will be).

Meta-RoCE transport error handling has two goals.

1. Simplicity of implementation – errors that are extremely unlikely need not be recovered, and may be treated as fatal to the QP even if they are in theory recoverable.

2. Isolation of the error – errors that can be made to affect only one flow or QP should be handled in a way that does not cascade to independent jobs.

This chapter describes how an endpoint experiencing an error reports it to the remote, how a failure at the responder is reported asynchronously, and how a QP and its resources are placed into an error state once an error is discovered.

5.1 Retriable Errors

The following recovery methods are supported,

FSN retry - A packet has been lost; it will be retransmitted (Chapter 4). If possible, retransmissions can be sent with different entropy values or on distinct ports to permit avoiding in-network faults. An FSN retry applies to any error that specifically affects a packet (e.g., Meta-RoCE CRC failure) or that could be corrected by retransmission (e.g., internal memory error). All errors that lead to FSN retries must be counted. No one counter covers the category; it is met by counting each cause separately, such as a retransmission on selective acknowledgment, a retransmission by timeout, and an HRNR.

Message retry - A message has experienced a non-fatal condition (e.g., RNR due to a missing receive WQE) that requires it to be held and restarted. Where tags are supported, the retransmission state held with the flow should be reclaimed so that other messages and tags associated with the QP are allowed to make progress, ignoring that certain FSNs were not delivered. Where tags are not supported, and there is no reason to send subsequent messages when the receiver

is known to be not ready, the message contents may be retried with the same flows and FSNs. All errors that lead to message level retry must be counted, likewise by cause, such as the BRNR probes sent (Section 4.10).

Flow entropy change - The successful delivery of packets through a given ECMP flow has become questionable, due to retransmission by timeout or persistent congestion that leads to a trivial window. In response, the sender may change the entropy value (UDP source port) to attempt to find a working, performant flow. Entropy changes should be counted and are unlimited.

Local port fail-over - When multiple ports are supported on a single QP, a port is allowed to fail without severing the overall QP. Local port failure may be directly observed by a device from carrier sense signals or may be inferred from repeated failure to transmit on multiple flows originating from this port. Local port fail-over should be addressed by clearing the port's ready bit, updating the Active Port Set sent in acknowledgments (Section 8.5.7), and reassigning affected flows to working ports using the port failover algorithm (Section 7.5.4). Software should be notified of the port failure.

Remote port fail-over - When multiple destination addresses are supported on a single QP, a failed remote port means that flow state may no longer be available. Remote port failure can be detected via unsuccessful flow entropy changes (Section 4.8), via the Active Port Set reported in acknowledgments (Section 8.5.7), or through software configuration of the topology wiring table (Section 7.5). If the local endpoint determines that a remote port has failed, it should reassign affected flows to a working port pair using the port failover algorithm (Section 7.5.4); the flow ID and FSN are unchanged, but the source and destination addresses in the encapsulating header reflect the new port pair. If the local endpoint is informed (e.g., via Active Port Status in the ack or via QP modification) that the remote port has recovered, it may restore flows to their original port assignments.

5.2 Recoverable error examples

The following error conditions have been identified for explicit treatment.

5.2.1 Invalid Flow ID

If a sender uses a flow id for which the receiver has not allocated a receive flow context (Section 7.3.1), the receiver should either:

- (preferred) allocate resources for reception on the new flow and accept traffic as normal.
- (not preferred) discard the packet and send the NAK with the unsupported ack status code.

A receiver may rate-limit these responses by a policy of its own choosing, discarding the packet where it suppresses the response.

5.2.2 Invalid Tag

If the tag in a received packet is not associated with the QP, the receiver should send a BRNR to indicate that the message was not accepted but future progress on the QP is allowed. The receiver may send an

HRNR to indicate that the message was not accepted and the initiator should wait until the tag is fully set up on the QP before sending further traffic. If the responder does not support tags at all, that is, the capabilities of the responding NIC do not include tag support, the operation will never succeed; there should be a NAK with the “Operational Error” code.

5.2.3 Invalid or not cached QP

If the QP in a received packet is not associated with an in-cache QP, and that QP cannot immediately be brought into cache while the received packet is buffered, the receiver should send an HRNR and notify the driver that a message for an out-of-cache QP has been received. If the QP does not exist, repeated RNR will identify the failure.

5.2.4 Source QP mismatch

If the source QP in a received packet does not match the expected remote QP for the destination QP context, the packet must be silently discarded and counted. This check guards against stale or misdelivered packets from a previous connection or a non-matching endpoint.

5.2.5 Undisclosed padding

The last packet of a message may hold more payload than the message requires, where the transmitter padded it without recording the count in the Padding field (Section 8.2). The receiver detects this only when the payload exceeds the length it holds for the message: the receive buffer length for a send, the message length for a write, or the requested length for a read response. An excess of at most 3 bytes is padding applied to reach a 4-byte boundary; the receiver must discard it and complete the message normally. Any larger excess is a length error, as is any excess where the packet is carried directly over Ethernet and the Padding field is authoritative (Section 4.1).

5.2.6 QP not in a valid state

If a data packet arrives for a QP that is not in the RTR or RTS state, the receiver should generate an error response:

- An HRNR NAK, indicating that the QP exists but is not yet ready to process data. This is the preferred response when the QP is expected to reach RTR, as the sender already has retry logic for HRNR and will back off and retry without escalating to a QP error.
- A NAK with the Operational Error code. This is appropriate when the QP is in a terminal state (e.g., error or reset) where retrying will not succeed. The code reports the responder’s state, since the request is not examined.

The response may be rate-limited to prevent a flood of error responses when a remote sender has not yet learned of the state change; while rate-limited, an individual packet may be silently discarded, and the sender will eventually retransmit by timeout.

5.3 Message-fatal errors

Errors in this section cannot be recovered, and must propagate to the application. The scope placed in error follows the failure, as each subsection describes. When the message that cannot be processed is associated with a WQE, it must be completed in error. When not associated with a WQE, e.g., for a read response or atomic response, an asynchronous notification of the error must be sent.

5.3.1 Reliability layer or connectivity failure

Repeated flow entropy changes or port fail-overs have failed to reestablish communication. The remote is either unreachable through the network or no longer operational. This is handled in two ways depending on the activity in progress.

- With a (non-virtual) WQE in progress (Requester side or Receive on Responder side), the WQE shall be completed in error. The QP should be placed in the error state, along with any associated send and receive queues.
- With a virtual WQE in progress (Error on RDMA Atomic or Read response), an asynchronous error event shall be reported identifying the tag the virtual WQE belongs to. The QP should be placed in the error state, along with any associated send and receive queues.

5.3.2 End-to-end message failure

An unrecoverable error condition has been detected for a message that is not associated with the reliability layer but is associated with a particular WQE. Examples include excessive RNRs, memory protection errors, and implementation specific errors that prevent the WQE from completing successfully.

If the error can be associated with a WQE then the WQE must be completed in error. The error will result in additional actions according to the type of WQE and configuration settings:

An error attributable to one tag's work queue places that queue in error (Section 5.3.3). The error should be confined to that tag. An error that cannot be attributed to a single tag, or that concerns the queue pair itself, places the queue pair in error, and with it every work queue the queue pair owns (Section 5.3.4).

5.3.3 Work queue error

When a work queue is placed in error, the following actions must be taken:

- All work requests remaining on the queue must be completed:
  - If the request is fully executed (i.e., all data have been sent and acknowledged) but has not been completed (i.e., completion did not occur because some packets of a prior message were not acknowledged) then it shall be completed with status "Complete - flushed."
  - else if the request is not fully executed then it must be completed with the status "Flushed in Error."
- Any further work requests posted to the queue while it is in the Error state will be flushed in error.
- The error state is terminal for the queue as it stands. Software returns the queue to the Reset state, from which it is re-established, or destroys it. Returning a single tag's work queue to Reset, leaving the rest of the queue pair operational, is permitted.

This applies to a send queue, to a receive queue, and to a shared receive queue. Support for shared receive queues is optional.

5.3.4 QP error

When a QP is placed in error, every work queue it owns is placed in error as described in Section 5.3.3. A private receive queue shall also be placed in the error state. A shared receive queue's state shall not change (Section 3.4.1). A buffer already drawn from it for a message that will not now complete is flushed in error, and is not returned to the shared queue.

5.3.5 CQ error

When a WQE has completed, whether successfully or otherwise, a CQE is generated on the associated CQ. If that CQ is full, then a CQ overflow error occurs. Sizing a completion queue to hold every completion it may receive is the application's responsibility.

- An asynchronous error event must be reported on the completion queue that overflowed.
- An event should also be reported on the tag whose completion could not be written, once for each such tag rather than once for each refused completion.
- Whether the tag whose completion could not be written is placed in error may be configurable. The completion is lost either way.

5.4 Requester side errors

Error	Description	Handling
FSN not acked. Retry limit not exceeded	Responder detected a missing FSN.	Requester may retry (retransmit) the request.
FSN not acked. Retry limit exceeded	Responder detected a missing FSN. Requester performed retries but all attempts failed.	Requester should attempt Flow Level Failover, if enabled, else WQE error
Local timeout error. Retry limit not exceeded	No Ack response from Responder for one or more packets on a flow within timer interval.	Requester may retry the request.
Local timeout error. Retry limit exceeded	No Ack response from Responder for one or more packets on a flow within timer interval. Requester performed retries but all attempts failed.	Requester should attempt flow level failover, if enabled, else WQE error.
HRNR-Nak retry error. Retry limit not exceeded	Hardware RNR-Nak received for a message.	Requester should schedule the message for retry after indicated time.

HRNR-Nak retry error. Retry limit exceeded	Hardware RNR-Nak received for a message but the message has reached the limit for allowed HRNR retries; The limit may be configured as a duration.	WQE error.
BRNR-Nak retry error. Retry limit not exceeded	Buffer RNR-Nak received.	Requester should free the specific FSNs noted in the BRNR-Nak and schedule those packets for retry, under a different FSN, after indicated time.
BRNR-Nak retry error. Retry limit exceeded	Buffer RNR-Nak received for a message but the message has reached the limit for allowed BRNR retries; The limit may be configured as a duration.	WQE error
Flow level timeout exceeded. Local port migration limit not exceeded	Retransmission timeouts on a flow have reached the configured number of retries on the current entropy value (Section 4.8).	Flow level failure: change the flow's entropy value and retransmit under the same Flow ID and FSN.
Flow level timeout exceeded. Local port migration limit exceeded	Entropy changes have not restored forward progress, and the flow has reached the local port migration limit.	Port level failover (Section 7.5.4).
WQE timeout exceeded	WQE execution has taken longer than the maximum configured time.	WQE error.
Unsupported Opcode	Responder detected a defined but unimplemented opcode	WQE error
Local Memory Protection Error	Requester detected an L_key error or other local memory issue	WQE error
R_Key error	Responder detected an R_Key error	WQE error
Remote Operation Error	Catch all for other errors detected by Responder and reported back to Requester with a NAK	WQE error
Local Operation Error - WQE	Invalid WQE contents (illegal opcode or other)	WQE error
Local Operation Error - no valid WQE	Bad QP number in WQE, other WQE formatting issues, and other implementation-specific errors that make the WQE incoherent and not possible to associate with a Queue	Drop Silently
Local resource error - Bad configuration	Improperly configured local resources prevent requester from executing request	WQE error
Remote length error	Remote node reports an invalid request	WQE error

Read or atomic response MSN mismatch or missing	A read response or atomic (AtomicAck) response is received whose MSN does not match an outstanding read or atomic request. As with responder-side MSN validation (Section 5.5), this can arise from sequence-number wrap, a delayed packet, or a malformed sender, and is handled non-destructively.	Non-fatal; the requester should drop the response without acknowledging it, as for an unexpected acknowledgment. The QP remains operational.
Ghost/unexpected acknowledgment	Ack received that does not match expected sequence number ranges	Drop silently
CQ overflow	Successfully executed WQE cannot be completed due to lack of space in associated CQ	CQ error

5.5 Responder side errors

When a received data packet could trigger multiple error conditions simultaneously, the responder should evaluate checks in the following order and act on the first applicable error:

1. **Can the packet be delivered?** For example: do the QP fields match, is the flow supported, and, for data packets, is the FSN within the receiver's window and not already delivered? If any of these checks fail, the packet is handled at the reliability layer without examining message-level or payload-level fields; it is dropped or rejected as appropriate, with an acknowledgment scheduled on the flow if needed. No field of a dropped packet is applied, including the UNA field. A packet delayed until after the sequence space wrapped would otherwise advance CumulativeFSN past data the receiver never took.

2. **Can the message be delivered?** For example: does the MSN correspond to a message already validated and in progress, or, if not, is it within hardware capability to track? If the receiver is creating a message context, does the CSN match a pending receive WQE? For writes, are the RETH fields valid? These checks require that the packet has a trackable FSN.

   A receiver should validate the MSN against the range of messages it is currently prepared to track: from the Cumulative-Responder-Delivered MSN (CDMSN, Section 3.4.4) up to its capacity to track concurrent in-flight messages, which may be smaller than the number a sender can place in flight at once. An otherwise-valid packet whose MSN falls outside this range is most likely a delayed or ghost packet, a sequence-number wrap, or a sender placing more messages in flight than the receiver can track. Such a packet should be rejected non-destructively with an RNR-class NAK (BRNR or HRNR); the offending FSN is not positively acknowledged, neither cumulatively nor in the FSN bit vector, and the QP remains operational.

3. **Do the contents of the packet match the message?** For example: does the POSN fit within the expected message length? These checks require a valid message context established by the preceding checks.

When a packet triggers multiple error conditions at the same evaluation level, the responder may report any of the applicable error codes. For example, a write with a zero-byte payload (NAK-Invalid Request)

targeting an address that is not valid for the R_Key (NAK-Address Invalid) may receive a NAK of either code. Implementations are not required to check all conditions or to prioritize one error code over another within the same level.

In the Handling column below, *WQE error* means the WQE is completed in error as described in Section 5.3.2, and *QP error* means the QP is placed in the error state defined in Section 5.3.4, which the QP does not leave without software returning it to Reset or destroying it.

A NAK that places the QP in error is reserved for a request the initiator should not have sent: an unrecognized or unimplemented opcode, a misaligned or unsupported atomic, or a zero-byte payload on an opcode that forbids one. An exhausted resource will free again, so the request is answered with an RNR-class NAK and a backoff, leaving the QP operational.

Error	**Description**	**Response**	**Handling**
FSN gap detected	Packet received with FSN beyond the next expected FSN. Packet loss can be inferred.	SACK with appropriate bit(s) not set can be scheduled	No further action.
FSN out of range	Packet received outside of the window.	Drop received data; Schedule acknowledgment on the flow	
MSN outside tracked range	Valid, in-window FSN but the MSN is outside the range the receiver can track (at or before the CDMSN, or beyond its tracking capacity). Most likely a delayed or ghost packet, a sequence-number wrap, or a sender exceeding the receiver's tracking capacity.	RNR (BRNR or HRNR, selectable) with backoff; do not positively acknowledge the offending FSN, neither cumulatively nor in the FSN bit vector	No further action; QP remains operational.
Hardware resource not ready	Valid request received but HW resources needed to process request are not available.	HRNR-NAK	No further action.
Buffer resource not ready	Valid request received but the RWQE needed to process the request is not available.	BRNR-NAK	No further action.
Unrecognized transport type	Opcode bits 7:5 are not "110"	Drop silently	No further action
Unrecognized Meta-RoCE opcode	Opcode bits 7:5 are "110" but bits 4:0 do not match a defined opcode	NAK-Invalid Request	QP error
Unsupported Opcode	A defined opcode that is not implemented by this device was received on this QP	NAK-Invalid Request	QP error

Local Memory Protection Error	Responder detected an L_key error or other local memory issue	NAK-Address Invalid or NAK-Operational Error	WQE error
R_Key error	Responder detected an R_Key error	NAK-Address Invalid	Message error not associated with a WQE
Memory region does not exist	Destination memory region referenced by R_Key and address does not exist	NAK-Address Invalid	Message error not associated with a WQE
Memory region not writable	Destination memory region is not writable (write or atomic to a read-only region)	NAK-Address Invalid	Message error not associated with a WQE
Memory region not readable	Destination memory region is not readable (read request to a write-only region)	NAK-Address Invalid	Message error not associated with a WQE
R_Key invalidation failure	Send with Invalidate received but the R_Key does not exist, is already invalid, or was not created with remote invalidation permission. Data may have been delivered to the receive buffer.	NAK-Address Invalid	WQE error
Remote Operation Error	Catch all for other errors detected by Responder and reported back to Requester with a NAK	NAK-Operational Error	WQE error
Local Operation Error - WQE	Invalid WQE contents (illegal opcode or other)	NAK - Operational error	WQE error
Forbidden zero-byte payload	Data packet with message length zero received for an opcode that does not permit zero-byte payloads (Table 8.2)	NAK-Invalid Request	QP error
Length error	Incoming send message packets are out of range of the receive buffer (POSN is too large or last frame is too large), exceeding the receive request length by more than the undisclosed padding a transmitter could have applied (Section 5.2.5). (There is no direct error for an undersized send message; messages of uncertain size should be passed with write or write/immediate.)	NAK - Invalid Request	WQE error

CQ overflow	Successfully executed WQE cannot be completed due to lack of space in associated CQ	Ack may still be sent, as message is complete from Sender perspective	CQ error
Invalid Atomic Request	Atomic request received with a misaligned address or an unsupported size or opcode	Nak - Invalid Request	QP error
Atomic/RDMA Read resources exceeded	Valid RDMA Read or atomic request received, but the number of outstanding read and atomic requests would exceed an implementation-defined limit, which may apply per QP, device-wide, or both.	HRNR-NAK	No further action.
FSN not acked. Retry limit not exceeded	Requester (of Read or Atomic) detected a missing response FSN.	None	Responder should retransmit.
FSN not acked. Retry limit exceeded	Requester (of Read or Atomic) detected a missing response FSN. Requester performed retries but all attempts failed.	None	Responder should attempt Flow Level Failover, if enabled, else WQE error
Response timeout error. Retry limit not exceeded	No response Ack from Requester for one or more packets on a flow within timer interval.	None	Responder may retry the request.
Response timeout error. Retry limit exceeded	No Ack response from Requester for one or more packets on a flow within timer interval. Requester performed retries but all attempts failed.	None	Responder should attempt flow level failover, if enabled, else WQE error.

5.5.1 Memory region validation

The responder must validate that the memory region referenced by an RDMA Write, RDMA Read, or Atomic request exists, is accessible with the provided R_Key, and has the required permissions (writable for writes and atomics, readable for reads). If validation fails, the responder sends a NAK with the Address Invalid code (Section 8.5.6).

5.5.2 Unrecognized opcodes

Opcode handling depends on the transport-type bits (7:5), which determine the format of the base header. The base header, which includes the source and destination QP, is common to every Meta-RoCE packet, so whether the receiver can construct a response follows from whether those bits are recognized:

- If bits 7:5 are not “110” (not Meta-RoCE), the base header format is unknown; no QP context can be recovered and no response can be constructed. The packet must be silently discarded.
- If bits 7:5 are “110” but bits 4:0 do not match a defined Meta-RoCE opcode, the base header, and thus the source and destination QP, is known even though the operation is not. The responder must send a NAK with the Invalid Request code, constructed from the base header and the receiver’s own connection state; it does not parse or rely on the unrecognized remainder of the packet.

Part III

Scheduling of Data Transmission: Flows, Congestion, Limits

Chapter 6

Congestion Control

Meta-RoCE congestion control algorithms must support managing traffic across many virtual flows on the same QP. As discussed in Chapter 7, different virtual flows are given different entropy values and are likely to take different physical paths through the network. Different flows may therefore have different round-trip times, congestion states, and other properties.

There are two congestion control (CC) modes in Meta-RoCE:

1. **Baseline CC:** (Required) A simple congestion control algorithm that reacts to explicit congestion signaling, defined in Section 6.1.

2. **Advanced CC:** (Recommended) An implementation-defined congestion control algorithm, using as input the signals defined in Section 6.2.

The output of the overall congestion control algorithm is a set of per-flow congestion windows that limit the number of unacknowledged packets that may be sent on each flow. The sum of these per-flow windows represents a per-QP window. The role of congestion control is to reduce unnecessary retransmissions and queuing delay that can lead to poor performance, allowing transfers to complete as fast as possible.

Both Baseline CC and Advanced/Programmable CC are hierarchical congestion window-based protocols, with windows defined at the per-flow level and the per-QP level.

Meta-RoCE also provides a mechanism for congestion avoidance, described in Section 6.3, where the receiver signals to the sender how fast it wishes to receive data. The sender converts this signal into a cap on its QP congestion window.

6.1 Baseline Congestion Control

The Baseline CC protocol in Meta-RoCE is a sender-driven, window-based congestion control protocol that responds to ECN and loss to determine the correct window size both at a per-QP and per-flow granularity. To permit ECN to work requires that all data packets be sent with the ECN-capable-transport setting using the conventional ECT(0) code point. (See IETF RFC 3168 section 5 for detail on the two ECT code points.)

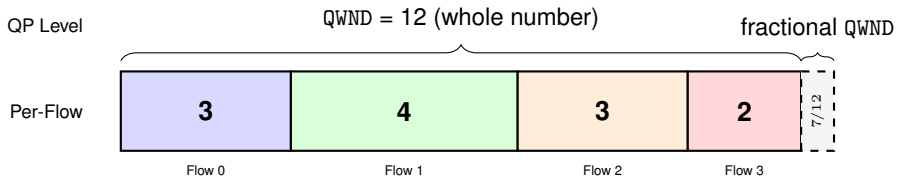


Figure 6.1: Decomposition of the QP-level window (QWND) into per-flow windows (FWND[f]). Each segment represents one flow's FWND value; the whole-number QWND equals the sum of all FWND[f] values ($3+4+3+2 = 12$). The dashed segment represents the fractional portion ($7/12$) used for tracking additive increase toward the next integer increment of QWND.

6.1.1 QP and Flow Windows

The two different granularities of windows in Meta-RoCE address two complementary aspects of traffic management on the network:

1. QP-Level Window (QWND): This window, which is responsive to congestion across all flows, limits the number of outstanding packets on the entire QP. QWND can be efficiently stored with two fields:

   (a) The whole-number portion, representing the packets that may be in flight at a time. In this design, fractional packets are not permitted to be transmitted or assigned to flows, so QWND typically refers to this whole number portion.

   (b) The fractional (numerator) portion, used only for tracking incremental changes to QWND, representing how close the QWND is to incrementing the whole number portion; when the fractional exceeds the whole-number, the fractional is reduced by the whole-number, and the whole-number is incremented by 1.

   Concretely, the overall QWND values are 1, 2, 2 $1/2$, 3, 3 $1/3$, 3 $2/3$, 4, 4 $1/4$, etc. This mechanism translates the "additive increase" goal, increasing by a constant number, ϵ, of packets per round trip time, into a per-packet operation, incrementing by ϵ/QWND on each ack. With a QWND worth of acks, integer QWND should increase by ϵ.

2. Per-Flow Windows (FWND[f]): These windows dictate the balance of traffic across available active flows for this QP. Figure 6.1 illustrates how the QP-level QWND decomposes into per-flow FWND values, with the fractional portion tracking progress toward the next integer increment. The FWND[f] may be limited to a configurable FWND_{max}.

In Meta-RoCE, whole-number $\text{QWND} = \sum_f \text{FWND[f]}$ at all times. When whole-number QWND is incremented, a flow window is also incremented.

Available Window

The available window determines whether a flow can send a packet. It is computed from three per-flow values:

- snd.nxt[f]: The lowest FSN that has not yet been sent on this flow
- snd.una[f]: The lowest FSN that has not yet been contiguously ACKed

- `snd.inflate[f]`: The number of FSNs between `snd.una[f]` and `snd.nxt[f]` that have been selectively ACKed or named in the RNR bitset. Both have left the network, so neither should occupy the window.

The available window for flow `f` is then:

$$\texttt{AWND[f]} = \texttt{FWND[f]} + \texttt{snd.inflate[f]} - (\texttt{snd.nxt[f]} - \texttt{snd.una[f]})$$

The QP-level available window is the sum across all flows: $\texttt{AWND.QP} = \sum_{\texttt{f}} \texttt{AWND[f]}$. If `FWND[f]` is reduced by more than the number of packets acknowledged, `AWND.QP` may be zero or negative even though some individual flows have available window.

To recover a loss, $\texttt{snd.nxt[f]} - \texttt{snd.una[f]}$ reaches about $2 \times \texttt{FWND[f]}$. The span is `FWND[f]` when the loss occurs, and the $\texttt{FWND[f]} - 1$ packets after it are selectively acknowledged into `snd.inflate[f]`, freeing window for that many more. Where the out-of-order window is limited to X frames, and smooth packet flow while recovering from loss is desired, $\texttt{FWND}_{max}$ should be configured to be no larger than $X/2$.

The use of `AWND.QP` and `AWND[f]` in packet scheduling is discussed in Chapter 7.

6.1.2 Baseline CC Theory of Operation

`QWND` and `FWND[f]` have two different goals: `QWND` limits the total outstanding data on the network, while the individual `FWND[f]` serve to balance traffic across flows. The Baseline CC algorithm defines a hierarchy of congestion control, where decisions about window increase or decrease are made based on QP-level state; changes to the whole number portion of `QWND` are then allocated to one or more flows. This is in contrast to, for example, a scheme where each flow runs its own CC protocol on its `FWND` and `QWND` is just the sum of the `FWND`s.

Congestion-Responsive `QWND` Control

Implementations of Meta-RoCE shall support the following baseline congestion control algorithm:

- On ACK of any packet that did not experience congestion, increment the fractional portion of `QWND` by ϵ, *unless*:
  (a) `QWND` is at or above the limit set by Rate Control Notification (RCN, Section 6.3), $\texttt{QWND} \geq \texttt{QWND}_{max}$ (Section 6.3.3), or
  (b) $\epsilon > \texttt{QWND}$, in which case the growth is comparable to TCP slow-start (increment `FWND` and `QWND` by 1 per acknowledged packet), or
  (c) the flow is not window limited. A flow is window limited when the number of outstanding packets on the flow at the time the ack arrived was constrained by `FWND[f]`, preventing new packets from being sent. Alternate methods of determining whether the flow is window limited are acceptable.

```
if flow f was window limited:
```

$$\texttt{QWND} = \min\left(\texttt{QWND} + \frac{\min(\epsilon, \texttt{QWND})}{\texttt{QWND}},\ \texttt{QWND}_{max}\right)$$

- On ACK of any packet that experienced congestion, decrement the fractional portion of `QWND` by $\beta \times$ `QWND`:

$$\texttt{QWND} = \max(\texttt{QWND} - \beta,\ \texttt{QWND}_{min})$$

This reduction can also be accomplished by decrementing the integer portion of QWND (and FWND) by one, while increasing the fractional QWND by $1 - \beta \times \texttt{QWND}$. The fractional (numerator) QWND will likely be greater than QWND, allowing the next flow to claim an increment in FWND. The effect is to maintain the same QWND-level response while both probabilistically shifting packets off of the flow that experienced congestion to another flow and slightly delaying the next packet transmission.

- On detection of the loss of N packets, apply a similar reduction using λ instead (λ for loss, β for the ECN-marked case above):

$$\texttt{QWND} = \max\left(\texttt{QWND} - \lambda N,\ \texttt{QWND}_{min}\right)$$

If a full QWND of packets is ACKed without experiencing congestion, QWND will additively increase by ϵ. Similarly, if a full QWND of packets is ACKed, and every packet experiences congestion, QWND will multiplicatively decrease by a factor of β. That is, the per-packet reaction of β achieves a multiplicative decrease of $\beta \times \texttt{QWND}$ per round trip time, comparable to TCP's multiplicative decrease but applied incrementally on each acknowledgment rather than as a single reduction event. $\epsilon = 1,\ \beta = \frac{1}{2},\ \lambda = 1$ is a reasonable starting point. These parameters are summarized in Table 6.1.

A small value of β or λ may lead to no reduction in even the fractional QWND when QWND is already small, i.e., $\beta \times \texttt{QWND} < 1$. This is intentional, encouraging an alternate uncongested flow to be used.

Parameter	Default	Min	Max	Description
ϵ	1	1*	100	Per-uncongested-packet fractional window increase
β	$1/2$	$1/8$†	1	Per-congested-packet window reduction
λ	1	$1/8$ †	1	Per-lost-packet window reduction
fast_start_burst	4	1	100‡	Packets to send in a burst per flow in fast start

*Configuration should not allow values below 1 unless an alternate congestion control algorithm is in effect. Advanced/Programmable CC may set this value to zero to completely control windows.

†Configuration should not allow values below $1/8$ unless an alternate congestion control algorithm is in effect. Fractions may be stored as the operand of a bit shift; Advanced/Programmable CC may set this value to a very small fraction (in practice, a large bit shift) to completely control windows.

‡This maximum is illustrative; an implementation may impose a smaller maximum.

Table 6.1: Baseline Congestion Control Parameters: Global defaults that may be altered on a per-QP basis by advanced/programmable CC.

Updating `FWND[f]`

To maintain the invariant that `QWND` $= \sum_f$ `FWND[f]`, any change to the whole-number portion of `QWND` must be reflected in one or more `FWND[f]`.

For example, if `QWND` was 5 and fractional `QWND` 4, the overall `QWND` is $5\frac{4}{5}$. If the QP used $\epsilon = 1$, an ack will increment the `QWND` to $5\frac{5}{5}$. Converting `QWND` to $6\frac{0}{6}$ occurs with an increase to a `FWND`, which is referred to as the flow "claiming" the window increase below.

Maintaining the invariant is achieved with the following rules:

- On ACK of a packet from flow `f` that did not experience congestion when `QWND` is less than `QWND`$_{max}$ and `FWND` is less than `FWND`$_{max}$, if the fractional `QWND` increase makes it greater than or equal to `QWND`'s whole number portion, a flow window may increase. Whether the whole-number `QWND` is incremented and, if so, which flow window is increased can be determined by the implementation. A flow may be ineligible to claim the increase if its measured round trip time is relatively high (item 3) or if its `FWND[f]` has reached `FWND`$_{max}$. If no flow is eligible, fractional `QWND` may remain greater than integer `QWND`.

  1. As described in Section 7.4.2 below, if there is a flow being bootstrapped, the bootstrapping flow must claim the new flow window, unless a smaller eligible flow claims it.
  2. A simple default is for the flow that increments the fractional `QWND` above integer `QWND` to claim the increase.
  3. To aid in load balancing, a flow must be able to skip claiming the increase if the recent observation of round trip time is higher than the known smoothed round trip time. Restated, if the acknowledgment provides evidence that the flow is more loaded than typical, a different path should claim the increment. This capability is required but may be disabled in operation.
  4. Optional: To reduce burstiness, a flow may be selected for `FWND` increase by other means, e.g., round-robin. Eagerly crediting another flow avoids sending two consecutive packets to a single flow, but requires keeping track of a second eligible flow.
  5. Optional: Alternately to reduce burstiness, a flow may claim the window increase (incrementing integer QWND and `FWND`) but delay sending for a pacing interval, $RTT/(2\times$`FWND`$)$, before sending the new packet.

- On decrement of the whole number portion of `QWND` due to detected congestion or loss, decrease `FWND[f]` by the same amount. If a single flow is congested, only that flow will reduce. If that flow window reduces to zero, the flow is deactivated as described in Section 7.4.

This design probabilistically allocates `QWND` increases to flow windows in proportion to their number of packets in flight, for those flows that are not congested. Congested flows will miss the chance to increase the fractional portion across the boundary. The most recent flow to have an (uncongested) acknowledgment is the best candidate for additional window.

6.1.3 Fast Start

Selective ACK loss recovery on flow `f` is only useful if `FWND[f]` > 1. Fast Start balances exploring additional flows against having multiple packets on each flow, while transmitting data at the beginning of an operation before acknowledgments have been received.

Many packet-to-flow schedules are valid toward accomplishing the goal. A straightforward one uses a defined constant, `fast_start_burst` to represent the number of packets to put on a flow before starting to send on a new one. Figures 6.2, 6.3 and 6.4 show three such schedules.

The fast start algorithm is as follows:

ENTRY When a Send, RDMA Write, or RDMA Read Response is pushed onto the send queue on an idle QP, that QP enters fast start mode.

INITIAL STATE All `FWND[f]` and `QWND` are set to zero.

WINDOW ADJUSTMENT Increment `FWND[f]` and `QWND` values as packets are sent, in contrast to baseline that may add as acknowledgments are received. The QP is permitted to transmit data, as if the available window is $\texttt{QWND}_{max}$, until the EXIT condition.

WHICH FLOW Fast start should begin on flow 0, which every QP has (Section 7.3). Where a QP's flows span more than one port, fast start must begin a flow on every available port. Modes for choosing the next flow are described in Figures 6.2, 6.3 and 6.4; if using fast start burst (Fig 6.2), a round robin counter of flows can advance if the current `FWND` is greater than fast start burst.

EXIT Exit fast start mode upon any of:

- receiving an ack on any flow while there is pending data to be sent,
- when receiving an ack on any flow indicative of congestion (nonzero CNP field in the ack indicating data was ECN marked),
- when $\texttt{QWND} \geq \texttt{QWND}_{max}$, or
- after an estimated RTT has elapsed since the beginning of fast start, while there is pending data to be sent.

The pending-traffic qualifier avoids exiting fast start prematurely. In a negotiation sequence of a small request, a small reply, then a large message, the large message benefits from continued fast start rather than a window sized for the small messages. If the sender exhausts its data before an acknowledgment returns, fast start has not yet estimated the network capacity.

The estimated RTT is the one the QP already holds, which fast start re-entry preserves (Section 4.5.1). A QP that has not yet measured one uses a configurable default, which may be seeded from the QP timeout (Section 10.4) or set by the driver. This condition bounds how long a sender transmits with no feedback at all.

UNUSED AFTER FAST START If not all flows are visited in fast start mode, those flows with zero `FWND[f]`will not be used until `QWND` grows after fast start to a value suitable for adding a new flow, described in Section 7.4.

REENTRY A QP having exited fast start may determine whether to re-enter by policy.

- A QP is not required to re-enter fast start when the QP is idle; a previously idle QP would restart with the `FWND[f]`and `QWND` it had previously. Optionally, a QP may re-enter fast start after being idle for a configurable interval (`fast_start_idle_interval`, in ms).
- A QP may re-enter fast start mode if it has discarded congestion state (`QWND`, `FWND`) (e.g., if QP state was restored from a cache that does not keep that information).

During fast start, the window increments on transmission rather than on acknowledgment: each packet sent increases `FWND[f]` and `QWND` for the flow it is placed on. This continues until an acknowledgment arrives or the exit condition is reached.

Once an acknowledgment arrives on a given flow, that flow's window has room for additional packets and transmission on that flow may continue. However, flows that have not yet received an acknowledgment remain window-limited to the packets already in flight; the sender does not send additional packets on those flows until their own acknowledgments arrive. Acknowledgments may arrive out of order across flows—an acknowledgment on flow 2 may arrive before the acknowledgment on flow 0. If flow 0 is slow or congested, it is correct to stop transmission on flow 0 until an acknowledgment advances that flow's window. This per-flow gating ensures that fast start does not continue to load a flow that is not making progress.

Flow	F0	F1	F2	F3	F4	F5	F6	F7
FSN-0	0	4	8	12	16	20	24	28
FSN-1	1	5	9	13	17	21	25	29
FSN-2	2	6	10	14	18	22	26	30
FSN-3	3	7	11	15	19	23	27	31
FSN-4	32	33	34	35	36	37	38	39
FSN-5	40	41						

Figure 6.2: Transmission order in fast start using the fast start burst. The QP-configured flow limit is 8, and fast start burst is 4. The message comprises 42 packets, with POSN 0 through 41. Once all flows have fast start burst packets, fast start transitions to normal round robin across flows.

The exit criteria for fast start are intended to apply when an RTT worth of transmission has been initiated. An acknowledgment provides an explicit signal that an RTT has passed. Exiting after an estimated (or in the absence of prior conversation, a default) RTT protects against sending excessive data toward an unresponsive receiver. Exiting when `QWND` reaches $\texttt{QWND}_{max}$ respects the receiver's rate limit, but is only practical when there has been prior conversation or there is a reasonable default. Exiting when out of data to send is not recommended; if used, a second send or write message that is queued after a short delay may have to wait for the first to be acknowledged and will be window limited to the size of the first message.

Port	0	1	0	1	0	1	0	1
Flow	F0	F1	F2	F3	F4	F5	F6	F7
FSN-0	0	1	8	9	16	17	24	25
FSN-1	2	3	10	11	18	19	26	27
FSN-2	4	5	12	13	20	21	28	29
FSN-3	6	7	14	15	22	23	30	31
FSN-4	32	33	34	35	36	37	38	39
FSN-5	40	41						

Figure 6.3: Transmission order in fast start where two output ports are used for the same QP. The port associated with a flow in this example is flow number mod 2: even flows on port 0, odd flows on port 1. (The assignment of flows to ports should prevent inherent load imbalance; in this example, another QP should use flow number plus one mod two to start with port 1.) Fast start places a packet, round robin across ports for the first flow of each port, until the new flow on each port has reached fast start burst, then proceeds to the next set of flows, etc.

Flow	F0	F1	F2	F3	F4	F5	F6	F7
FSN-0	0	1	5	10	16	23	31	40
FSN-1	2	3	7	12	18	25	33	42
FSN-2	4	6	9	14	20	27	35	
FSN-3	8	11	13	17	24	30	38	
FSN-4	15	19	21	22	28	34		
FSN-5	26	29	32	36	37	39		
FSN-6	41							

Figure 6.4: Interleaved transmission order in fast start. The objective is to automatically scale the number of used flows to the size of the message while avoiding sending consecutive packets on the same flow. The first POSN for a flow is arranged as a lazy caterer's sequence after 0 and 1 (+4, then +5, then +6, then +7, etc.). The second POSN for a flow is two after the first. Other spacing between POSN on the same flow is at least the FSN (i.e., FSN-3 is at least three POSNs after FSN-2). The next POSN, 43, would go on flow 6, FSN-4 because that is the lowest FSN where it is allowed.

6.2 Advanced/Programmable Congestion Control

This section specifies the capability a Meta-RoCE implementation provides for a congestion control algorithm defined outside this specification. A deployment that finds the baseline sufficient need not define such an algorithm. Where an algorithm is defined, its use is negotiated out of band. Section 6.2.1 puts limits on how it operates and Section 6.2.2 lists the signals available to it.

Advanced congestion control is meant to enable adaptation to new environments and new objectives. A deployment may share the network with traffic under a different congestion control, or may weight fairness, loss avoidance, and performance goals differently. A device implementing the rest of the transport should leave room for a different algorithm.

Programmable logic should be able to modify `QWND` and `FWND` as well as parameters used by the baseline algorithm. To take complete control over congestion response, programmable logic may set ϵ to zero, which disables the baseline increase, and may disable the baseline reduction on a congestion mark and on loss. An algorithm that reduces only where a mark or a loss coincides with high delay needs those reductions disabled. The programmable congestion control algorithm may instead set these parameters to ensure continuous automatic reaction to congestion changes while altering how aggressive the reaction to normal events is.

Representative software-defined congestion control approaches include:

- TCP Cubic-like response. In Cubic, the size of the congestion window depends on the time T since the last congestion event and the size of the window at that point $\texttt{QWND}_{congestion}$. This increases fairness between flows of different round trip times because the increase is a function of time rather than of acks received. Because the cubic algorithm would overwrite `QWND` and `FWND` for multiple flows, leaving the baseline CC in place to be periodically overwritten is likely acceptable.

$$\texttt{QWND} = 0.4 \times \left(T - \sqrt[3]{\frac{0.3 \times \texttt{QWND}_{congestion}}{0.4}} \right)^3 + \texttt{QWND}_{congestion}$$

- TCP Hybla-like response. Hybla was built to improve performance of satellite links, but effectively it sets ϵ to scale with RTT, where, for example, if ϵ is 1 for 5μs connections, it would be 2 for 10μs connections and 20 for 100μs connections. With large networks, such adaptation might enable better fairness between 1-hop, 3-hop, and 5-hop flows.

- Swift-like response. Swift is delay based and makes a point of not using ECN. With a computed target delay, it allows additive increase only when the delay is less than the target, and decrease once per RTT if above the target. A Swift algorithm could be approximated by: a) reducing the window in programmable logic based on the difference between RTT and target delay when RTT is greater, b) temporarily setting λ to zero if reduced, and c) temporarily setting ϵ to zero if delay is above the target, nonzero otherwise. It would take substantial effort to independently adjust ϵ and λ on specific flows. Swift requires some complexity in computing the target delay: It is the sum of a constant added for each “hop” and a function that lets small windows work through large queues $\frac{1}{\sqrt{\texttt{QWND}}}$ (the target delay decreases as `QWND` increases).

6.2.1 Constraints

Meta-RoCE relies on a framework of congestion-window-based control schemes, which all Advanced CC implementations must adhere to. An Advanced CC algorithm sets the baseline parameters ϵ, β, and λ

and the rate control parameters γ and ω per queue pair, which shape the window response over the life of the connection. It may also set `QWND` and `FWND[f]` directly for an immediate change. The invariant `QWND` $= \sum_f$ `FWND[f]` holds however the windows are set.

Implementations *should not* run an independent congestion control protocol per flow.

Intended use of the controls. An Advanced CC algorithm evaluates a connection occasionally rather than on every acknowledgment, so its work is to set the connection up to succeed over the next several round trips. Adjusting ϵ is the primary means. A high-delay path calls for a larger ϵ, since fewer round trips complete in a given time and each must carry more of the growth. Small delay and a history of no congestion marks likewise support a larger ϵ. Reducing ϵ, γ, and ω, or increasing β and λ, slows growth and deepens the reaction to congestion.

Writing `QWND` and `FWND` directly changes the windows at once. Selectively increasing multiple `FWND[f]` reacts quickly to newly available bandwidth. Reducing `FWND` directly is likely to be appropriate only for small, likely single packet, reductions. The reason not to prefer direct `FWND` adjustment is that it has the potential to create bursts on a sudden increase and reinforce them on a sudden decrease.

Where an implementation provides a per-QP rate limiter, it serves to damp transient bursts within sustained traffic. The configured rate is expected to exceed the average rate given by `QWND`/`RTT`, tightly to focus on burst control, loosely to permit immediate transmission. Rate control remains the work of the window-based control of `QWND` and `FWND[f]`.

6.2.2 Available Signals

An Advanced CC algorithm may use any combination of the signals below.

QP-Level Signals, Required

1. `QWND` - full packet value; fractional value optional
2. At least one of (the other can be computed):
   - `AWND.QP`
   - `QWND` - `AWND.QP` (Data in flight)
3. `MINRTT.QP` (minimum RTT observed; may be cleared when read by the advanced CC algorithm or cleared after idle) (Cleared may be represented as a 32-bit -1 value or 0.)
4. `SRTT.QP` (smoothed RTT, from each ack, averaging in $1/\alpha_{qp}$ of the current value, per QP; this is the same as `RateWeightedRTT` defined in Section 6.3.2)
5. Cumulative packets delivered CE received since the last evaluation, aggregated from the CNP counts across acknowledgments. The name follows the Linux TCP sender variable that counts packets received with an ECN mark since the last evaluation.
6. Cumulative packets delivered since last evaluation (from multiple acknowledgments)
7. Time of last advanced CC evaluation
8. Retransmission count (may be cleared when read by the algorithm, or cleared after idle)

9. Timeout retransmission count (may be cleared when read by the algorithm, or cleared after idle)
10. At least one of (the other can be computed):
    - $\texttt{QWND}_{max}$, as calculated by hardware from the remotely specified TargetRate.
    - The raw TargetRate field from the most recent acknowledgment.

QP-Level Signals, Recommended

1. `MDEV.QP` (averaged difference between measured round trip time and `SRTT.QP`)
2. Approximate count of remaining packets to be sent on WQEs of this QP, to permit adjustment based on remaining data to send (where very large, stability might be preferred; where small, finishing the transfer quickly might be preferred). This may be approximated based on the pending packets on the current WQE and the number of WQEs remaining in the queue, without specific information about the size of WQEs that have not yet entered processing. It is not required to determine the number of packets that will be sent by WQEs that have not entered processing. The remaining packets to be sent does not include packets to be retransmitted.

Flow-Level Signals, Required

1. `FWND[f]`
2. At least one of (the other can be computed):
    - `AWND[f]`
    - `FWND[f] - AWND[f]` (Data in flight for all flows)
3. Cumulative delivered CE since last evaluation (aggregated from the CNP counts in multiple acknowledgments)
4. Cumulative delivered since last evaluation (from multiple acknowledgments)
5. Retransmission count (may be cleared when read by the algorithm, or cleared after idle)

Flow-Level Signals, Recommended

1. `SRTT[f]` (smoothed RTT, from each ack, averaging in $1/\alpha_f$ of the current value, per flow)
2. Timeout retransmission count (may be cleared when read by the algorithm, or cleared after idle)

Flow-Level Signals, Optional

1. `MDEV[f]` (average difference between the measured RTT and `SRTT[f]`, see Section 4.5.2)
2. Inbound TTL, which can be used by delay-based algorithms (e.g., Swift) to derive a target delay based on hop count through the network topology.

6.2.3 Algorithm state storage

The advanced CC implementation will need to store state from one invocation to the next, to determine whether an RTT has passed since the last recomputation, to store the prior value of variables to stabilize responses, or to alternate between internal states (e.g., whether the algorithm is temporarily experimenting with faster- or slower-than-expected transfers). At least 16 bytes of congestion state information should be provided; it may be discarded if the connection is idle or must be evicted from cache.

An implementation should also provide a device-wide telemetry space of 64 counters that the algorithm sets and software reads. The algorithm defines what each counter means: a device-wide count, for example how often a decision was made, or a value from a selected recently active queue pair. An algorithm may also mirror an input signal into this space, so that software sees the value the algorithm acted on.

6.3 Receiver Driven Rate Limiting

Incast at the final hop of the network is known to be one of the most likely places that congestion occurs. Additionally, the receiver knows how much bandwidth is available at this hop and how many senders are actively transmitting toward it. In Meta-RoCE, receivers include a value in every ACK that signals a `TargetRate` that they allocate to receive from the sender. This signal is the Rate Control Notification (RCN). It needs no packet of its own. The sender can then adjust the amount of data it sends to the network by setting $\texttt{QWND}_{max}$ as described in Section 6.3.2 to converge toward this `TargetRate`.

6.3.1 Signals Available to Calculate `TargetRate`

The `TargetRate` value is an expected "fair share" bitrate that alters how senders modify congestion windows to avoid problems of incast. That objective leads to a simplified formula, excluding clamps, tuning parameters, or any per-QP offsets for prioritization.

$$\texttt{TargetRate} = \left\lceil \frac{\text{Total Link Rate (Gbps)}}{\text{Estimated Concurrent Senders}} \right\rceil + \text{Target Rate Offset} \tag{6.1}$$

The "Total Link Rate (Gbps)" should be computed based on potentially changing factors:

- Where the interface has multiple ports that can fail independently, the total rate depends on the number of working ports.
- Where the interface has links with varying speed (e.g., can be run at either 400Gbps or 800Gbps), the actual link rate should be used.

The advertised `TargetRate` should also be capped at the maximum rate at which the receiver can process a single QP. This may be well below the link rate when QP processing is partitioned across independent pipelines or threads with QPs statically assigned, since no QP is served faster than its assigned pipeline or thread. Advertising the full link rate would then invite a window far larger than the receiver can drain, accumulating unprocessable backlog. The per-QP processing capacity, not the link rate, bounds `TargetRate`.

Signals primarily for computing Estimated Concurrent Senders

- Number of QPs that have RECV WQEs posted.

- Number of QPs that are actively receiving data on any opcode: a receive WQE, a virtual WQE associated with an inbound write, or an outstanding read request has received some but not all of its data.
- Number of QPs where most recently received rate hints field is non-zero *and* (from the item above) there is a WQE that has received some but not all data.

The third signal is the most precise by including only the senders that are likely to continue sending. It is a rough lower bound on how many flows will share the inbound bandwidth in the next round trip time. The other options provide rough upper bound estimates of what could appear. For workloads with very short messages, the lower bound alone may provide a poor estimate.

Signals primarily for computing Target Rate Offset

The Target Rate Offset computation is implementation-defined; the following signals are recommended inputs.

- Cumulative ECN CE observed since last evaluation, aggregated across incoming data packets. This information is meant to help measure whether the recent target rate appears to be an overestimate, leading to a higher than expected fraction of congestion marked packets, or an underestimate where no congestion is observed.
- Cumulative received data packet count since last evaluation (aggregated across incoming data packets). This is meant to help estimate whether a large or small fraction of traffic was CE marked, and whether the inbound utilization is high or low. If low and uncongested, the Target Rate Offset will likely increase.
- Observations of buffer buildup in delivering data to memory that reduce the effective rate of the network interface (e.g., due to bus contention).

6.3.2 Converting `TargetRate` to a `QWND` cap

The rate of flow `f` is `FWND[f]`/`RTT[f]` and the overall rate of the QP is the sum of the individual flows' rates:

$$\texttt{EffectiveRate} = \sum_f \frac{\texttt{FWND[f]}}{\texttt{RTT[f]}}$$

Meta-RoCE implementations shall calculate `RateWeightedRTT` so that

$$\texttt{EffectiveRate} = \frac{\texttt{QWND}}{\texttt{RateWeightedRTT}}$$

is also true, and the implementation can ensure that the transmitting QP's `EffectiveRate` $\approx$ `TargetRate` by setting

$$\texttt{QWND}_{max} = \max(\texttt{QWND}_{min}, \gamma \cdot \texttt{TargetRate} \cdot (\texttt{RateWeightedRTT} + \omega)).$$

The parameters are as follows:

- $\gamma \geq 1$ permits an increase to the limit that is proportional to RTT. This reduces the influence of RCN by creating a looser limit, relying more on ECN based congestion control. Increasing γ reduces the chance of underutilization at the cost of increased network buffer use. γ should never need to exceed 2, and is likely best at 1.0.

- $\omega \geq 0$ allocates a fixed headroom in units of delay, and may account for processing delays that are NIC based and do not feature in the network RTT estimates, for example, the delay in fetching packet payload via DMA.

- `RateWeightedRTT` compensates for the different `RTT[f]` and `FWND[f]` across flows. `RateWeightedRTT` is measured by sampling the RTT on each flow at a frequency of every M packets (where M does not vary by flow) and averaging all samples at the QP level. See Section 4.5.1 for the wire protocol method of measurement.

- `TargetRate` is the most recent desired rate received from the destination, reduced if necessary to the local output capacity. Typically, networks will be symmetric and no reduction will be needed: all interfaces in a cluster have the same capacity; however, under partial failure conditions, e.g., if a multi-port transmitter is working despite a failed port, it may be able to transmit only at a smaller rate. If the interface does not have the capacity to send at the receiver's desired rate, the correct window should be based on the sender's capacity.

An acceptable alternative to `RateWeightedRTT` would use the minimum observed RTT across any flow of the QP, which has the advantage of being more stable (does not increase with load) but a disadvantage in that it requires that the minimum RTT be representative of unloaded latency.

6.3.3 Reduction to target rate

When a sender notices that it is sending above the RCN-defined limit, i.e., $\texttt{QWND} > \texttt{QWND}_{max}$, it must reduce `QWND` on every ack. `QWND` is not allowed to grow above $\texttt{QWND}_{max}$, but $\texttt{QWND}_{max}$ can fall below `QWND` when `TargetRate` or `RateWeightedRTT` decrease, or when a local port fails. A reduction smaller than a whole packet moves only the fractional portion, leaving `FWND[f]` unchanged until it carries. The sender's latitude is in γ and ω, which set how $\texttt{QWND}_{max}$ follows the target rate, not in whether to converge on it. Figure 6.5 shows how the reduction behavior changes as `QWND` moves through three zones relative to $\texttt{QWND}_{max}$. There are four modes:

- **Protected flow**: ($\texttt{QWND} > \texttt{f} \times \texttt{fwnd}_{floor}$ and $\texttt{FWND[f]} \leq \texttt{fwnd}_{floor}$) the flow with id `f` may not have its `FWND[f]` reduced further by RCN. There is sufficient `QWND` for the flow to remain active, but the flow is at or below $\texttt{fwnd}_{floor}$. The first half of the expression uses the flow ID to deterministically protect low-numbered flows; protecting the flows that happen to be active and have window above $\texttt{fwnd}_{floor}$ would require more complex logic.

- **Waiting phase**: ($\texttt{QWND} > 2 \times \texttt{QWND}_{max}$) reduce `QWND` by one packet per acknowledged packet. No new packets will be sent while acknowledgments are received.

- **Rate-halving phase**: ($2 \times \texttt{QWND}_{max} \geq \texttt{QWND} > \texttt{QWND}_{max}$) reduce `QWND` by one half packet per acknowledged packet. One new packet should be sent for every two packets acknowledged.

- **Normal**: ($\texttt{QWND}_{max} > \texttt{QWND}$) no reduction necessary.

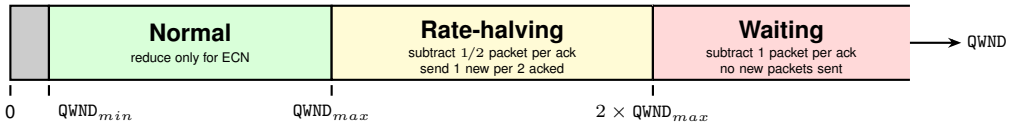


Figure 6.5: Rate reduction zones as a function of QWND relative to QWND$_{max}$. In the normal zone, no reduction is applied. In the rate-halving zone, QWND decreases by a half packet per acknowledgment and one new packet is sent for every two acknowledged. In the waiting zone, QWND decreases by one packet per acknowledgment and no new packets are sent. The protected-flow condition (not shown) independently prevents reduction of a flow's FWND below fwnd$_{floor}$.

An implementation should not proactively reduce QWND directly to QWND$_{max}$ by iterating across all FWND[f]. Doing so does not subtract packets from the network while they are in flight, and would limit the speed of recovery if QWND$_{max}$ is restored. An implementation may attempt to additionally process congestion signals, e.g., to continue to reduce by a full packet even in the rate-halving phase if an acknowledgment contains a congestion signal. Doing so may aid in avoiding congested flows, but the reduction by at least one half packet in response to being over QWND$_{max}$ is likely to already be the maximum response to an ECN signal.

The QWND$_{max}$ value is sensitive to round trip time; a sender must not update the RTT component of QWND$_{max}$ until after processing any per-flow FWND[f] reduction. This is necessary to prevent a Meta-RoCE flow that has an uncongested physical network path from reducing the smoothed round trip time, thus reducing the QWND$_{max}$, and then incorrectly reducing its own FWND[f]. Conversely, a flow with a large measured round trip time would increase the smoothed round trip time, increasing the QWND$_{max}$ and avoiding a likely helpful reduction in its FWND[f]. By recomputing QWND$_{max}$ later, this source of traffic imbalance can be avoided.

The fwnd$_{floor}$ value configures a minimum FWND for each flow that RCN reduction does not go below, except that high-numbered flows may be given FWND $= 0$ to permit reducing QWND. For example, assume fwnd$_{floor} = 2$ and QWND$_{max} = 8$. This rule prevents reduction of flows 0–3 below FWND $= 2$. Flow 5 may be reduced when QWND is 10. Flow 0 may never be reduced by RCN to be below fwnd$_{floor}$, since QWND is always greater than 0. Flow 1 may be reduced below fwnd$_{floor}$ only if QWND $\leq$ fwnd$_{floor}$. A reasonable setting of fwnd$_{floor}$ is between QWND$_{min}$and fast_start_burst. The term "floor" indicates that the window can descend below this value in some conditions, as opposed to "min" where it cannot.

6.3.4 Interaction with ECN Marking

While reducing to the target rate, the window is reduced in Section 6.3.3 by a packet per one or two acknowledged packets, depending on whether in the rate-halving phase. If ECN marks are also calling for a window reduction, they should be ignored. For example, in the rate-halving phase, when an acknowledgment for two packets is received, the sender reduces the window by only one packet, not by one for the rate-halving and another for the two congestion marks.

When both signals are active (QWND $>$ QWND$_{max}$ and CNP is nonzero in acks), both the destination and the network are communicating that the sender's window is too large. The QWND$_{max}$ provides an estimate of what the correct window should be. The CNP signal is, in this case, redundant, as the sender already knows that it is expected to slow toward a target rate.

Once reaching QWND $\leq$ QWND$_{max}$, it is likely that further acknowledgments will continue to carry ECN marks as the overloaded in-network queue continues to drain, or additional contending traffic dissipates. Depending on in-network factors, this continued congestion signal may cause QWND to temporarily fall below

its target. Larger additive increase (larger ϵ), reduced response to ECN (smaller β), packet pacing, or Section 6.2 advanced congestion control may be used to limit underutilization during convergence.

Chapter 7

Multipath and Flow Selection

To load balance traffic and reduce congestion in the network, Meta-RoCE spreads data from a single QP across multiple Meta-RoCE flows. Each Meta-RoCE flow uses a distinct entropy value, making it likely that different Meta-RoCE flows traverse different physical paths through the network; however, the mapping is not guaranteed to be one-to-one, and multiple Meta-RoCE flows may share a physical path. As described in Chapter 4, loss detection and recovery happen along each Meta-RoCE flow. When the network is configured to keep packets having identical entropy values on the same physical path (i.e., ECMP), Meta-RoCE can adapt per-flow rates to localized congestion and failures to balance load.

While implementations and network configurations may select other fields, the UDP source port should be the default choice to enforce this ordering and entropy requirement across flows, allowing for typical 5-tuple hashing on `(srcIP, dstIP, srcPort, dstPort, protoNum)`.

A transmitter changes a flow's entropy value during operation, seeking a different path when the current one is failing or congested. A flow is identified by its Flow ID, a fixed property that scopes the flow sequence numbers within it. An implementation must be able to change the entropy value while the Flow ID and the flow's reliability state continue unchanged. What steers a flow is changeable, and what identifies it is not.

7.1 Selection of Flow Entropy Values

When a QP is first created, or the number of flows on the QP changes, an entropy value must be selected for each flow. The recommended approach is to maintain a device-global counter over the ephemeral UDP port range 49152 to 65535, providing 14 bits of entropy for a given connection. When a flow is created and assigned to a QP, or the entropy value of a flow changes, the value of this counter is used and the counter is incremented by 1. The entropy counter and value can be maintained as a 14 bit counter that wraps, and when it is used, the top two bits in the UDP source port are set to one. Alternatively, the flow entropy value can be maintained with all 16 bits, and when wrapping to zero, it is set to 49152 (0xC000). The UDP source port is the conventional carrier of the entropy value; other hashed fields, such as the IPv6 flow label, may serve instead.

Implementations may choose to maintain this counter at the QP level. Implementations should not use random values; the counter approach is more likely to, with minimal state, reduce reuse of a value.

7.1.1 Acknowledgment Entropy Values

The acknowledgment should use the same source port as the data frame that is being acknowledged. This ensures that the entropy value associated with the return direction changes when the forward direction's entropy changes. When changing entropy values is necessary to avoid a temporary in-network fault, matching the acknowledgment's entropy allows the change to address faults on the return physical path as well as the forward physical path. Where this source port is not available, an implementation may use a function of the source port to select the source port; it is required only that the acknowledgment's source port change with high probability when the data packet's source port changes.

That is, acknowledgment generation should use header fields from the incoming data packet as illustrated in Figure 7.1.

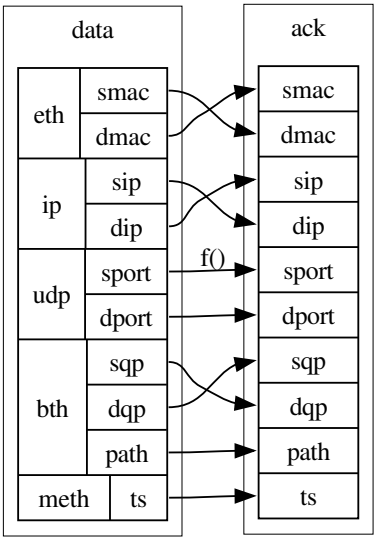


Figure 7.1: Acknowledgment generation may use received data packet headers to construct the response. Meta-RoCE uses a designated destination UDP port, meaning that data and ack destination UDP ports will be identical. One method of ensuring ack entropy values change with the sender's data entropy is to copy the source port from data to ack.

7.2 Transmit Packet Scheduling

Selection of which packet to send next requires a choice at two different levels: first across QPs and then across flows inside the selected QP. A QP is available for scheduling when $\texttt{AWND.QP} > 0$. A flow within an available QP is available for scheduling when $\texttt{AWND[f]} > 0$. (See Section 6.1.1 for definitions.)

A flow within its own available window may send even where $\texttt{AWND.QP}$ is not positive, since the sum can be driven below zero by other flows whose $\texttt{FWND[f]}$ was reduced below what they have outstanding. However, to promote fair use of transmit bandwidth across simultaneously ready QPs, the transmit scheduler

should rotate across ready QPs first, then among ready flows within the QP. A QP with a single flow that always has capacity should not be given fewer opportunities to transmit than a QP with many flows.

Implementations may define how many frames are sent per scheduling event, both at the QP level and at the flow level. Per-flow congestion state will be sampled more often the smaller this is; if possible, one packet should be sent on a single flow per scheduling visit to a QP.

7.3 Number of Flows per QP

Each QP is assigned a maximum number of flows and associated hardware resources that it is eligible to use. The assignment is done by the control plane when the QP is initialized. The number of active flows within this maximum number, and the corresponding hardware resources, can change dynamically. Flows are scoped to a QP: a flow ID identifies a flow within that QP and is not shared across QPs.

A limit on transmit flows may be distinct from a limit on receive flows. Only one mismatch has an effect: where the transmit flow count at one end of the QP exceeds the receive flow count at the other, the excess flows meet a flow ID unsupported NAK until the receiver allocates for them (Section 5.2.1).

Each direction must retain at least one flow; the flow count is never zero, so flow 0 is always present.

7.3.1 Flow context

The state an implementation holds for one flow is its *flow context*. Transmit and receive contexts are distinct, and a QP may hold different numbers of each. Each occupies hardware resources, which is why flow counts are limited and assigned when the QP is initialized. When a flow is inactive most of its context may be reclaimed, but the transmitter must retain snd.una and the receiver rcv.nxt or CumulativeFSN (Section 7.4.1).

Both transmit and receive contexts hold:

- **Identity.** The Flow ID, which is fixed and scopes the flow's sequence numbers (Section 2.5).
- **Steering.** What the direction currently sends with. At the transmitter, the data entropy and, on a multi-port QP, the selected port pair, each of which may change during operation (Sections 7.1 and 7.5.2). At the receiver, the acknowledgment's entropy and ports may be taken from each data packet, so that nothing is held, or may be held, for example as a cached acknowledgment header regenerated when the data's steering changes (Section 7.1.1).
- **FSN state.** The direction's position in the FSN space, and a tracker of which FSNs are outstanding, delivered or refused (Chapter 4). At the transmitter the tracker binds each outstanding FSN to the packet it carried, *(QP,Tag,Opcode,MSN,POSN)* (Section 2.5), so that a retransmission can be rebuilt and an acknowledgment attributed to its message. At the receiver it holds the Cumulative FSN, Max FSN and bit vectors that acknowledgments report (Section 4.2.3).

A transmit context also holds:

- **Retransmission.** The retransmission timer, the retransmission cursor snd.cur (Section 4.6), the count of timeouts on the current entropy value (Section 4.8), and, where kept per flow, the smoothed round trip time, its deviation and the timeout derived from them (Section 4.5.2).
- **Window.** `FWND[f]` and `AWND[f]` (Chapter 6).

- **Activation.** The flow's state in Figure 7.2.

A receive context also holds:

- **Congestion feedback.** The count of CE-marked packets received since the last acknowledgment (Section 8.5.1).
- **Pending acknowledgment.** Conditions that require an acknowledgment not yet sent (Section 4.2.2).

7.4 Adding (activating) and Removing (in-activating) Flows

The specific policy used to determine when and how to add a flow and when to remove a flow after fast start (Section 6.1.3) is complete is up to the implementation, and may depend on hardware resource availability. Figure 7.2 shows the sender-side state machine that governs these transitions, including activation on a new message, acknowledgment-driven state changes, and timeout-driven recovery.

7.4.1 Inactive transmit flows

A flow f is necessarily removed from the active set of flows when its `FWND[f]` is decremented to zero. It should return to a pool of "inactive" flows from which it can be reused. If decrementing `FWND[f]` would also decrement `QWND` below $\texttt{QWND}_{min}$, both windows must not be decremented, and the flow must remain active.

A flow not visited during fast start is similarly "inactive".

Inactive flows have no packets in flight (all packets have been acknowledged, or repeated timeouts have led to a clear indication of failure), and their resources may be reclaimed. At the transmitter, the next usable FSN (snd.una, Section 4.4.1) must be retained, and suffices to reconstitute snd.nxt and the empty array of unacknowledged FSNs; a flow returning to the active set continues its sequence rather than restarting it. Receivers may infer that a flow is inactive when all in-order data has been acknowledged, and must retain rcv.nxt or CumulativeFSN, which suffices to generate correct cumulative acknowledgments for well-delayed data or where the flow has appeared inactive due to an in-network fault.

7.4.2 Activating transmit flows

An implementation must provide a mechanism for activating previously unused flows, allowing path diversity to increase as the rate does. The recommended mechanism determines that another flow should be made active, chooses the flow, bootstraps the chosen flow to a reasonable starting window, and completes the process to be able to move on to the next new flow. Its steps are described in more detail below.

- To determine that a flow should be made active, implementations may determine whether adding a flow would make the average `FWND[f]` closer to a target `average_window`.

  $$\texttt{average_window} \times (\texttt{active_flow_count} + 1) < \texttt{QWND}$$

  and

  $$\texttt{active_flow_count} < \texttt{configured_max_flow_count}$$

  That is, on average, can enough packets be kept in flight to keep `average_window` packets on every flow after adding a new flow. The `average_window` value may be limited to be a small power of 2 (1–5) to simplify the multiplication operation into a shift.

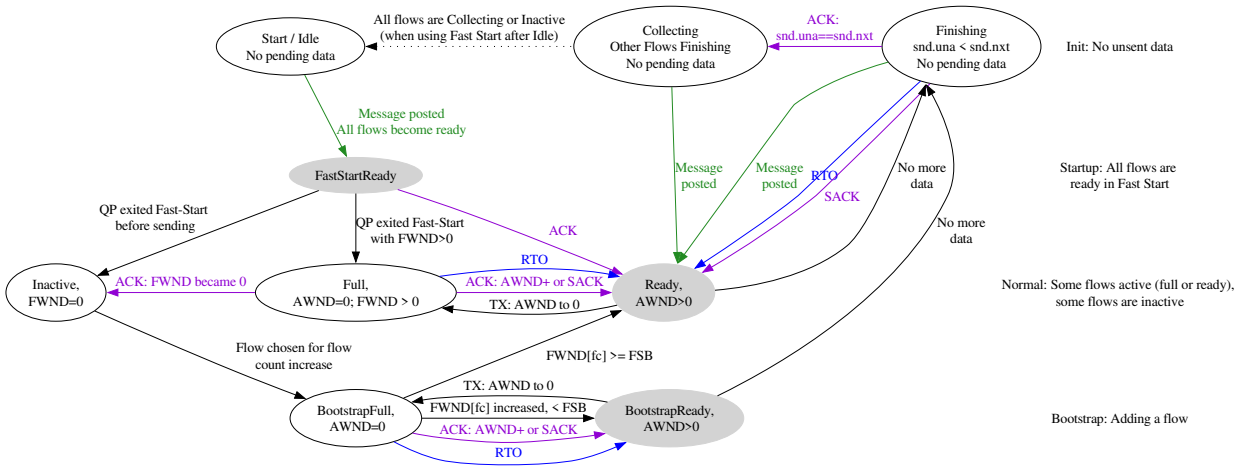


Figure 7.2: Flow state machine, sender side. All flows start in the "idle" state. Flow states that represent a flow that is available for scheduling are shaded. Transitions on a new transmission WQE being posted appear in green, labeled "Message posted." Transitions that occur on an acknowledgment, including SACK indicating a need for retransmission, for this flow are colored in violet. Transitions on retransmission timeout (RTO) firing appear in blue. Some edges omitted for clarity; e.g., direct transition to Finishing is possible from Full or FastStartReady states. Self edges are omitted for clarity; e.g., ACKs received in state "Ready" will leave the flow in that state.

For example, a QWND of 50 should have at least four flows with a target average flow window of 12: $12 \times (3 + 1) < 50$. There may be more; this step indicates only that 3 is insufficient.

- To choose a flow, implementations should prefer the numerically lowest among configured available flows. This becomes the *chosen flow*, f_c. A flow claimed in this way should have a new entropy value (UDP source port).

- To bootstrap f_c, the sender must credit `FWND` increments to `FWND`$[f_c]$ until it reaches `fast_start_burst`, or until a congestion signal arrives on f_c. A flow that is otherwise eligible for the increase and whose `FWND[f]` is below `fast_start_burst` may claim it instead, being no larger than the size f_c is growing to.

- To promote the flow to fully active, the state variable remembering that f_c is the chosen flow is reset.

- If no congestion has been observed, the steps may now repeat, as the overall window has increased by the `fast_start_burst` packets required to activate a new flow.

Additional optimizations or constraints are possible, e.g., to abandon the chosen f_c if any congestion is observed on this new flow. The threshold for adding new flows (i.e., the expected average window) may increase when resources are scarce.

7.5 Multi-port Interfaces

Meta-RoCE expects that a QP belongs to a network interface device, but that the Meta-RoCE flows of that QP may be associated with different physical ports connected to different network switches. Multiple physical interface ports may connect the Meta-RoCE device to the network, and the Meta-RoCE transport protocol determines on which physical interface port each packet will be sent.

There are two modes:

multi-port , in which each port on a sender may be configured to send across a network that enables directly addressing one or more ports at the receiver; different flows through the network have different source and destination addresses for the same connection, and

multi-plane , in which each port on a sender is only able to send to exactly one port at the receiver; different flows through the network on distinct planes may use the same source and destination addresses.

The Meta-RoCE protocol is designed to enable both network designs.

This version of the Meta-RoCE protocol expects to enable a maximum of 16 addressable ports or planes. Implementations may be designed for more, but in-protocol signaling of port availability is limited to 16.

7.5.1 Requirements

Having multiple independent ports on a NIC means that operation can continue while ports are down or are subject to maintenance. The partial failure and maintenance cases to be handled are as follows.

- Support for “draining” planes: disabling transmission through a plane in preparation for safe maintenance. Disabling transmission should not require disabling reception.
- The ability to immediately identify and react to multiple switch and port failures without errors visible to the QP, diverting traffic away from in-network failure.
- The ability to identify and react to switch and port recovery with limited churn, restoring previously diverted traffic.

7.5.2 State

There are seven key tables of state required to support multiport.

1. Local port number to interface (source) address. (Global to the NIC; address set via network interface driver)
2. Remote port number to interface (destination) address. (Per QP; set during QP configuration.)
3. Local port ready to transmit set. (Global to the NIC; managed as the intersection of configuration (transmit enabled) and port link status)
4. Remote port ready set. (Per QP, updated from acks with debouncing.)

5. Topology wiring that specifies which local ports can reach which remote ports. (Global to the NIC to represent the expected topology. A global table cannot express wiring that differs between QPs, so an implementation needing such differences must support per-QP exceptions. The remote port enabled set below is already per QP.)

6. Remote port enabled set. (Per QP, if used to selectively disable communication to certain destination ports as a compact alternative to wiring.)

7. Selected port pair. (Per flow, may store whether first choice or an nth alternate is used, or store specific port identifying information.)

The same addresses may be associated with more than one port in items 1 and 2, when used in a multi-plane configuration.

7.5.3 Initial Flow to Port Assignment

The initial assignment of flows to ports should distribute flows evenly across available ports. If every QP maps flow 0 to port 0, inherent load imbalance results, because fast start (Section 6.1.3) begins transmitting on flow 0, and reduction to target rate (Section 6.3.3) protects low-numbered flows the most.

The recommended approach computes a base port for flow 0 from a hash of the connection's QP numbers and a NIC-specific seed:

$$\texttt{base} = H(\texttt{local QP}, \texttt{remote QP}, \texttt{seed}) \bmod P,$$

where P is the number of available ports. Flow i is then assigned to port $(\texttt{base} + i) \bmod P$, so successive flows occupy consecutive ports counting upward from the base. The NIC-specific seed spreads the base port across NICs, and the QP-number inputs spread it across QPs on the same NIC, so the load-favored low-numbered flows do not concentrate on any one port. This assignment, and the failover direction of Section 7.5.4, are illustrated in Figure 7.3.

The base and its offsets identify the flow's local (source) port. In the multi-plane configuration each local port reaches the identically numbered remote port (Table 7.1), so this single index also fixes the destination, and Figure 7.3 describes the whole port pair. In the multi-port configuration, source and destination ports are chosen independently (Section 7.5.4); the same consecutive, base-relative counting is recommended for the source port, and an implementation may select the destination port either identically or from an independently hashed base among the ports reachable in the topology. Failover then adjusts whichever port of the pair has become unusable.

7.5.4 Failover Algorithm

When transmitting on a flow, the sender:

1. Extracts the port pair associated with the flow; this port pair may be explicitly represented by four bits for each of local and remote port index;

2. Confirms that the local port is ready to transmit, the remote port is ready, the remote port is enabled, and the port pair exists in the topology. Where the local port has failed, the sender must invalidate the cached port pair assignment and fail over before sending. Remote port status, learned from acknowledgments, is permitted to take effect only on RTO if the result is cached.

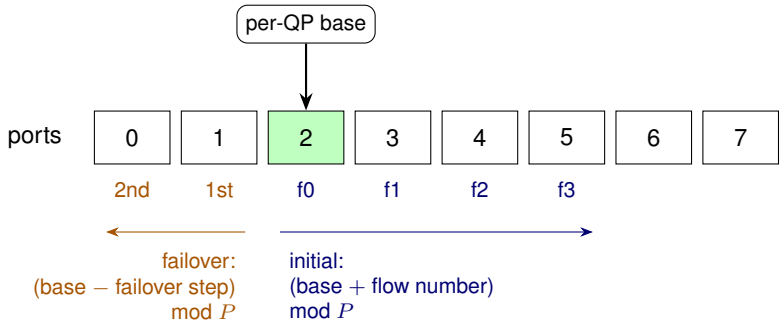


Figure 7.3: Recommended flow-to-port assignment. A hash of the QP numbers and a NIC-specific seed selects the base port for flow 0 (here port 2). Initial flow assignment counts upward from the base; failover selection counts downward, skipping unusable ports. Both wrap modulo the port count P.

3. If all relevant bits are set, transmit.
4. Construct the reduced wiring topology: zero out any connections associated with inactive or disabled ports.
5. Select the alternate port pair from the reduced wiring topology, while trying to preserve the working port.
   (a) If the local port failed, but the remote port is working, find another usable local port, if any.
   (b) If the remote port failed, but the local port is working, find another usable remote port, if any.
   (c) If there is no such port, choose a working bit at random (pick a random index into the reduced wiring topology, find the first successor that is a 1).
   (d) Each usable alternative should have equal probability of being chosen; always selecting the "next" port will lead to overload as every other NIC follows the same process.

Recommended failover selection. The following is one recommended mechanism; implementations may use other algorithms provided they spread failover targets across senders (step 5). Deterministically failing to the failed port plus one (the numerically next port) is explicitly not permitted, because every NIC following that rule concentrates diverted traffic on the same successor port.

The recommended method counts *downward* from the base port (Section 7.5.3), opposite to the upward direction of the initial assignment. The first port needed for failover is $(\texttt{base} - 1) \bmod P$, the next is $(\texttt{base} - 2) \bmod P$, and so on, tracked by a per-QP failover counter. The counter advances again until it reaches a usable port, skipping any that have failed, are disabled, or are the port being failed away from. Counting downward keeps failover targets away from the ports carrying initial, load-favored flows. Because the base port differs across NICs and QPs, a fixed counting direction does not synchronize failover targets across senders, providing the load spreading that step 5 requires without per-selection randomness.

It is assumed that connectivity is symmetric (a local port can both send and receive with a remote port). Asymmetric topologies, in which a local port can send to but not receive from a given remote port, are not supported by this protocol; such configurations should be excluded from the topology wiring table.

		remote port							
		1	2	3	4	5	6	7	8
local port	1	1	-	-	-	-	-	-	-
	2	-	1	-	-	-	-	-	-
	3	-	-	1	-	-	-	-	-
	4	-	-	-	1	-	-	-	-
	5	-	-	-	-	1	-	-	-
	6	-	-	-	-	-	1	-	-
	7	-	-	-	-	-	-	1	-
	8	-	-	-	-	-	-	-	1

Table 7.1: Multiplane topology wiring: each local port is connected to the identically numbered remote port

		remote port							
		1	2	3	4	5	6	7	8
local port	1	1	1	1	1	1	1	1	1
	2	1	1	1	1	1	1	1	1
	3	1	1	1	1	1	1	1	1
	4	1	1	1	1	1	1	1	1
	5	1	1	1	1	1	1	1	1
	6	1	1	1	1	1	1	1	1
	7	1	1	1	1	1	1	1	1
	8	1	1	1	1	1	1	1	1

Table 7.2: Full multiport topology wiring: each local port is connected to each remote port

7.5.5 Port status maintenance

Implementations with multiple ports should keep track of which ports are working and active. A port must be marked not-ready when a link-down event is detected by the physical layer (e.g., loss of signal). A port may also be marked not-ready by software for maintenance or draining. If the failure of a link is detected, the ready bit must be cleared.

7.5.6 Port status reporting in acknowledgment

With multiple ports, a failed connection to a switch or a failed switch no longer means that the NIC is unreachable; it means only that certain paths through the network will not work. Each multiport NIC may report back to senders the set of working ports in an ack. This is a direct copy of the local port status bitset into the ack.

		remote port							
		1	2	3	4	5	6	7	8
local port	1	1	1	-	-	-	-	-	-
	2	1	1	-	-	-	-	-	-
	3	-	-	1	1	-	-	-	-
	4	-	-	1	1	-	-	-	-
	5	-	-	-	-	1	1	-	-
	6	-	-	-	-	1	1	-	-
	7	-	-	-	-	-	-	1	1
	8	-	-	-	-	-	-	1	1

Table 7.3: Partial multiport topology wiring example: pairs of ports can reach each other

7.5.7 Hysteresis / debouncing

A remote port's transition to not-ready, learned from an acknowledgment, should be accepted immediately. Future traffic following the Section 7.5.4 algorithm will be immediately diverted away from the port.

A transition to ready should only be accepted a configurable duration after the most recent transition to not-ready. The duration should be large enough to avoid reacting to stale information and small enough to permit timely recovery; five milliseconds is likely sufficient. This requires keeping a timestamp associated with the last transition to not-ready. Only after the debouncing period can a transition to ready be processed, returning diverted Meta-RoCE flows to the previously disabled port pair.

Because port reachability is evaluated during ack processing, only active QPs are informed about, and thus process, port status changes. Inactive QPs may not notice a short-duration port failure and recovery. Conversely, the first packet sent to a recipient after a period of inactivity may be directed to a port that has since failed; alternate flow retransmission (Section 4.8) is expected to correct if the initial packets reach only the failed port.

Part IV

Headers

Chapter 8

Packet Headers

This chapter presents the headers of Meta-RoCE packets. Meta-RoCE defines its own base, data (METH), selective ack (SAETH), and RNR-Cancel headers. It borrows from RoCEv2 the Read/write (RETH), Immediate (ImmDt), Invalidate (IETH), Atomic (AtomicETH) and Atomic Ack (AtomicAckETH) headers. A control packet format (Section 8.7) is provided to support an extensible exchange of information.

In the diagrams of this section, the bit markers in the heading (31 down to zero) represent the significance of the bit if each 32-bit word (corresponding to a row in the diagram) is stored in most-significant-byte order. For example, given a pointer to the packet header, (`void *p`), the following would extract flow ID and opcode:

```
void extract_path_id_and_opcode(void *p, int *path_id, int *opcode) {
  uint32_t w = ntohl(((uint32_t *)p)[0]);
  *path_id = w & 0xfff;
  *opcode = w>>24;
}
```

As described in Section 4.1, Meta-RoCE packets, including data, acknowledgment, and control, end with a Meta-RoCE CRC of 32 bits that covers the headers, payload, and any padding.

A transmitter pads the payload as necessary to make the packet length a multiple of 4 bytes, and, when carried directly over Ethernet, to reach the minimum frame size (Section 4.1). The number of padding bytes is recorded in the Padding field of the data header (Section 8.2). A receiver may accept a payload that is not padded to a 4-byte boundary; the mCRC is byte-granular, so alignment is not required for correctness.

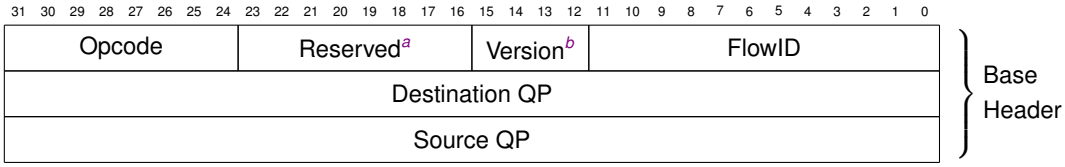
In all packet headers, reserved fields must be set to zero by transmitters implementing this version of the specification. Receivers of packets must not reject packets where reserved fields have nonzero values.

8.1 Base Header

The base header format is shown in Figure 8.1.

8.1.1 Opcodes

Meta-RoCE Connected Mode uses the value “110” in bits 7:5 of the RoCEv2 opcode to indicate the transport type. This is one of two values reserved for manufacturer defined opcodes. Bits 4:0 specify individual



[a]Reserved fields are set to zero on transmission; ignored on receipt
[b]Version must be zero; verified to be zero on receipt

Figure 8.1: Base header format

opcodes and attempt to follow the general pattern defined in RoCEv2. Where RoCEv2 defines a “first”, “middle”, and “last” code, Meta-RoCE adopts the “first” opcode and leaves the remaining two reserved.

Table 8.1 enumerates the Opcodes supported by the Meta-RoCE Connected Mode. The Additional Headers describes which headers follow the base header for each Opcode. All Meta-RoCE data packets have the METH (Section 8.2). Wherever possible, the standard RoCE headers are used.

8.1.2 Version

The version field shall be zero. Packets received with a nonzero version field must be silently discarded and counted.

8.1.3 Flow ID

The sequential Flow ID associated with this data flow identifies which flow (set of flow sequence numbers) this packet belongs to. Implementations need not support 4,096 flows.

In environments where multiple addresses are bound to an endpoint the “path” may be asymmetric, using one set of source and destination addresses for data flowing in one direction, and a different set of addresses in the other. The acknowledgment on a flow should be sent to the source address of the data.

Restated, data traffic from node A to B flow 1 may have different source and destination addresses as ack traffic from node A to B path 1, because the ack traffic will mirror the addresses of data traffic from node B to A flow 1.

8.1.4 Destination and Source QP

The destination QP determines the context for the data at the recipient. The source QP identifies the sender for error responses.

Both fields are 32 bits. A verbs QP number is in practice limited to 24, which existing implementations validate against, but that limit is not inherent to RDMA and the header leaves room for a wider identifier. No tag is encoded in these fields; the data header has the Tag (Section 8.2). Where a verbs interface packs a tag into the high bits of its own QP number (Section 10.6), that packing is local to the interface.

8.1.5 Packet validation

Every Meta-RoCE packet is checked against the base header and the receiver’s own state before its opcode is interpreted:

Opcode [7:5]	Opcode [4:0]	Description	Additional headers
000	xxxxx	RoCEv2 RC	
011	xxxxx	RoCEv2 UD	
110	00000	Send	METH, Pyld
110	00001	Send with Immediate Present[a]	METH, ImmDt[b] Pyld
110	00010	RDMA Write with Immediate Present	METH, RETH, ImmDt, Pyld
110	00011	Send with Invalidate[c]	METH, IETH, Pyld
110	00100	Send Last[d]	METH, Pyld
110	00110	RDMA Write	METH, RETH, Pyld
110	00111	RDMA Write with Immediate	METH, RETH, Pyld[e]
110	01100	RDMA Read Request	METH, RETH
110	01101	RDMA Read Request with Immediate Present[f]	METH, RETH, ImmDt
110	01111	RDMA Read Response	METH, Pyld
110	10000	RNR-Cancel, Receiver Is Ready	RNR-Cancel
110	10001	Selective Ack	SAETH
110	10010	Atomic Ack (Response)	METH, AtomicAckETH
110	10011	Reserved for Atomics	METH, AtomicETH
110	10100	FetchAdd	METH, AtomicETH
110	10101	CmpSwap	METH, AtomicETH
110	1011x	Reserved for Atomics	METH, AtomicETH
110	11110	Reliable Control[g]	METH, Control, Pyld
110	11111	Unreliable Control	Control, Pyld

[a] "Immediate Present" indicates that this specific packet has an immediate header

[b] The Send with Immediate Present opcode appears only in the last packet of a send with immediate operation, because both send and send-with-immediate operations expect a receive WQE to be present.

[c] The Send with Invalidate opcode appears only in the last packet of a send-with-invalidate operation. All preceding packets use the Send opcode (00000). This mirrors the Send with Immediate Present pattern.

[d] The Send Last opcode marks the final packet of a send operation. Send with Immediate Present and Send with Invalidate also mark the final packet; Send Last is used when neither immediate data nor invalidation is needed. The message length may be shorter than the posted receive buffer.

[e] Write with Immediate opcode is used on all packets of the write-with-immediate message except the last, which uses "RDMA Write with Immediate Present" above. The receiver uses the Immediate Present opcode to match MSN and CSN to a known receive WQE for the write-with-immediate.

[f] Read/ImmDt may not be supported

[g] Support for reliable control is optional.

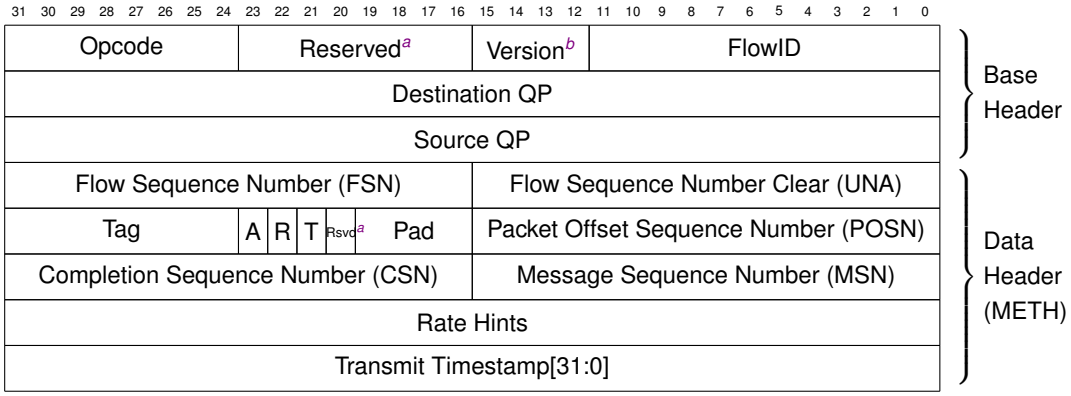
Table 8.1: Opcodes defined in the header. 17 of 32 are allocated here, with two more reserved for atomics; those that do not appear in this table are reserved.

- the packet is long enough for the smallest valid packet of its opcode, which for an acknowledgment is the acknowledgment header with no bit vectors;

- the mCRC is valid (Section 4.1);
- the transport-type bits are “110” (Section 5.5.2);
- the version field is zero (Section 8.1.2);
- the destination QP is one the receiver holds (Section 5.2.3) and in a state to receive (Section 5.2.6);
- the source QP matches the expected remote QP (Section 5.2.4);
- the Flow ID is within the supported range (Section 5.2.1).

These checks are common to every packet and logically precede any processing specific to an opcode. Criteria defined for a particular packet type, such as the acknowledgment acceptance criteria of Section 4.2.3, apply to a packet that has already passed them.

8.2 Data Packet Header (METH)



[a]Reserved fields are set to zero on transmission; ignored on receipt
[b]Version must be zero; verified to be zero on receipt

Figure 8.2: Packet header format, Operation Type == Send || Read Response

The sequence number fields in the data packet header (FSN, POSN, MSN, CSN) are defined in Chapter 2.

The tag field provides an alternate mapping of messages to queues, where supported.

The ack request “A” bit indicates that this data packet requests an immediate acknowledgment with echoed timestamp. See Section 4.2.1.

The Retransmission “R” bit indicates that this data packet has been previously transmitted. Packets with this bit set should be counted at the receiver as a necessary (was not previously seen) or spurious retransmission.

The Timeout “T” bit indicates that this data packet was scheduled for retransmission due to timeout, not due to fast retransmission via selective acknowledgment of out-of-sequence data. Packets with this bit set should be counted at the receiver as a necessary (was not previously seen) or a spurious retransmission by timeout. The “T” bit allows better parameter tuning, clearer implementation testing, and better diagnosis of in-network faults. Spurious “T” retransmissions might indicate a too-small value of minimum RTO, while spurious “not-T” retransmissions indicate an incorrect setting for in-network reorder tolerance. Non-spurious “T” indicate congestion substantially in excess of in network buffering that likely requires correction. Non-spurious “not-T” retransmissions indicate modest congestion that may not be performance-limiting.

Both bits are undefined on packets sent after an HRNR, and neither is set on data sent again after a BRNR-NAK (Section 4.6).

The Padding field counts the padding bytes, zero to fifteen, that follow the payload. Subtracting it from the framed length gives the payload length. A transmitter should set this field, and must set it when the packet is carried directly over Ethernet (Section 4.1). A nonzero value is authoritative in any encapsulation. A zero value does not assure that no padding is present, because a transmitter that does not set the field also leaves it zero; Section 5.2.5 describes how a receiver resolves undisclosed padding.

8.2.1 Rate hints / Desired Rate

Rate hints in the data header represent an approximate expected speed of this transfer in gigabits per second (bits per nanosecond). This value is set differently in three cases:

- In fast start, this may be set to the link rate or to the share of the link rate that can be used (link rate divided by the number of ready sender QPs).
- In normal congestion control operation, the desired rate should be set as a function of `QWND`.

$$\mathrm{QWND} \times \mathrm{MTU} \times 8 / \mathrm{RTT\ ns}$$

  where MTU is the queue pair's MTU in bytes (Section 2.3.1). To simplify implementation by avoiding a division operation, the queue pair's smoothed RTT may be quantized and used as a shift. That is, at a 4096 byte MTU, where the RTT is between 6144 ($2^{12} + 2^{11}$) and 12,288ns ($2^{13} + 2^{12}$), assume it is 8192 ns and set the desired rate to `QWND << 2`. (Converting QWND from 4096 byte packets to bits shifts left by 12 + 3; dividing by 8192 shifts right by 13; combined, this is a left shift by 2.) When SRTT lies between 12,288 and 24,576, shift left by 1. The transmitted desired rate must be clamped to be no smaller than 1 and no greater than the capacity of the link.
- When reaching the end of the transfer, where the amount of data left to transmit is less than `QWND`, determined by this being the last WQE to process and the WQE having few packets remaining to transmit, the desired rate should be set to zero.

Whether this value is nonzero is used in Section 6.3.1.

8.2.2 Transmit Timestamp

A retransmission's timestamp is the time it is retransmitted, not the time the original was sent, which is what leaves round trip time measurement free of retransmission ambiguity (Section 4.5.2).

The transmit timestamp should support approximately nanosecond precision. It need not be a product of a synchronized clock: different stations may have their own free-running clocks and should not assume that a received timestamp can be compared to the local clock directly. It need not reflect precisely nanoseconds: a cycle counter of a 0.8GHz clock would be acceptable. Nanosecond-level precision enables observation of the 41ns delay of a 4KB packet through an 800 Gbps link, which in turn enables observing queue delays of such packets on a lightly congested network link. Microsecond-level precision would only permit, at best, observing queuing delays of 100 kilobytes on an 800 Gbps link.

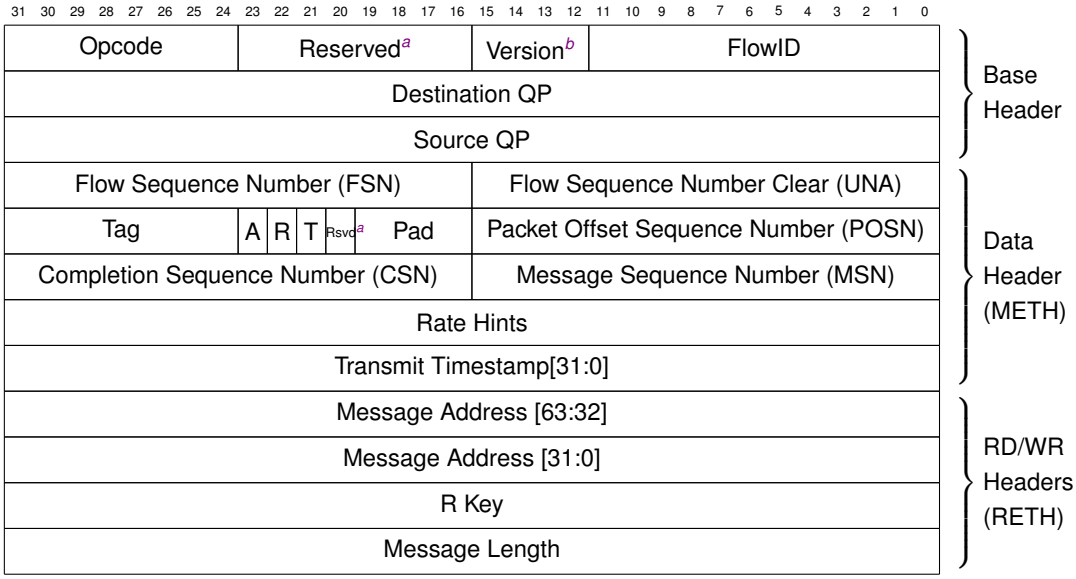
8.2.3 Zero-byte opcodes

Table 8.2 identifies which opcodes permit zero-byte payloads. Data packets with a message length of zero where zero-byte payloads are forbidden must be discarded and counted; the responder may send a NAK with the Invalid Request code.

Opcode	Description	Zero data headers
110 00000	Send	Permitted
110 00001	Send with Immediate Present	Permitted
110 00010	RDMA Write with Immediate Present	Permitted
110 00011	Send with Invalidate	Permitted
110 00100	Send Last	Permitted
110 00110	RDMA Write	Forbidden
110 00111	RDMA Write with Immediate	Forbidden
110 01100	RDMA Read Request	Not applicable (fixed size request).
110 01101	RDMA Read Request with Immediate Present	Not applicable (fixed size request).
110 01111	RDMA Read Response	Permitted; A read request of length zero confirms flush of prior writes.
110 10000	RNR-Cancel, Receiver Is Ready	Not applicable.
110 10001	Selective Ack	Not applicable.
110 10010	Atomic Ack (Response)	Not applicable (fixed size atomic).
110 10011	Reserved for Atomics	Not applicable (fixed size atomic).
110 10100	FetchAdd	Not applicable (fixed size atomic).
110 10101	CmpSwap	Not applicable (fixed size atomic).
110 1011x	Reserved for Atomics	Not applicable (fixed size atomic).
110 11110	Reliable Control[g]	Permitted (entirety of message may be in header, e.g., ping)
110 11111	Unreliable Control	Permitted (entirety of message may be in header)

Table 8.2: Opcodes that permit zero-byte payloads. Variable length operations support zero byte payloads except where the operation has limited meaning (zero byte writes).

8.3 Read Request and Write Headers



[a]Reserved fields are set to zero on transmission; ignored on receipt
[b]Version must be zero; verified to be zero on receipt

Figure 8.3: Packet header format, Operation Type == Read Request || Write

The read request header is as RETH in RoCEv2. The Message Length is the length of the overall write or read request. The sender must provide identical RETH fields (address, R Key, and length) in all packets of the same write message. The receiver must validate the RETH fields at least once per message. It is not required to re-validate the RETH on every packet of a multi-packet write; an implementation may use a cached, already validated copy of the RETH information to deliver the packet data, or may raise an error if the RETH fields are inconsistent across packets of the same message.

8.4 Immediate Data (ImmDt)

The immediate is a 32-bit field that appears after the RETH header, either in the read request or in the last packet of a write. For a send with immediate, the RETH header is absent and the immediate appears in the last packet.

The immediate data is sent in big endian order. The verbs work request expects big endian order from the application and the verbs completion provides big endian order to the application. That is, it is expected that the protocol transmit the 32-bit value as provided without byte swap.

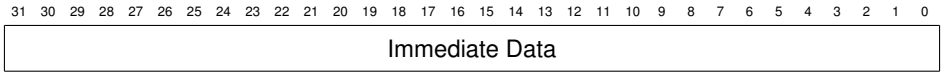


Figure 8.4: Immediate Data header

8.5 Ack Packet

The Meta-RoCE acknowledgment comprises a selective ack header, optional FSN bit vector to identify out-of-order delivered frames, and an optional RNR bit vector to identify received-but-rejected frames. The full acknowledgment packet layout is shown in Figure 8.5. Bit vectors are 256 bits (32 bytes) long, but abbreviating the selective acknowledgment bitset on the wire should be expected. The size of an acknowledgment header is 20 bytes (32 if counting the 12-byte base header), plus 32 if only the FSN bit vector is present or 64 if both FSN and RNR bit vector are present. A transmitter must include the RNR bit vector when the "Ack Status" field indicates a BRNR (010xxxxxb), and must include the FSN bit vector when "Ack Status" is 0x1 or the RNR bit vector is present. Both bit vectors may be sent with zeroes with other status codes, and convey no information to the receiver. These rules constrain how an acknowledgment is built; they are not conditions a receiver validates.

The Ack Status determines which bit vectors appear. Their length is the transmitter's choice, because a receiver acts on the acknowledgment header and accepts any length of vector beyond it. A transmitter that tracks fewer than 256 out-of-order frames has no bits to place in the remainder of a vector, and may omit the trailing zeroes to save bytes on the wire. A received acknowledgment that has "Ack Status" 0x1 and has an FSN bit vector smaller than 32 bytes but long enough to include the bits between the cumulative acknowledgment (Cumulative FSN) and the highest sequence number received (Max FSN) must be accepted. The vector spans the 256 sequence numbers after the Cumulative FSN, so where Max FSN lies beyond that span the full 32 bytes are the most that can be sent. Although not preferred, a receiver need not process the FSN bit vector; it may instead advance only the cumulative portion of the acknowledgment (Cumulative FSN), treating the acknowledgment as if it had "Ack Status" zero (in-order) and had no FSN bit vector. Restated, any received Meta-RoCE acknowledgment with a complete acknowledgment header and that passes the acceptance criteria of Section 4.2.3 must have the cumulative FSN processed, and a receiver must accept an acknowledgment with "Ack Status" 0x1 whose FSN bit vector is abbreviated to the bits between the Cumulative FSN and the Max FSN. Past packet validation (Section 8.1.5), the conditions enumerated in Section 4.2.3 are the only grounds for discarding an acknowledgment; how the bit vectors are sent is not among them.

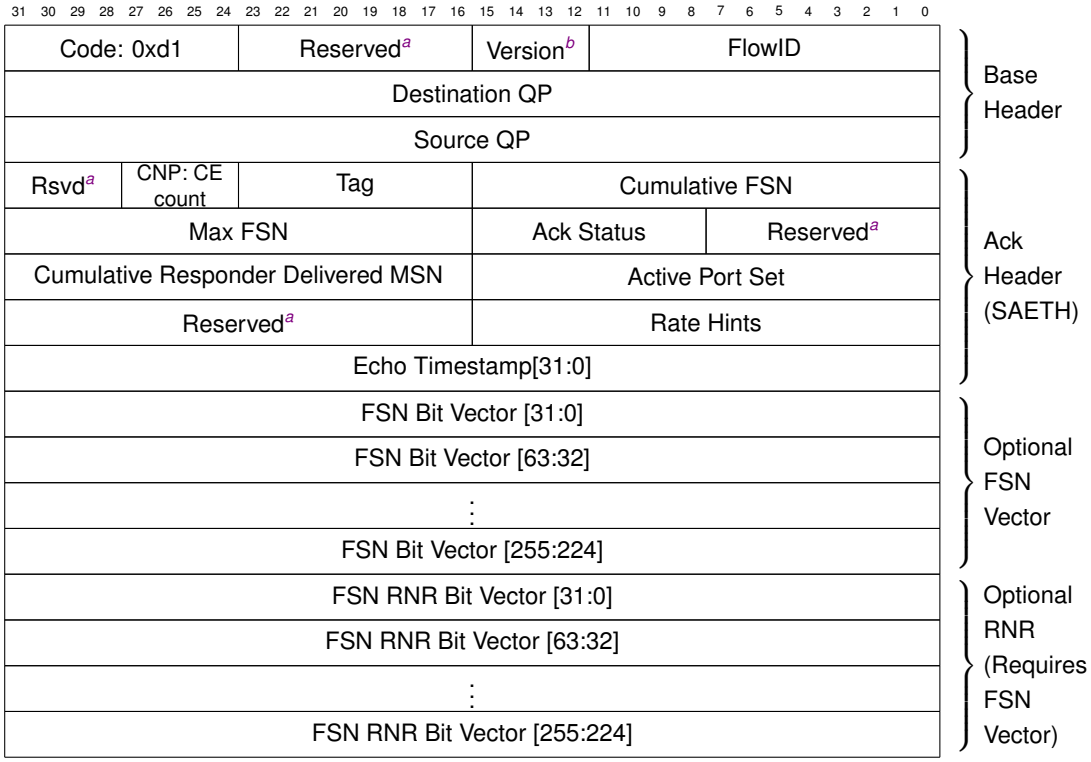
A BRNR-NAK must include the full 32-byte FSN bit vector, so that the length of the RNR bit vector following it can be taken from the framed length of the packet. A receiver that finds a BRNR-NAK too short for both processes the Cumulative FSN and discards the remainder.

The FSN and RNR bit vectors may be only partially populated: a receiver may track any number, larger or smaller than 256, of frames that are received out of order or that are being explicitly given B-RNR. As a concrete example, it is permitted for an implementation to track only 128 out-of-order delivered packets (FSN bit vector) and 8 B-RNR packets. The remaining zeroes in the bit vector convey no information to the sender, which will retransmit as needed. Conversely, it is permitted for an implementation to track 1024 out of order delivered packets, advancing the cumulative FSN when possible.

It is expected that acknowledgments will be sent using a different traffic class from data packets.

8.5.1 CNP Congestion notification

The congestion notification field is a four-bit count of the CE-marked packets received since the last ack was sent. This counter is incremented upon receiving a CE-marked packet, and cleared when read to populate the ack being sent. It is permitted for this congestion signal to be unreliable: if the ack is lost, a transient delay will be the only reaction. The counter saturates at 15. An acknowledgment is sent at or before that point (Section 4.2.2), reporting and clearing the count.



[a]Reserved fields are set to zero on transmission; ignored on receipt
[b]Version must be zero; verified to be zero on receipt

Figure 8.5: Ack packet format. The optional FSN vector is present when the Ack Status is Out-of-order or BRNR. The optional RNR vector is present when the Ack Status is BRNR. A vector present where the status code does not call for one is ignored. The FSN vector may be abbreviated only when the RNR vector is absent.

8.5.2 Tag

Zero where tags are not supported. Where tags are supported, this tag field provides the scope of the CDMSN field (Section 8.5.5). The tag field must be set for all NAKs.

In a BRNR-NAK the tag field scopes the CDMSN as usual and does not identify a tag to hold. A BRNR applies to frames, those named in the RNR bitset. The initiator maps them to tags through its own binding of *(Flow,FSN)* to *(Tag,Opcode,MSN,POSN)* (Section 2.5).

8.5.3 Cumulative FSN

The Cumulative FSN is defined in Section 4.2.3 as the sequence number being cumulatively acknowledged. For NAK error codes other than BRNR-NAK, the field instead holds the FSN of the packet that is being negatively acknowledged (Table 8.5).

Where the receiver has no state from which to draw either value, it sets the field from the incoming

header's UNA field, to UNA-1. This is comparable to how the acknowledgment sequence number is set on a TCP RST packet when no state exists.

8.5.4 Max FSN

The Max FSN field is the highest FSN that has been fully acknowledged (received and will be delivered to memory). It is primarily an optimization that permits a sender to iterate through fewer bits in the selective acknowledgment bit vector. Max FSN may exceed CumulativeFSN by more than 256 if the receiver tracks more out of order packets than the bit vector can express directly. In an ACK, SACK, or BRNR-NAK, Max FSN must identify a packet the receiver has accounted for: one that is fully acknowledged, or, in a BRNR-NAK, one whose bit is set in the RNR bitset. The remaining acknowledgment types instead give the FSN of the packet that led to the NAK (Table 8.5). See Section 4.2.3 for detail.

8.5.5 Cumulative Responder Delivered MSN (CDMSN)

The CDMSN field confirms from responder to initiator that writes, sends, read requests, and atomics have been delivered. Completions at the responder may not yet have been posted.

The CDMSN is valid for in order (ack status 0), out of order (ack status 1), and BRNR (ack status 010xxxxx). For negative acknowledgments other than BRNR-NAK (NAK, HRNR), the CDMSN field provides the MSN of the errored message (e.g., for an address error); it is not a cumulative value in this context. The initial value is one less than the initial expected MSN configured for the QP and Tag.

Where tags are supported, this MSN is the one that is associated with the Tag specified in the Tag field of the acknowledgment. The sender of the acknowledgment may choose to send the MSN associated with the Tag of the most recently received data packet, to send the MSN associated with the Tag of the most recently delivered message, to rotate among active tags round robin, or some combination. Upon receipt of duplicate data, the Tag/MSN combination in the ack should be that matching the received tag.

For negative acknowledgments where the CDMSN is the MSN of the errored message, the Tag field must also be set to the Tag of the errored message.

8.5.6 Ack Status

The Ack status field comprises the following potential values, based on the RoCEv2 ack syndrome.

The reserved codes in Table 8.3 are NAKs whose specific handling is not yet defined. An initiator that receives an Ack Status it does not recognize treats it as a NAK, placing into error the tag and work request identified by the Tag, Cumulative FSN and CDMSN fields. A code defined later therefore reaches an implementation that predates it as an error on the right message, rather than as something it must have anticipated.

RNR Backoff

Encoded RNR backoff time is defined in Table 8.4. A nonzero value represents the number of bits to shift 5 microseconds left. That is, possible requested retry durations are, in microseconds: 0, 10, 20, 40, 80, 160, 320, 640, . . . , 163840. This backoff time is a recommendation from the receiver based on what is necessary for the receiver to become ready for the packet.

A new RNR backoff timer overwrites the previous, whether larger or smaller.

When the retried message is received, another RNR may be sent, the receipt of which will reset the timer to the newly requested timer duration, which may be unchanged from the previous.

Ack Status [7:5]	Ack Status [4:0]	Description
000	00000	ACK; all data in sequence; No FSN bitset follows
000	00001	ACK; some data received out of order; FSN bitset follows
001	TTTTT	HRNR-NAK, TTTTT = encoded backoff time; No bitset follows
010	TTTTT	BRNR-NAK, TTTTT = encoded backoff time; FSN and RNR bitset follows[a]
011	00000	Nak: Flow ID unsupported / too large
011	00001	Nak: Invalid Request
011	00010	Nak: Address invalid / Access Error[b]
011	00011	Nak: Operational Error[c]
011	00100	Nak: Reserved
011	00101	Nak: Reserved
011	0011x	Nak: Reserved

[a] BRNR-NAK is used whenever an RNR bitset is required, including when selectively acknowledging data that was processed.
[b] In RoCEv2, “Remote Access Error”
[c] In RoCEv2, “Remote Operational Error”

Table 8.3: Ack status codes. Only BRNR-NAK identifies the specific packets that could not be immediately delivered. All other Nak codes indicate that an error occurred associated with a packet with FSN equal to the CumulativeFSN field.

The RNR retry timer need not be precise, and may fire after the requested hint time, but should not fire early.

Backoff Time [4:0]	Backoff Time As Integer	Description
00000	0	0 μs: Immediate retry (transient failure)
0xxxx	1–15	5 μs $\times 2^n$: 10 μs to 163.84 ms
1xxxx	>= 16	Reserved

Table 8.4: RNR backoff time encoding.

HRNR vs BRNR

The processing difference between BRNR and HRNR is in which resources are missing. In BRNR, the QP and the state of the flows are present, but the buffer (e.g., the receive WQE matching the received send packet) is not. In this case of a missing buffer, other tags and their other receives may make progress, and identifying which FSNs are acknowledged normally and which FSNs were received without being able to deliver (RNR) has value, leading to the RNR bitset. In HRNR, the QP is either not initialized or not present in the necessary cache and ready to accept data. Because the necessary QP and flow state are not known, individual RNR bits would not be meaningful. Similarly, the message sequence fields of an HRNR-NAK say

little, the responder having set them without the QP state that would make them reliable. They may still serve to match an HRNR to its soliciting packet. For HRNR, all data transmission on all tags must wait for the QP to be activated. An HRNR-NAK therefore has neither bitset. An initiator that receives one with either ignores the bitset and processes the rest of the acknowledgment as usual.

Field	ACK	SACK	BRNR	HRNR	Flow ID Error	Inval/Op/Addr Error
Opcode	0xd1					
Version	0					
Flow ID	Set normally, echoing received flow ID					
Destination QP	The expected remote QP; on HRNR, the received source QP					
Source QP	Set normally, from received destination QP					
Reserved (all)	0, Ignored					
CNP	May be set normally, one if congestion marked, else zero					
Tag	Set normally			Tag of packet that led to NAK		
Cumulative FSN	Set normally			FSN of packet that led to NAK		
Max FSN	Set normally			FSN of packet that led to NAK		
Ack Status	0	1	0x40–0x5f	0x20–0x3f	0x60	0x61–0x63
CDMSN	Set normally			MSN of packet that led to NAK		
Rate Hints	Set normally			Reserved	Supported flow count	Reserved
Echo Timestamp	Set normally					
FSN Bitset	Not present	Present		Not present		
RNR Bitset	Not present		Present	Not present		

Table 8.5: ACK/NAK Header Fields. Tag and MSN fields may be used to identify the failed message for Operational Error and Address Error.

Flow ID Unsupported / Too Large

The recipient of a NAK with the flow unsupported code should retransmit all frames on the unsupported flow on a numerically smaller, supported flow. See Section 5.2.1.

Because flow 0 has no numerically smaller flow to move to, a Flow ID Unsupported NAK must not identify flow 0, and a sender that receives one naming flow 0 must ignore it.

The number of supported flows is placed in the rate hints field. That is, where eight flows are configured and a frame with flow id 15 is received, the NAK should contain a rate hint of 8. A flow id of 15 is “too large” because it exceeds the number of supported flows.

Including the number of valid flows in the Flow ID Unsupported NAK permits simple suppression on a per-QP basis when NAK generation is not a fast-path operation. For example, an implementation may store a flag that a NAK has been sent since the last positive ACK or in a recent time interval and only send a NAK if that bit is clear or the ACK request bit has been set. This is an implementation dependent tradeoff between the cost of maintaining state and the efficiency of NAK generation.

8.5.7 Active Port Set

The active port set represents the ready status of each port on a multi-port (multi-plane) NIC. A set (1) bit represents that the port is active. An eight-port NIC with all ports active will have value 255 (0xFF). A four-port NIC with all ports active will have value 15 (0xF). A NIC with only the first port active will have value 1 (0x1).

A value of 0 indicates a lack of support for active port set transmission; receiving an ack implies that at least one port is working. A single port NIC does not benefit from transmitting the state of the single port.

The recipient of an acknowledgment updates its remote port ready set (Section 7.5.2) from the bits for ports its topology wiring includes, and ignores the others. A peer with more ports than the recipient knows of is then understood rather than rejected.

One bit per port means the field expresses at most 16. A NIC with more ports cannot signal all of them, so the field is undefined for it. Such a system might give each QP a subset of ports and report the active port set for that subset, or group its ports into at most 16 groups that share fate, one bit reporting the state of a whole group.

8.5.8 Rate Hints / Permitted Gbps

The rate hints field represents the expected number of gigabits per second that would represent an approximately fair share of the links being used. For example, an 800 Gbps NIC receiving on two active queue pairs should set this value to 400. A value of 1 is the minimum; a value of 0 indicates that the field is to be ignored.

The receiver of this rate hint will multiply this value by the RTT, divide by 8 bits per byte and divide by the queue pair's MTU in bytes to determine the $\texttt{QWND}_{max}$. At a 4096 byte MTU, these two divisions can be implemented efficiently as right shift by 15. When the RTT is kept in nanoseconds (10^{-9} seconds) and the bitrate expressed in gigabits per second (10^{9} bits per second), the product of the two is a bandwidth delay product in bits.

$\texttt{QWND}_{max}$ cannot be set to a value smaller than the configured $\texttt{QWND}_{min}$. The smallest such value is 1; a small rate of 3 Gbps at 10 μs RTT reaches this. Because a bitrate smaller than 1 Gbps would lead to a bandwidth delay product smaller than one packet for small-network round trip times, it is not necessary to have higher precision.

8.5.9 Ack Packet FSN Bit Vector

The FSN bitset in the ack packet is formatted where the zeroth bit (which represents the packet at CumulativeFSN+1) is the least significant bit of the 32-bit, most-significant-byte-first integer at "FSN Bit Vector [31:0]". That is, to find whether an FSN bit is set, one could use the following algorithm.

```
struct ack_header {
  ...
  uint16_t cumulative_fsn;
  ...
  uint32_t fsn_bit_vector[8];
};

bool is_bit_set(const struct ack_header *a, uint8_t bit) {
  uint32_t w = ntohl(a->fsn_bit_vector[bit/32]);
  return ((w>>(bit % 32)) & 0x1);
```

```
}

bool is_fsn_bit_set(const struct ack_header *a, uint16_t fsn) {
  uint16_t offset = fsn - a->cumulative_fsn - 1;
  return offset < 256 && is_bit_set(a, (uint8_t)offset);
}

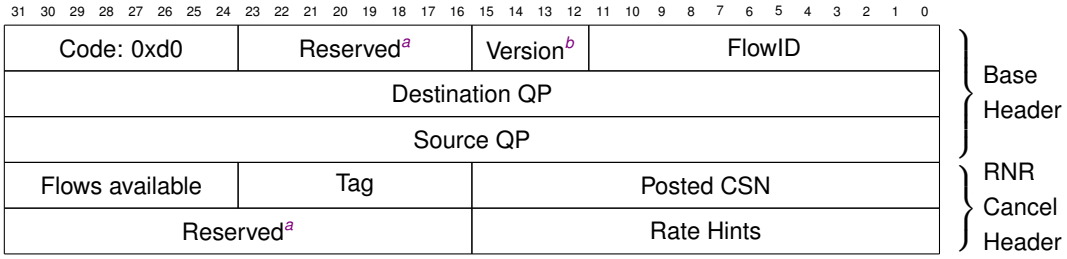
bool is_fsn_acked(const struct ack_header *a, uint16_t fsn) {
  return !after(fsn, a->cumulative_fsn) || is_fsn_bit_set(a, fsn);
}
```

8.5.10 Ack Packet RNR Bit Vector

Bits that are set in the BRNR bit vector indicate packets that have been received before a receive WQE or similar resource has been made available. The sender must return the corresponding frame (qp/tag/msn/posn) to a queue for BRNR retransmission and not reuse the flow/FSN combination for this data, as described in Section 4.10. The bit representing a frame can be set in either the acknowledgment FSN bit vector (Section 8.5.9) OR the RNR bit vector. It may not appear in both, because the data was not delivered to memory and cannot be fully acknowledged.

A sender that receives a BRNR-NAK whose RNR bit vector is absent or shorter than 32 bytes treats the absent bits as zero. The frames those bits would have named are then neither fully acknowledged nor explicitly given BRNR, and are recovered by ordinary loss detection. This is the same treatment given to the trailing zeroes of a bit vector from a receiver that tracks fewer than 256 frames, and the acknowledgment must not be discarded on this basis.

8.6 RNR-Cancel / Receiver Is Ready Packet



[a]Reserved fields are set to zero on transmission; ignored on receipt
[b]Version must be zero; verified to be zero on receipt

Figure 8.6: RNR-Cancel packet format. Section 4.10.1 describes when one is sent.

The RNR-Cancel packet format appears in Figure 8.6. When a cancel is sent, and how many, is described in Section 4.10.1.

The Tag field identifies the tag whose hold the cancel ends, and the Posted CSN the highest CSN with a receive WQE posted on it (Section 2.4). Flows available is the number of receive flows the responder can accept, as in the supported flow count of the ping header (Section 8.7.1). The Rate Hints field is as in an acknowledgment (Section 8.5.8). In computing it, the responder may count the queue pair that is about to resume among its active senders, though it is not yet sending.

8.7 Control Packet Header

Two control opcodes are defined (Table 8.1). Unreliable control, the subject of this section, has no METH and is sent without acknowledgment or retransmission. Reliable control has a METH and is delivered by the transport as any other reliable message is; supporting it is optional.

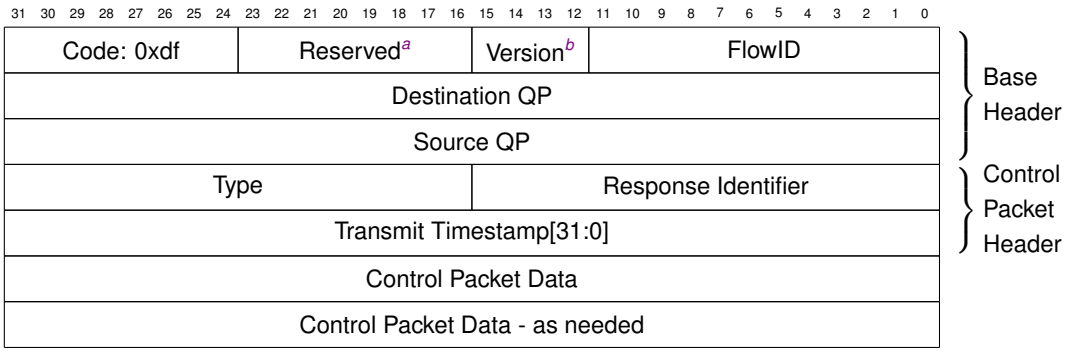
The QP numbers in an unreliable control packet need not be validated by the hardware when firmware or software process the control message. For example, the control message may be inquiring about the state of a not-yet-ready QP or one that is in the error state, where the endpoints may not exist or be connected. A reliable control message is sent on an established QP and is validated as any other packet on that QP is.

The QP fields of an unreliable control packet need not name a connection. An initiator asking about a queue pair it believes exists sets both fields to that pair's numbers; an initiator asking independently of any queue pair sets an arbitrary destination and a source it has set aside for the purpose. An echo response swaps the two, as an acknowledgment does (Section 8.1.4).

An unreliable control packet has no FSN, so its flow ID scopes no sequence space and is expected to be zero. Where the message itself concerns a flow, as an echo request does (Section 8.7.1), the field identifies that flow.

When exchanged, control packets may match request packets with response packets using the response identifier field, set by the initiator and echoed by the responder.

A control packet has variable length and no Padding field. Where padding was added to reach a minimum frame size it is part of the control packet data, and an echo response returns that data as received. A control message whose handling depends on the length of its data should have a length of its own.



[a]Reserved fields are set to zero on transmission; ignored on receipt
[b]Version must be zero; verified to be zero on receipt

Figure 8.7: Control packet format

8.7.1 Ping (Echo) Control Packet Headers

Echo and echo response carry the same header and variable length payload. The payload is echoed into the response. The ping message interrogates the responder about the state of flows, QPs, and tags. The flow ID in the base header is the flow being asked about, which error states 5 and 6 report on, and the Tag field is the tag, which error state 3 reports on. A response repeats the tag and flow of the request it answers. The Active Port Set is as in an acknowledgment (Section 8.5.7), a request with the requester's ready ports and a response the responder's. The timestamp is the requester's transmit time, echoed unchanged in the response, so that the exchange measures a round trip.

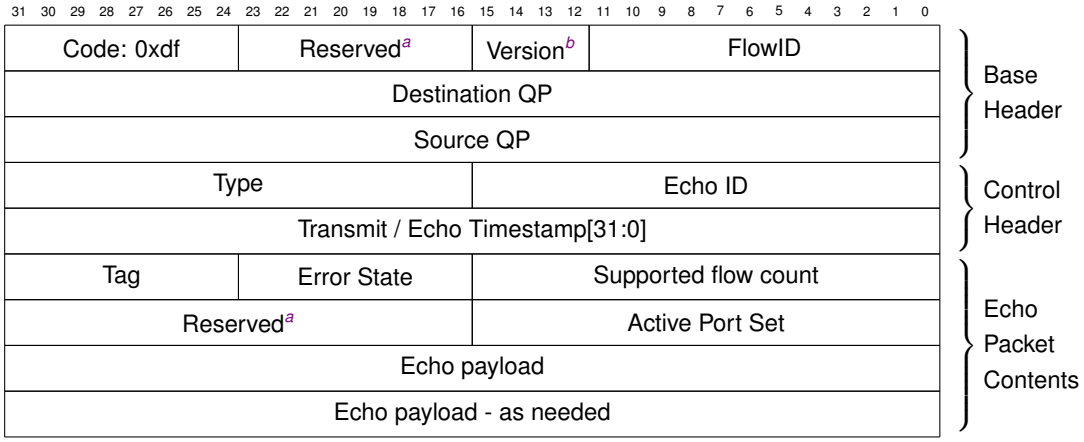
Echo is control message type 0, the echo response is control message type 1. The ID field allows the sender to match responses.

The error state of the ping is zero in the echo request header, but should be populated in the echo response using one of the codes in Table 8.6 if any of the conditions are true.

Error State	Description
0	No reported error
1	QP not present / unknown
2	Reserved
3	Tag not associated with QP
4	Source address not associated with QP
5	Destination address not associated with flow
6	Flow ID unsupported / too large

Table 8.6: Echo response error state codes. A single code may be returned even if multiple errors apply.

The supported flow count in the ping header shall be set to the number of supported TX flows on the sender side (for informational purposes) and shall be updated in the ping response by the receiver to reflect



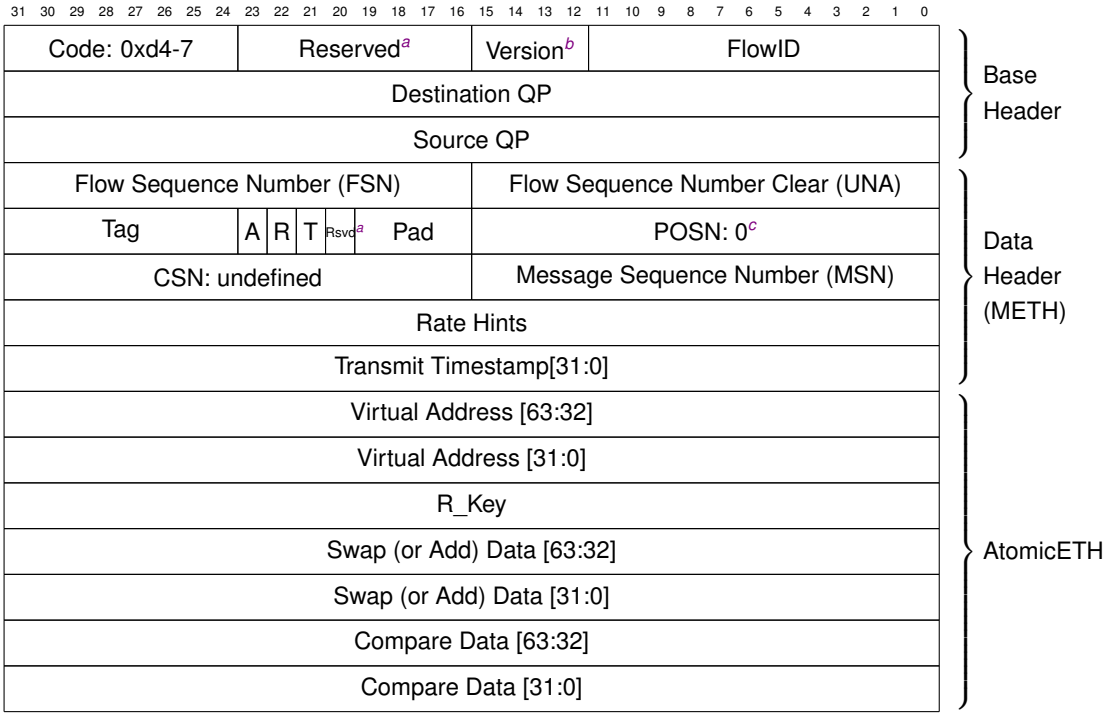
[a]Reserved fields are set to zero on transmission; ignored on receipt
[b]Version must be zero; verified to be zero on receipt

Figure 8.8: Echo request and echo response packet format.

the number of supported RX flows on the receive side. A firmware deciding how to allocate flow-related resources to QPs may use this information to match TX configuration to RX or determine whether there is a potential for error.

8.8 Atomic Headers

8.8.1 Atomic Extended Transport Header (AtomicETH)



[a]Reserved fields are set to zero on transmission; ignored on receipt
[b]Version must be zero; verified to be zero on receipt
[c]POSN is zero; the message has only one packet

Figure 8.9: Atomic Extended Transport Header (AtomicETH)

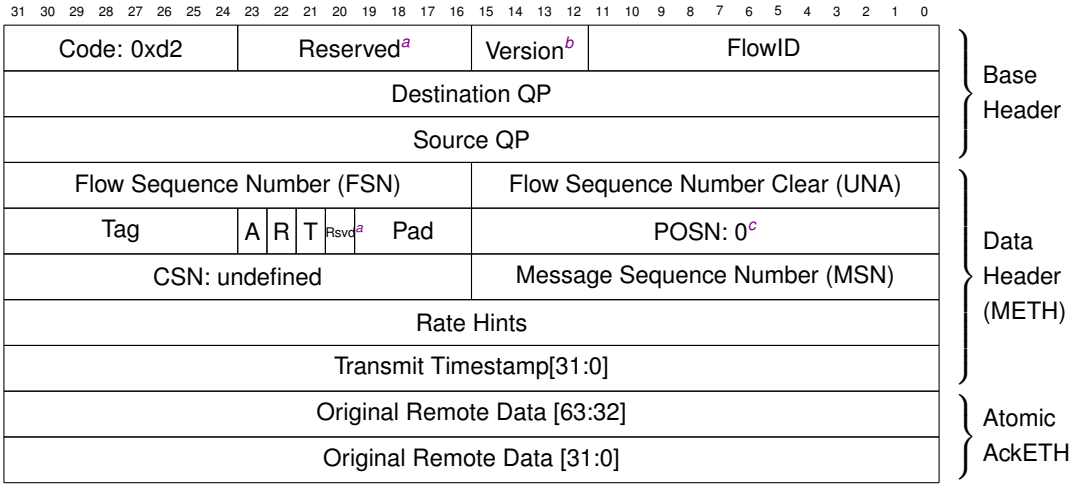
The atomic is a single packet message, meaning POSN is always zero, and has an undefined CSN because there is no corresponding receive WQE.

8.8.2 Atomic Ack Extended Transport Header (AtomicAckETH)

The Atomic Ack is the response to the Atomic operation, containing the previous state of the field. For FetchAdd, this contains the fetched value before addition. For CmpSwap, this contains the value before the comparison (which may or may not have matched the compare data in the request).

The atomic ack is a single packet message, meaning POSN is always zero, and carries the MSN of the original atomic.

Note: the “Atomic Ack” does not acknowledge data. The FSN of the atomic request is acknowledged by the normal rules (Section 4.2.2), as for any other data packet, or implicitly by the response where the sender implements that optimization (Section 4.2.1).



[a]Reserved fields are set to zero on transmission; ignored on receipt
[b]Version must be zero; verified to be zero on receipt
[c]POSN is zero; the message has only one packet

Figure 8.10: Atomic ACK Extended Transport Header (AtomicAckETH)

8.8.3 Send Last Opcode

Three opcodes mark the final packet of a send operation: Send with Immediate Present, Send with Invalidate, and Send Last. Send Last is used when neither immediate data nor R_Key invalidation is needed. Support for Send Last is optional, and where it is supported its use can be disabled, at the device and per queue pair. Without it, a plain send ends on a final packet shorter than the MTU, or exactly fills the receive buffer. Where it is unsupported, send with immediate conveys a limited-size message at the cost of an immediate.

The send message length may be shorter than the posted receive buffer. On receiving a last-packet opcode (Send Last, Send with Immediate Present, or Send with Invalidate), the responder learns the final POSN of the message: no packets with a higher POSN will arrive. If packets arrived out of order, the responder must still wait for all preceding POSNs to be delivered before completing the receive WQE. The completion reports the actual amount of data delivered, which may be less than the posted buffer length.

The delivered length of a send is the sum of its packets' payload lengths, so padding on the last packet that the transmitter did not record in the Padding field (Section 8.2) is delivered as data and inflates the reported length. Where the message length is not a multiple of 4 bytes and the message is shorter than the receive buffer, setting the Padding field is what gives the send an exact delivered length.

All preceding packets of a multi-packet send use the plain Send opcode (00000) regardless of which last-packet variant ends the message.

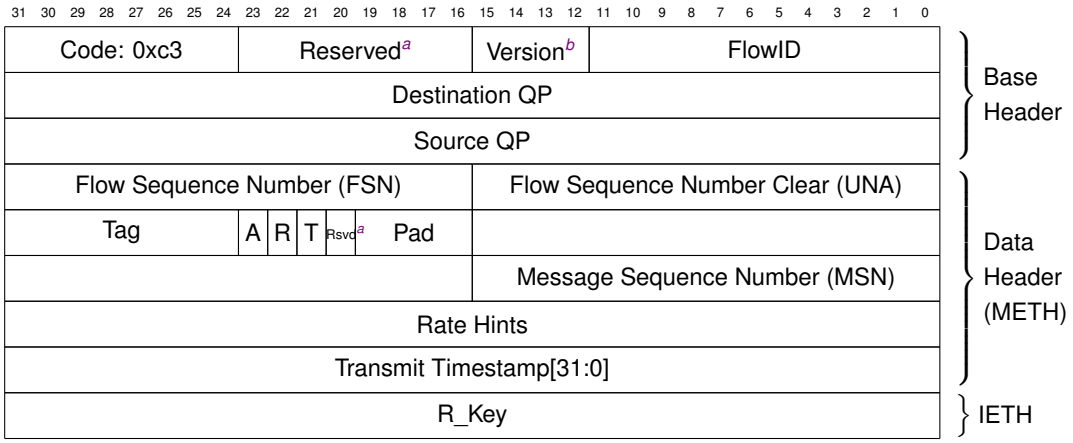
8.8.4 Invalidate Extended Transport Header (IETH)

The Invalidate Extended Transport Header is used with the Send with Invalidate opcode. Only the last packet of a send-with-invalidate message uses this opcode and the IETH header; all preceding packets use the plain Send opcode (00000). This mirrors the pattern used for Send with Immediate Present, where the

immediate data header appears only on the last packet.

The IETH comprises a single R_Key that the responder invalidates upon successful completion of the send operation. The invalidation is atomic with the completion: when the responder posts the CQE for this message, the specified R_Key is no longer valid for remote access.

Send with Invalidate and Send with Immediate are mutually exclusive; there is no combined opcode.



[a]Reserved fields are set to zero on transmission; ignored on receipt

[b]Version must be zero; verified to be zero on receipt

Figure 8.11: Invalidate Extended Transport Header (IETH). The R_Key identifies the memory region to be invalidated at the responder. This header follows the METH and precedes the payload.

The IETH format is identical to the RoCEv2 IETH.

Part V

Implementation

Chapter 9

Implementation Conformance Tables

Section 9.1 states what an implementation provides and at what level. Section 9.2 states the rules a conforming transmitter's packets satisfy.

9.1 Required and Optional Implementation Features

In the following, must, should, and may represent levels of expected features, drawn roughly from IETF RFC-2119.

MUST Specifies a behavior that is necessary for interoperability, suitability for expected deployment, correct operation, or device management. "MUST" behaviors in protocol are considered necessary for interoperation. "MUST" configurations are considered necessary for adapting to expected deployments. "MUST" counters are considered necessary for management.

Should Specifies a behavior that is likely to improve performance, reliability, readiness for future deployment, and manageability. "Should" behaviors in protocol are expected, but if missing would hurt only performance, not correctness. "Should" counters are desirable for adding detail and simplifying debugging.

may Specifies a behavior that is optional. "may" behaviors are not expected of an implementation, and implementations that differ in whether they provide one must still interoperate.

Sections may define required behaviors within optional features. The heading states the level of the feature, and the entries below apply to an implementation that provides it.

The counters below are largely not discussed in the body of this document, which reads better for describing a behavior without also saying that it is counted. Implementations are expected to instrument differently, and are not limited to this list. Some events cannot be attributed to a queue pair, and are counted per device only.

Feature	Level	Reference
RDMA Operations		
Configure Outstanding read and atomic requests (global)	may	5.5, Tbl 10.1
Configure Outstanding read and atomic requests (per-QP)	MUST	5.5, Tbl 10.1
max_rd_atomic in verbs. The responder's own limit is implementation-defined.		
Send	MUST	8.1.1
Send/imm	Should	8.1.1
Send with Invalidate	may	8.8.4
Write	MUST	8.1.1
Write/imm	MUST	8.1.1
Read	Should	8.1.1
Read/imm	Should	8.1.1
Atomics *(see Atomic Operations)*	Should	8.1.1
Responder capabilities		
Accepts a send shorter than the posted receive buffer	Should	8.8.3
The completion reports the delivered length.		
Supports more than one outstanding read	Should	3.2.3
Atomic Operations	Should	8.1.1
Atomic FetchAdd	MUST	8.1.1
Atomic CmpSwap	MUST	8.1.1
Sends Invalid Request NAK, QP error, for a misaligned or unsupported atomic	MUST	5.5
Responder supports more than one atomic	Should	5.5
Sequence Numbers		
Configure initial MSN, RX (per-QP)	MUST	2.2, 10.3
Configure initial MSN, RX (per-Tag)	Should	2.2, 10.3
The initial MSN must be configurable via modify-qp IBV_QP_RQ_PSN		
Configure initial MSN, TX (per-QP)	MUST	2.2, 10.3
Configure initial MSN, TX (per-Tag)	Should	2.2, 10.3
The initial MSN must be configurable via modify-qp IBV_QP_SQ_PSN		
Configure initial CSN, RX (per-QP)	MUST	2.2, 10.3
Configure initial CSN, RX (per-Tag)	Should	2.2, 10.3
The initial CSN should be configurable via modify-qp IBV_QP_RQ_PSN		
Configure initial CSN, TX (per-QP)	MUST	2.2, 10.3
Configure initial CSN, TX (per-Tag)	Should	2.2, 10.3
The initial CSN should be configurable via modify-qp IBV_QP_SQ_PSN		
Configure initial CSN limited to 255 (8-bits) (per-QP)	may	2.2, 10.3
Configure initial CSN limited to 255 (8-bits) (per-Tag)	may	2.2, 10.3
Support for configuring the initial CSN is permitted to be limited to what can pass through a 24-bit IBV_QP_RQ_PSN and IBV_QP_SQ_PSN.		
MSN and CSN can wrap	MUST	2.2
POSN cannot wrap	MUST	2.3
Delivers each FSN exactly once	MUST	2.5

Additionally prevents delivery of duplicate data at the per-message POSN level	may	2.3
Sends every packet of a message at full payload size, except the last	MUST	2.3
Retransmits data under the (Flow,FSN) it was first sent with, so the same data is not outstanding under two at once	MUST	2.5
Sends no packet it is unable to retransmit	MUST	4.4.1
MSN and CSN are per tag	MUST	2.2
Ignores the CSN on an operation that consumes no receive work queue entry	Should	3.4
Tags		
QP supports at least four tags	Should	10.6
QP supports up to 256 tags	may	10.6
Tag has a separate MSN/CSN space	MUST	2.2
Rejects tag creation with a non-matching protection domain	MUST	10.6
Each tag supports a separate work queue	Should	10.6
When using verbs library, separate work queues are expected		
Each tag supports a separate completion queue	may	10.6
Ordering is preserved within a tag	MUST	3
Eventually sends a CDMSN for all active tags	MUST	3.4.4
CDMSN acknowledgment on an ack is for most recently received tag	may	8.5.5
CDMSN acknowledgment on an ack round robins across tags	may	8.5.5
Ordering		
Write/imm sequenced after write	MUST	3.2.1
Does not execute an atomic until prior vRWQEs are complete	MUST	3.4.3
Network-reordered sends match corresponding recv	MUST	3.4
Acknowledgment never received before data reaches memory.	MUST	4.2.2
Provides an execution fence that holds a work request and those that follow until prior work requests complete	MUST	3.1, 3.4.3
In ordered mode, posts a message's completion only after all of its received data is delivered and all prior messages are complete, so completions appear in work request order	MUST	3.3.1
Unordered completion mode	may	3.3.2
Selects unordered completion per tag, and per send and receive queue	MUST	3.3.2
Completes receive work queue entries out of order	MUST	3.3.2
Completes send work queue entries out of order	Should	3.3.2
Honors the completion fence bit on a work queue entry	Should	3.3.3
Shared receive queue	may	3.4.1
Takes a buffer in message arrival order	MUST	3.4.1
Does not deadlock where messages arrive out of order	MUST	3.4.1
By bounding admission to the queue's length past the next expected CSN, or by reserving the skipped receives privately.		
Mixes tags on private and shared receive queues on one queue pair	may	3.4.1
Flushes a drawn buffer in error rather than returning it to the queue	MUST	5.3.4
CRC Error detection		
Discards packets with invalid CRC	MUST	4.1

Does not reject packets with zero UDP checksum	MUST	4.1
Counts packets discarded for an invalid CRC (global)	MUST	4.1
Encapsulation		
Configure UDP destination port (global)	MUST	4.1
2766 by default.		
Configure QP MTU (per-QP)	MUST	2.3.1, Tbl 10.1
Fixed once the first work request is posted, and agreed by both endpoints.		
4096 byte RDMA MTU Supported	MUST	2.3.1
1024 byte RDMA MTU Supported	Should	2.3.1
2048 byte RDMA MTU Supported	Should	2.3.1
8192 byte RDMA MTU Supported	Should	2.3.1
IPv6/UDP	MUST	4.1
IPv4/UDP	Should	4.1
Ethernet *(see Ethernet Encapsulation)*	may	4.1
ESUN	may	4.1
IPv6/PSP	Should	4.1
Under PSP encapsulation, does not carry mCRC	MUST	4.1
The PSP integrity check value covers the packet end to end.		
IPv6 no UDP	may	4.1
IPv4 no UDP	may	4.1
Does not reject packets with nonzero reserved fields	MUST	8
Discards packets with nonzero version field	MUST	8.1.2
Discards a packet shorter than the smallest valid packet for its opcode	MUST	8.1.5
Validates source QP matches expected remote QP	MUST	5.2.4
Pads the payload to a 4-byte boundary	MUST	8
Accepts payloads not padded to a 4-byte boundary	may	8
Sets the Padding field to the number of padding bytes added	Should	8.2, 8
Derives payload length from a nonzero Padding field	MUST	8.2
Outside direct Ethernet, discards up to 3 bytes of undisclosed padding	MUST	5.2.5
Counts packets discarded for a source QP mismatch (global)	MUST	5.2.4
Counts packets discarded for a source QP mismatch (per-QP)	may	5.2.4
Counts packets discarded as shorter than their opcode allows (global)	MUST	8.1.5
Counts packets discarded for a nonzero version field (global)	MUST	8.1.2
Ethernet Encapsulation	may	4.1
Pads to the minimum frame size	MUST	4.1
Records the pad count in the Padding field	MUST	4.1
Does not carry mCRC	MUST	4.1
Acknowledgment request bit		
At end of window	Should	4.2.1
At end of batch	Should	4.2.1
On short packets	Should	4.2.1
When flow window is small	Should	4.2.1

On retransmissions	MUST	4.2.1
On first packet during fast start	MUST	4.2.1
On first packet of each flow during fast start	Should	4.2.1
When out of packets to send	MUST	4.2.1
Acknowledgment generation		
A single acknowledgment may satisfy several of these conditions at once; each prepares an acknowledgment rather than requiring its own transmission.		
When requested	MUST	4.2.2
When received out of order	MUST	4.2.2
Before the count of CE-marked packets would exceed 15	MUST	4.2.2, 8.5.1
When received out of window (e.g. old retransmission)	MUST	4.2.2
Omits the ack request bit on a read or atomic request, settling the request on its response instead	may	4.2.1
At end of message	Should	4.2.2
Counts acks sent (global)	MUST	
Counts acks sent (per-QP)	may	
Counts acks sent with CNP (global)	MUST	
Counts acks sent with CNP (per-QP)	may	
Counts acks sent with OOO code (SACK) (global)	MUST	
Counts acks sent with OOO code (SACK) (per-QP)	may	
Acknowledgment contents		
Processes the cumulative FSN of every accepted acknowledgment	MUST	8.5
Required whatever the Ack Status, and whether or not a bit vector is present.		
Accepts an Ack Status 0x1 acknowledgment whose FSN bit vector is abbreviated, where it spans CumulativeFSN through MaxFSN	MUST	8.5
Where MaxFSN lies more than 256 beyond CumulativeFSN, the full 32 bytes are the most that can be sent.		
Accepts a BRNR-NAK whose RNR bit vector is absent or abbreviated	MUST	8.5.10
A bit the vector does not carry is read as zero, marking that packet not RNR'd.		
Processes the cumulative FSN of a BRNR-NAK too short to carry both bit vectors	MUST	8.5
The remainder is discarded; the acknowledgment itself is not.		
Ignores a bit vector the Ack Status does not call for, accepting the acknowledgment	MUST	8.5, 8.5.6
Processes the FSN bit vector of a received SACK	Should	8.5.9
A receiver that does not process the bit vector advances only the cumulative FSN, treating the acknowledgment as in-order.		
Rejects an acknowledgment with a MaxFSN before snd.una	MUST	4.2.3
Rejects an acknowledgment with a CumulativeFSN at or after snd.nxt	MUST	4.2.3
Rejects an acknowledgment with a MaxFSN at or after snd.nxt	MUST	4.2.3
Rejects an acknowledgment with a CDMSN at or after the Next Transmit MSN	MUST	4.2.3
In implementations that use the CDMSN to complete messages, for a Tag the sender holds.		
Rejects an acknowledgment with a MaxFSN before its CumulativeFSN	MUST	4.2.3

Rejects an acknowledgment older than MaxAckAge round trip times	MUST	4.2.3
No part of it is applied, including the cumulative FSN.		
Sends an FSN bit vector on an acknowledgment whose Ack Status is 0x1	MUST	8.5
Sends an RNR bit vector on a BRNR-NAK	MUST	8.5
Sends the full 32-byte FSN bit vector on a BRNR-NAK	MUST	8.5
The length of the RNR bit vector then follows from the framed length of the packet.		
Counts acks discarded for a CumulativeFSN at or after snd.nxt (global)	MUST	4.2.3
Counts acks discarded for a CumulativeFSN at or after snd.nxt (per-QP)	may	4.2.3
Counts acks discarded for a CDMSN at or after the Next Tx MSN (global)	MUST	4.2.3
Counts acks discarded for a CDMSN at or after the Next Tx MSN (per-QP)	may	4.2.3
Counts acks discarded for a MaxFSN before the CumulativeFSN (global)	may	4.2.3
Counts acks discarded for a MaxFSN before the CumulativeFSN (per-QP)	may	4.2.3
Counts acks discarded as older than MaxAckAge (global)	MUST	4.2.3
Counts acks discarded as older than MaxAckAge (per-QP)	may	4.2.3
RTO/RTT		
Configure Maximum RTO upper bound (ms) (global)	MUST	4.5.2, 4.2.3
Configure Maximum RTO upper bound (ms) (per-QP)	Should	4.5.2, 4.2.3
Configure Minimum RTO lower bound (μs) (global)	MUST	4.5.2
Configure Minimum RTO lower bound (μs) (per-QP)	Should	4.5.2
Configure Offset RTO ρ (μs) (global)	MUST	4.5.2
Configure Offset RTO ρ (μs) (per-QP)	Should	4.5.2
Configure RTO retransmission count (single packet, n packets, or unlimited) (global)	MUST	4.5.3
Configure RTO retransmission count (single packet, n packets, or unlimited) (per-QP)	Should	4.5.3
Configure RTO retries before changing flow entropy (global)	may	4.8
Configure RTO retries before changing flow entropy (per-QP)	may	4.8
Configure Linear RTO retries before exponential backoff (global)	may	4.5.2
Configure Linear RTO retries before exponential backoff (per-QP)	may	4.5.2
Number of successive timeouts retransmitted with linear (constant) spacing before switching to exponential back-off, analogous to Linux tcp_syn_linear_timeouts.		
Configure Default RTT estimate before the first measurement (global)	MUST	6.1.3, 10.4
Configure Default RTT estimate before the first measurement (per-QP)	Should	6.1.3, 10.4
Used by the fast start exit timer on a QP that has not yet measured an RTT. May be seeded from the QP timeout or set by the driver.		
Configure MaxAckAge (round trip times) (global)	MUST	4.2.3
Configure MaxAckAge (round trip times) (per-QP)	may	4.2.3
Age past which an acknowledgment is discarded, defaulting to 64. Configured below 32768/$FWND_{max}$ so that the flow sequence space cannot wrap within the permitted age.		
Configure RTT smoothing alpha_qp (global)	MUST	4.5.1

Configure RTT smoothing alpha_f (global)	may	4.5.1
Applies where per-flow RTT estimation is maintained.		
Accepts a QP timeout value of at least 20	MUST	10.4
Keeps one retransmission timer per flow, reset on every new packet sent on it	MUST	4.5.2
Resets the RTO timer on an acknowledgment only to a sooner expiration	MUST	4.5.2
Advances the transmit timestamp at approximately nanosecond granularity	Should	8.2.2
A cycle counter of a 0.8GHz clock is acceptable.		
Initializes smoothed RTT and deviation on first measurement	Should	4.5.1
Smoothed RTT and mean deviation persist across idle periods and fast-start re-entry	MUST	4.5.1
Per-QP smoothed RTT tracking	MUST	4.5.1
Per-QP mean RTT deviation	MUST	4.5.2
Per-Flow smoothed RTT tracking	Should	4.5.1
Per-Flow mean RTT deviation	Should	4.5.2
Minimum RTO can be at least 5ms	may	4.5.2
Minimum RTO can be set to 10μs	may	4.5.2
Maximum RTO can be at least 100ms	may	4.5.2
Maximum RTO can be set to 1ms	may	4.5.2
Retransmissions by timeout not prevented by congestion window	Should	4.5.3
Optional exponential RTO backoff after linear backoff	may	4.5.2
Counts retransmissions by timeout sent (global)	MUST	4.5.3
Counts retransmissions by timeout sent (per-QP)	may	4.5.3
Counts bucketized measured rtt samples (global)	Should	4.5.1
Counts bucketized measured rtt samples (per-QP)	may	4.5.1
Reliability / SACK / Retransmission		
Configure Enable or disable retransmission by SACK (global)	MUST	4.6.1
Configure Enable or disable retransmission by SACK (per-QP)	may	4.6.1
Configure Immediate or delayed retransmissions by SACK (global)	MUST	4.6.1
Configure Immediate or delayed retransmissions by SACK (per-QP)	may	4.6.1
Configure SACK reordering tolerance, a fraction of the flow window (global)	MUST	4.6.1
Configure SACK reordering tolerance, a fraction of the flow window (per-QP)	may	4.6.1
How far below MaxFSN snd.cur may not advance, in the delayed mode.		
Configure Traffic class for ack traffic (global)	MUST	4.2.2
Configure Traffic class for ack traffic (per-QP)	may	4.2.2
ACKs in alternate (expected to be higher priority) traffic class	Should	4.2.2
Not delayed / collapsed by more than 2 microseconds	Should	4.2.2
Sends ACK with correct state upon unexpected, out-of-sequence frame	MUST	4.2.2
Can set $\texttt{FWND}_{max}$to 256	MUST	6.1.1
Can set $\texttt{FWND}_{max}$to 128	MUST	6.1.1
Can set $\texttt{FWND}_{max}$to 64	MUST	6.1.1
Solicits a CDMSN by retransmitting prior data where none is pending	may	8.5.5
Can retransmit by SACK on the first indication of a missing frame	MUST	4.6

Scheduled against other queued work, so it need not be the next packet sent.		
Can retransmit by SACK on delayed indication of a missing frame, to tolerate in-network packet spray	MUST	4.6.1
Does not retransmit an FSN again on a later selective acknowledgment that still shows it missing, absent an intervening RTO	MUST	4.6
Does not retransmit by selective acknowledgment beyond the MaxFSN it reports	MUST	4.6
Those beyond MaxFSN are not lost until a later ack reports them or the RTO expires.		
Continues retransmission after a partial RTO burst from the burst endpoint	MUST	4.6
Does not advance MaxFSN to a received-but-discarded out-of-window packet	MUST	4.2.3, 8.5.4
Retransmissions are marked		
In the retransmission bit of the data header	MUST	4.6, 8.2
When by timeout, in the timeout bit of the data header	Should	4.6, 8.2
In the hop limit field of the encapsulating header, absent an entropy change	may	4.2.1, 4.6
Expected where the Meta-RoCE frame is encapsulated with PSP.		
Counts retransmissions sent (global)	MUST	
Counts retransmissions sent (per-QP)	may	
Counts retransmissions by selective acknowledgment sent (global)	MUST	4.6
Counts retransmissions by selective acknowledgment sent (per-QP)	may	4.6
Counts spurious retransmissions received (global)	MUST	8.2
Counts spurious retransmissions received (per-QP)	may	8.2
a spurious retransmission is a duplicate packet, already acked.		
Counts necessary retransmissions received (global)	MUST	8.2
Counts necessary retransmissions received (per-QP)	may	8.2
a necessary retransmission is a marked retransmission that has not previously been received.		
RNR		
Configure Default B-RNR backoff time (global)	MUST	8.5.6
Configure Default H-RNR backoff time (global)	MUST	8.5.6
Holds data transmission in response to H-RNR	MUST	8.5.6
Continues acknowledgments, read responses, and atomic acknowledgments while held by H-RNR or B-RNR	MUST	4.2.2
Sends H-RNR when a hold for synchronization is required	Should	8.5.6
RNR-Cancel *(see RNR-Cancel)*	Should	4.10.1
Drops an RNR-Cancel that finds no transmission held	MUST	4.10.1
Receive RNR-Cancel will restart sender	MUST	4.10.1
Restarts every ready tag on an RNR-Cancel that follows an HRNR	MUST	4.10.1
The initiator knows the hold covered every tag, so one cancel ends it.		
Receive ack of packet within BRNR'd message will restart sender	Should	4.10.2
Does not BRNR a packet of a message after acknowledging a packet of that message	MUST	4.10.2
Transmits B-RNR when corresponding RWQE is not present	MUST	4.10
Handles incoming B-RNR by transmitting prior data on a new (Flow,FSN)	MUST	4.10

Halts new traffic on a Tag and MSN identified by a received RNR bitset, and on later messages of that Tag identified by a CSN	MUST	4.10
Advances the UNA field past FSNs acknowledged as delivered or set in the RNR bitset	MUST	4.10, 4.9
Holds CumulativeFSN at the highest contiguously delivered FSN before the first RNR'd packet	MUST	4.10
Advances CumulativeFSN on receipt of an accepted data packet whose UNA field exceeds it, releasing the per-FSN state below	MUST	4.10, 4.9
Resumes transmission of B-RNR'd message after required delay	MUST	8.5.6
Reads a nonzero RNR backoff field as a shift of 5 microseconds, giving 10 microseconds through 163.84 ms	MUST	8.5.6
Replaces the backoff timer with the duration a newly received RNR carries, whether larger or smaller	Should	8.5.6
Handles incoming B-RNR without blocking new and existing messages on concurrent tag	MUST	4.10
Counts BRNR probes sent (global)	MUST	4.10
Counts BRNR probes sent (per-QP)	may	4.10
Counts HRNR sent (global)	MUST	
Counts HRNR received (global)	MUST	
Counts HRNR received (per-QP)	may	
Counts BRNR acks sent (global)	MUST	
Counts BRNR acks sent (per-QP)	may	
Counts BRNR bits received (global)	MUST	
Counts BRNR bits received (per-QP)	may	
RNR-Cancel	Should	4.10.1
Sends RNR-Cancel when the hold is no longer required	MUST	4.10.1
Sends an RNR-Cancel for each BRNR'd tag as it becomes ready	MUST	4.10.1
Sends one RNR-Cancel for the queue pair after an HRNR	MUST	4.10.1
Where RNR-Cancel is supported on a shared receive queue, reduces Posted CSN to the next expected	MUST	4.10.1, 3.4.1
Send RNR-Cancel on RWQE post	Should	4.10.1
Send RNR-Cancel when QP enters cache	may	4.10.1
Send RNR-Cancel on RWQE post only after prior RNR	may	4.10.1
Standard IB/RoCE error handling interface		
Expectations Meta-RoCE inherits from IB/RoCE rather than defines. They are observable through the verbs interface rather than on the wire.		
Configure Place the tag in error on CQ overflow (global)	may	5.3.5
Configure Place the tag in error on CQ overflow (per-QP)	may	5.3.5
Completes a work request in error for a message-fatal error attributable to it	MUST	5.3, 5.3.2
Sends an asynchronous error notification for a message-fatal error not associated with a WQE	MUST	5.3
On placing a queue in error, completes a fully executed but uncompleted work request as Complete - flushed, and one not fully executed as Flushed in Error	MUST	5.3.3
Flushes in error any work request posted to a queue already in the error state	MUST	5.3.3

Places a private receive queue in error with its QP, a shared one not	MUST	5.3.4
On CQ overflow, reports an asynchronous error event on the completion queue that overflowed	MUST	5.3.5
On CQ overflow, reports a second asynchronous error event on the tag whose completion could not be written, once for each such tag	Should	5.3.5
Leaves a queue error state only when software returns the queue to Reset or destroys it	MUST	5.3.3
Confines an error to the tag it is attributable to, leaving the other tags of the queue pair operational	Should	5.3
Returns a single tag's work queue to Reset, leaving the rest of the queue pair operational	may	5.3.3
Errors		
Configure WQE timeout duration (global)	MUST	5.4
Configure WQE timeout duration (per-QP)	Should	5.4
Configure HRNR retry limit (global)	may	5.4
Configure HRNR retry limit (per-QP)	may	5.4
May be expressed as a duration.		
Configure BRNR retry limit (global)	may	5.4
Configure BRNR retry limit (per-QP)	may	5.4
May be expressed as a duration.		
Configure Local timeout retry count (retry_cnt) (per-QP)	MUST	10.4
Configure QP timeout (global)	MUST	10.4
Configure QP timeout (per-QP)	MUST	10.4
The QP timeout attribute, which the driver may treat as a hint or replace with its own configuration.		
Sends flow ID unsupported NAK	MUST	8.5.6, 5.2.1
Handles flow ID unsupported NAK	MUST	8.5.6
Handles flow ID unsupported NAK by rescheduling frames on lower numbered flows	Should	8.5.6
Rate-limits flow ID unsupported NAKs	may	5.2.1
Sends BRNR-NAK (or optionally HRNR) for unknown tag, or Operational Error where tags are unsupported	Should	8.5.6, 5.2.2
Handles BRNR-NAK timeout for a tag by completing in error	MUST	8.5.6
Completes a work request in progress in error when flow entropy changes and port fail-overs have failed to restore connectivity	MUST	5.3.1
Places the queue pair and its work queues in error when connectivity cannot be restored	Should	5.3.1
Sends Address Invalid NAK when destination memory region does not exist	MUST	5.5.1, 8.5.6
Sends Address Invalid NAK when destination memory region is not writable	MUST	5.5.1, 8.5.6
Sends Address Invalid NAK when destination memory region is not readable	MUST	5.5.1, 8.5.6
Sends Invalid Request NAK when destination memory region is undersized for received data	MUST	5.5
Sends Invalid Request NAK, QP error, for a forbidden zero-byte payload	MUST	8.2.3, 5.5
Sends Invalid Request NAK, QP error, for unrecognized or unsupported opcode	MUST	5.5.2, 5.5

Silently discards packets with unrecognized transport type (opcode bits 7:5 not 110)	MUST	5.5.2
Sends HRNR where the QP cannot immediately be brought into cache	Should	5.2.3
Generates an error response (HRNR or Operational Error NAK) for a packet to a QP not in RTR or RTS	Should	5.2.6
Rate-limits error responses for a QP not in RTR or RTS	may	5.2.6
Sends Address Invalid or Operational Error NAK on local memory protection (L_key) error	MUST	5.5
Invalidates the R_Key of a send with invalidate as the completion is posted	MUST	8.8.4
Sends Address Invalid NAK on R_Key invalidation failure for Send with Invalidate	may	8.8.4, 5.5
Validates the MSN against the range from the CDMSN to its capacity to track concurrent messages	Should	5.5, 3.4.4
Handles an out-of-range MSN non-destructively	MUST	5.5
Does not positively acknowledge the FSN of a packet rejected for an MSN outside the tracked range, neither cumulatively nor in the FSN bit vector	MUST	5.5
Drops the payload of a data packet whose FSN is outside the receive window	MUST	5.5
Does not apply any field of a dropped data packet, including UNA	MUST	5.5
A packet delayed until after the sequence space wrapped would otherwise advance CumulativeFSN.		
Evaluates responder error checks in order, acting on the first applicable	Should	5.5
Handles a packet failing packet-level validation at the reliability layer, without reporting a message-level or payload-level error	MUST	5.5
Rejects an out-of-range MSN with an RNR-class NAK, leaving the QP operational	Should	5.5
Answers a read or atomic beyond the outstanding limit with an HRNR-NAK	MUST	5.5
Handles a read or atomic response with mismatched or missing MSN non-fatally	MUST	5.4
Drops a read or atomic response whose MSN matches no outstanding request	Should	5.4
Counts packets discarded as zero-length where the opcode forbids it (global)	MUST	8.2.3
Counts Non-RNR NAKs sent (by code) (global)	MUST	
Counts Non-RNR NAKs received (by code) (global)	MUST	
Counts Non-RNR NAKs received (by code) (per-QP)	may	
Baseline Congestion Control		
Configure ϵ (additive increase) (global)	MUST	6.1.2, Tbl 6.1
Configure ϵ (additive increase) (per-QP)	Should	6.1.2, Tbl 6.1
Configure β (ECN decrease) (global)	MUST	6.1.2, Tbl 6.1
Configure β (ECN decrease) (per-QP)	Should	6.1.2, Tbl 6.1
Configure λ (loss decrease) (global)	MUST	6.1.2, Tbl 6.1
Configure λ (loss decrease) (per-QP)	Should	6.1.2, Tbl 6.1
Configure $\texttt{QWND}_{min}$(global)	MUST	6.1.2
Configure $\texttt{QWND}_{min}$(per-QP)	may	6.1.2
Configure $\texttt{FWND}_{max}$(global)	MUST	6.1.1, 4.2.3
Configure $\texttt{FWND}_{max}$(per-QP)	Should	6.1.1, 4.2.3

Bounded together with MaxAckAge, which must be less than 32768/$\texttt{FWND}_{max}$, so that the flow sequence space cannot wrap within the age an acknowledgment may reach.		
All data packets are sent with ECN-Capable-Transport ECT(0)	MUST	6.1
Counts ECN into CNP	MUST	8.5.1
Increments fractional QWND by ϵ per acked uncongested packet from a window-limited flow, unless $\texttt{QWND} \geq \texttt{QWND}_{max}$	MUST	6.1.2
Increments QWND and FWND by one per acked packet where $\epsilon > \texttt{QWND}$	MUST	6.1.2
Growth is then comparable to TCP slow start.		
Deactivates a flow whose FWND is reduced to zero by congestion or loss	MUST	6.1.2, 7.4.1
Uses CNP to reduce fwnd and qwnd	MUST	6.1.2
Treats a selectively acknowledged or RNR'd packet as no longer occupying the flow window, so an unfilled hole does not stall transmission on the flow	MUST	6.1.1
Does not decrement QWND below $\texttt{QWND}_{min}$	MUST	7.4.1
Keeps a flow active where decrementing FWND[f] would take QWND below $\texttt{QWND}_{min}$	MUST	7.4.1
Skip flow window increase when flow RTT exceeds smoothed RTT	MUST	6.1.2
Required capability; may be disabled in operation		
A bootstrapping flow claims window increases up to the fast start burst	MUST	6.1.2, 7.4.2
A congestion signal on that flow also ends the credit, and an eligible flow whose FWND is below the burst may claim an increase instead.		
Fast start *(see Fast Start)*	Should	6.1.3
Counts Times prevented from reducing below $\texttt{QWND}_{min}$(global)	MUST	
Counts Times prevented from reducing below $\texttt{QWND}_{min}$(per-QP)	may	
Counts Times prevented from exceeding $\texttt{QWND}_{max}$(global)	MUST	
Counts Times prevented from exceeding $\texttt{QWND}_{max}$(per-QP)	may	
Fast Start	Should	6.1.3
Configure Initial $\texttt{QWND}_{max}$(fast start $\texttt{QWND}_{max}$) (global)	MUST	6.1.3
Configure Initial $\texttt{QWND}_{max}$(fast start $\texttt{QWND}_{max}$) (per-QP)	Should	6.1.3
Configure Fast Start Burst (global)	MUST	6.1.3, Tbl 6.1
Configure Fast Start Burst (per-QP)	Should	6.1.3, Tbl 6.1
Configure Fast start idle interval (ms) (global)	may	6.1.3
Configure Fast start idle interval (ms) (per-QP)	may	6.1.3
Supports a fast start burst limit of at least 16	MUST	6.1.3
Supports a fast start burst limit of at least 100	Should	6.1.3
Enters fast start when a Send, RDMA Write, or RDMA Read Response is posted on an idle QP	Should	6.1.3
Begins fast start on flow 0	Should	6.1.3
Begins fast start on a flow of every available port	MUST	6.1.3, Fig 6.3
Recommended: round robin across ports, each filling to the burst before the next.		
After fast start exits, holds each flow until an acknowledgment arrives on it	MUST	6.1.3
Exits fast start on the first acknowledgment received while data is pending	MUST	6.1.3

Exits fast start on an acknowledgment whose CNP field is nonzero, whether or not data is pending	MUST	6.1.3
Exits fast start when QWND reaches $\texttt{QWND}_{max}$	MUST	6.1.3
Exits fast start after an estimated RTT with data pending, bounding transmission without feedback	MUST	6.1.3
Re-enter fast start after idle	may	6.1.3
Re-enters fast start on idle only after the configured idle interval	Should	6.1.3
Fast-start re-entry re-initializes the window but preserves the RTT estimate (SRTT, MDEV, RTO)	MUST	6.1.3
Advanced / Programmable Congestion Control, Outputs	Should	
Can read and modify ϵ (AIMD increase), per QP	MUST	6.2.1
Can read and modify λ (Loss backoff), per QP	MUST	6.2.1
Can read and modify β (ECN backoff), per QP	MUST	6.2.1
Can read and modify γ (RCN proportional), per QP	may	6.2.1, 6.3.2
Can read and modify ω (RCN constant), per QP	MUST	6.2.1, 6.3.2
Can modify QWND and FWND directly, per QP	Should	6.2, 6.2.1
Can disable the baseline reduction on a congestion mark and on loss	Should	6.2
Can read and modify QP rate limit	Should	6.2.1
where an additional rate limiter is available for pacing		
Advanced / Programmable Congestion Control, Inputs	Should	
Can read current QWND	MUST	6.2.2
Can read current QP AWND	MUST	6.2.2, 6.1.1
Can read per-QP SRTT, MINRTT	MUST	6.2.2
Can read RCN target rate	MUST	6.2.2
Can read all active flow windows	MUST	6.2.2
Can read count of packets delivered (clear on read)	MUST	6.2.2
Can read count of packets delivered and marked (clear on read)	MUST	6.2.2
Can read count of packets retransmitted (clear on read)	MUST	6.2.2
Can read count of packets retransmitted by timeout (clear on read)	MUST	6.2.2
Can determine the time since the last advanced CC evaluation	MUST	6.2.2, 6.2.3
satisfied by a device-supplied timestamp or by storing the current time in scratch state		
Can read per-flow AWND or per-flow data in flight	MUST	6.2.2
Can read per-flow count of packets delivered and marked (clear on read)	MUST	6.2.2
Can read per-flow count of packets delivered (clear on read)	MUST	6.2.2
Can read per-flow count of packets retransmitted (clear on read)	MUST	6.2.2
Can read per-QP MDEV	Should	6.2.2
Can read approximate count remaining packets to send	Should	6.2.2
Can read per-flow SRTT	Should	6.2.2
Can read per-flow count of packets retransmitted by timeout (clear on read)	Should	6.2.2
Can read QWND fractional	may	6.2.2
Can read per-flow MDEV	may	6.2.2
Can read the inbound TTL of a flow	may	6.2.2

Advanced / Programmable Congestion Control, Internal State	Should	
Can read and write to per-QP scratch state	Should	6.2.3
Provides at least 16 bytes of per-QP congestion state storage	Should	6.2.3
Can store current time into scratch state	Should	6.2.3
Can set and export algorithm-defined counters	Should	6.2.3
Receiver Driven Rate Limiting (RCN)		
Configure $\texttt{fwnd}_{floor}$(global)	MUST	6.3.3
Configure $\texttt{fwnd}_{floor}$(per-QP)	may	6.3.3
Configure γ (global)	MUST	6.3.2
Configure γ (per-QP)	may	6.3.2
Configure ω (global)	MUST	6.3.2
Configure ω (per-QP)	Should	6.3.2
Receiver sends target rate	MUST	6.3
Caps advertised target rate at the per-QP processing capacity	Should	6.3.1
Where per-QP throughput is limited by pipeline or thread partitioning below the link rate		
Reduces the applied target rate to the sender's own output capacity where that is lower	Should	6.3.2
Sender sends desired rate	Should	8.2.1
In fast start, sets the desired rate to the link rate, or to the link rate divided by the number of ready sender QPs	may	8.2.1
In normal operation, derives the desired rate as QWND $\times$ MTU $\times$ 8 / RTT ns	Should	8.2.1
Quantizing the smoothed RTT to a power of two makes the division a shift.		
Clamps the desired rate to no less than 1 and no more than the link capacity, in normal congestion control operation	MUST	8.2.1
Sender sends 0 desired rate within QWND of last message end	Should	8.2.1
Reduces QWND by half a packet per acked packet where 2 $\times$ $\texttt{QWND}_{max} \geq$ QWND $>$ $\texttt{QWND}_{max}$	MUST	6.3.3
Reduces QWND by a full packet per acked packet where QWND $>$ 2 $\times$ $\texttt{QWND}_{max}$	MUST	6.3.3
Does not reduce FWND[f] by RCN where QWND $>$ f $\times$ $\texttt{fwnd}_{floor}$and FWND[f] $\leq$ $\texttt{fwnd}_{floor}$	MUST	6.3.3
The first condition uses the flow ID so that low-numbered flows are protected deterministically.		
Does not reduce QWND directly to $\texttt{QWND}_{max}$by iterating across all FWND[f]	Should	6.3.3
Ignores an ECN-driven window reduction while reducing toward the RCN target rate	Should	6.3.4
Does not update the RTT component of $\texttt{QWND}_{max}$until after processing the per-flow FWND[f] reduction	MUST	6.3.3
Supports a γ of 1 through 2	Should	6.3.2
Converges the QP's effective rate on γ times the target rate received from the destination	MUST	6.3.2
Never sets $\texttt{QWND}_{max}$below the configured $\texttt{QWND}_{min}$	MUST	8.5.8, 6.3.2
Counts QWND reductions by RCN (global)	MUST	6.3.3
Counts QWND reductions by RCN (per-QP)	may	6.3.3

Counts Flow window to zero by RCN (global)	MUST	6.3.3
Counts Flow window to zero by RCN (per-QP)	may	6.3.3
Multipath		
Configure flow count (global)	MUST	7.3
Configure flow count (per-QP)	Should	7.3
Configure (independently) rx flow count (global)	Should	7.3
Configure (independently) rx flow count (per-QP)	Should	7.3
Configure (independently) tx flow count (global)	Should	7.3
Configure (independently) tx flow count (per-QP)	Should	7.3
To avoid a flow ID unsupported NAK, no greater than the peer's receive flow count.		
Always retains flow 0 in each direction	MUST	7.3
Sends new data on a flow only while its available window AWND[f] is positive	MUST	7.2, 6.1.1
Rotates the transmit scheduler across ready QPs before ready flows	Should	7.2
Adaptive rx flow count	Should	5.2.1
Consistent entropy per flow	MUST	7.1
Changes a flow's entropy value without changing its Flow ID, and keeps the flow's reliability state across the change	MUST	7
Selects an entropy value for each flow when a QP is created and when the flow count changes	MUST	7.1
Does not select entropy values at random	Should	7.1
Tx flow entropy changes on RTO	MUST	4.6, 4.8
Rx flow entropy changes on tx flow entropy change	MUST	7.1.1
Activates previously unused flows, so path diversity grows with the rate	MUST	7.4.2
Prefers the numerically lowest available flow when activating one	Should	7.4.2
Reactivates a flow at its next unused FSN, not at the start of the sequence	MUST	7.4.1
Acknowledges a reactivated flow from the retained CumulativeFSN	MUST	7.4.1
Counts entropy changes (global)	Should	5.1
Counts entropy changes (per-QP)	may	5.1
Multi-port	may	
Configure Local port migration limit (global)	MUST	5.4
Configure Local port migration limit (per-QP)	Should	5.4
Flow-level failures tolerated before escalating to port failover.		
Configure Port ready debounce duration (global)	Should	7.5.7
Accepts from any address bearing valid QP	MUST	8.1.5, 5.2.4
Flows of the same QP can use distinct destination addresses	MUST	10.8
Supports topology wiring other than one-to-one port pairing	may	7.5.2
Can distribute flows to output ports	MUST	7.5.3
Confirms the local port is ready, the remote port is ready and enabled, and the port pair exists in the topology, before transmitting on a flow	MUST	7.5.2
When ready to send, invalidates a cached port pair whose local port has failed	MUST	7.5.4
Can shift flows to working output ports	Should	5.1
Can shift flows to working destination port addresses	Should	5.1

Supports draining planes (disable tx without disabling rx)	Should	7.5
Marks a port not-ready on a physical-layer link-down event	MUST	7.5.5
Accepts a not-ready transition in the peer's port set immediately	Should	7.5.7
Initial flow-to-port assignment distributes flows evenly across ports	Should	7.5.3
Recommended base-relative flow-to-port assignment from a hash of QP numbers and a NIC seed	may	7.5.3
Does not deterministically fail over to the failed port plus one	MUST	7.5.4
Failover port selection gives each usable alternative equal probability	Should	7.5.4
Preserves the working port when failing over the other, where wiring allows	Should	7.5.4
Reports working port set in acknowledgments	may	7.5.5
Read and Write Headers		
Sends identical RETH address, R_Key, and length in every packet of a write message	MUST	8.3
Validates the RETH fields at least once per message	MUST	8.3
Errors if the RETH fields are inconsistent across packets of a message	may	8.3
Control Packets	Should	8.7
Supports reliable control, which carries a METH and is delivered as a reliable message	may	8.1.1, 8.7
Sends a flow ID of zero on a control packet that does not concern a flow	Should	8.7
Echoes the response identifier of a control request in its response	Should	8.7
Echoes the ping payload into the echo response	MUST	8.7.1
Reports the condition in the error state of an echo response	Should	8.7.1
Send Last opcode	Should	8.8.3
Alternatives are a short final packet, an exact-size receive, or immediate data.		
Configure whether Send Last is sent (global)	MUST	8.8.3
Configure whether Send Last is sent (per-QP)	MUST	8.8.3
Ends every send with a last-packet opcode	MUST	8.8.3
Only where supported and enabled.		

9.2 Packet validity

These rules describe what a conforming transmitter emits, each written so that it can be checked over a short packet capture. They are a test of the transmitter rather than a filter at the receiver: the grounds on which a receiver rejects a packet are in Sections 5.5 and 8.1.5, and breaking a rule here is not necessarily among them. A reserved field is the clearest case: it may carry meaning from a future revision of this specification, and a packet is not rejected for setting one.

The rules are grouped by header, in the order the headers appear, and within a group by the order the fields appear in that header. They have no requirement level, a packet either satisfying a rule or not. Where a field is not universally required, the obligation to set it is a feature in Section 9.1 and the rule here is phrased for the case where it is set.

Rule	Reference
All headers	
Reserved fields are zero	8
The packet length is a multiple of 4 bytes	8, 8
Base header	
The Version field is zero	8.1.2
The QP fields carry queue pair numbers, with no tag encoded in them	8.1.4
An acknowledgment or echo response swaps the answered packet's QP fields	8.1.4, Tbl 8.5
A last-packet opcode appears only on a send's final packet	8.8.3
Data header (METH)	
The UNA field is at or before the FSN in the same header	4.9
The Tag field carries the tag the work request identifies	8.2
A timeout retransmission sets R and T, repeating the FSN, POSN, and MSN	8.2
The Padding field, where nonzero, is the number of padding bytes appended	8
A read or atomic response carries the MSN of the request it answers	2.2
A Write or Write with Immediate carries a payload	8.2.3, Tbl 8.2
Table 8.2 gives the opcodes that permit a zero-byte payload.	
The desired rate is zero, or between 1 and the Gbps capacity of the link	8.2.1
A retransmission's Transmit Timestamp is the time it was retransmitted	8.2.2, 4.5.1
Read and write header (RETH)	
Every packet of a write message carries identical address, R_Key, and length	8.3
The Message Length is that of the whole write or read, not of the packet	8.3
No packet extends past the Message Length, at POSN times the payload size	8.3, 2.3
Every packet but the last is full sized, so POSN gives the offset of this one.	
The last packet of a write or read ends exactly at the Message Length	8.3, 2.3
Immediate data	
A write with immediate ends with the Immediate Present opcode	8.1.1
Its earlier packets carry Write with Immediate, not plain Write.	
Immediate data appears in a message's last packet	8.4
Immediate data follows the RETH, or the data header where there is none	8.4
A send with immediate carries no RETH.	
Immediate data is carried as the application provided it, without byte swap	8.4
Atomic headers	
An atomic or its ack is single-packet, so POSN is zero and CSN undefined	8.8.1
The addend of a FetchAdd is carried in the Swap/Add Data field	8.8.1
The Compare Data of a FetchAdd is zero	8.8.1
The field carries a CmpSwap's comparand, which a FetchAdd has no use for.	
Original Remote Data is the target's value from before the operation	8.8.2
For FetchAdd before the addition, for CmpSwap before the comparison.	
Acknowledgment header	
The Flow ID identifies the flow whose sequence space the FSN fields use	8.5
An FSN is meaningful only within a flow, so the field is what scopes it.	

CNP counts CE-marked packets since the previous ack, clearing on read	8.5.1
The four-bit field saturates at 15.	
On an ACK, SACK, or BRNR-NAK, the Tag scopes the CDMSN	8.5.2, 8.5.5
Zero where tags are not supported.	
A flow's Cumulative FSN is circularly non-decreasing	4.2.3
The Max FSN is at or after the Cumulative FSN	4.2.3
An HRNR-NAK or error NAK's sequence fields identify the offending packet	8.5.3, Tbl 8.5
Tag, Cumulative FSN, Max FSN, and CDMSN are not cumulative values there.	
The RNR backoff field is below 16	8.5.6, Tbl 8.4
On an ACK, SACK, or BRNR-NAK, the CDMSN is cumulative for its Tag	8.5.5
Those messages may not yet be delivered. It starts one below the initial MSN.	
On a NAK or HRNR-NAK, the CDMSN is the MSN of the message in error	8.5.5, Tbl 8.5
The Active Port Set carries one bit per port, set when that port is active	8.5.7
Zero means the signal is unsupported; the ack itself shows a port is up.	
On an ACK, SACK, or BRNR-NAK, the rate hint is in gigabits per second	8.5.8, Tbl 8.5
A value of 1 is the minimum, and 0 means the field is to be ignored.	
On a Flow ID Error NAK, the rate hint is the supported flow count	8.5.8, Tbl 8.5
On an HRNR-NAK or other error NAK, the rate hint field is reserved	8.5.8, Tbl 8.5
The Echo Timestamp is the timestamp most recently received on that flow	4.5.1
On out-of-order receipt that is not the timestamp of the packet sent first.	
An HRNR-NAK carries neither an FSN bit vector nor an RNR bit vector	8.5.6
Ack Status 0x1, or an RNR bit vector, implies an FSN bit vector	8.5
A BRNR-NAK carries the full 32-byte FSN bit vector	8.5.10
The RNR vector's length then follows from the framed length of the packet.	
A frame appears in the FSN bit vector or the RNR bit vector, never both	8.5.10
Data not delivered to memory cannot be fully acknowledged.	
Bit 0 of the FSN bit vector, CumulativeFSN+1, is the LSB of the first word	8.5.9
The words are big-endian, so that bit is the low bit of the word's last byte.	
RNR-Cancel header	
After a BRNR, the Tag is the tag that has become ready	8.6
After a BRNR, the Posted CSN is the highest CSN with a WQE posted on it	8.6, 2.4
After an HRNR, the Tag is zero and the Posted CSN is tag zero's	8.6
Flows available is the number of receive flows the responder can accept	8.6
The rate hint is in gigabits per second, as in an acknowledgment	8.6, 8.5.8
Control and ping headers	
A control response repeats the request's response identifier	8.7
An echo request's Flow ID and Tag are the flow and tag it asks about	8.7.1
An echo response repeats the request's Flow ID, Tag, and timestamp	8.7.1
An echo's Active Port Set is its own sender's, as in an acknowledgment	8.7.1, 8.5.7
The Error State of an echo request is zero	8.7.1
The Error State of an echo response is one of the defined codes	8.7.1
Zero where no error is reported.	

An echo response repeats the request's payload	8.7.1
An echo carries the sender's TX flow count, a response its own RX count	8.7.1
Encapsulation	
Under IPv6, the hop limit high bit marks a retransmission that kept entropy	4.2.1, 4.6
The UDP checksum is zero or correct for the payload	4.1
The mCRC is correct over the Meta-RoCE headers, payload, and any padding *Omitted under PSP and over Ethernet, where the ICV or the FCS covers it.*	4.1, Fig 4.1

Chapter 10

Verbs Library Integration

Meta-RoCE is independent of any application programming interface, and is presented through more than one. This chapter specifies how an implementation presents it through the ibverbs library, and its requirements apply to such an implementation. Meta-RoCE introduces features that require configuration beyond what the library offers, and this chapter covers

1. *QP type*
2. *Congestion Control Profile Index*: This includes the data class of service, transmit flow counts, receive flow count, epsilon, beta, lambda, gamma, omega, minimum and maximum RTO, the default RTT estimate before the first measurement, and SACK retransmission immediacy.
3. *Initial MSN/CSN*
4. *Retransmission timeout and persistent retry*: Reusing the `timeout` and `retry_cnt` attributes.
5. *QP MTU*: Reusing the `path_mtu` attribute, extended to 8192 bytes.
6. *Tags*: Adding a work queue and tag index to a QP.
7. *Multiple ports*: Setting multiple local and remote network addresses (GIDs) for multi-port / multi-plane communication.

This configuration takes place before the QPs transition to RTR state; these parameters do not change after that.

10.1 QP Type

An RDMA device can have a mix of different types of RoCEv2 QPs as well as ones that use Meta-RoCE. To specify the latter, set `qp_init_attr->qp_type` to `IBV_QPT_DRIVER`, the value `0xFF` that verbs reserves for a driver-specific queue pair type. The same value selects Meta-RoCE on `ibv_create_qp_ex()`. A device that supports only Meta-RoCE queue pairs has nothing to disambiguate.

10.2 Profile Selection

QPs have a set of parameters related to congestion control, the value of which depends on the specific combination of their endpoints. For instance, QPs with long distance between hosts would need to have a larger maximum RTO and a larger epsilon. A way to achieve this kind of customization is to create a set of profiles (e.g., based on distance) and attach them to QPs. The number of such profiles need not be large. A QP will be associated with a single profile. Profiles must be configurable, but need not change after QP initialization.

The selection between profiles may be by:

- *Prefix match of the destination GID(s)*: In this case, based on the prefix of the destination GID(s), the driver determines at runtime which profile to use.
- *Device-specific configuration*: Profiles selected by the application through a device-specific method at runtime.

10.3 Initial CSN/MSN

The message sequence numbers correspond roughly to the RoCE PSN, which is not used in Meta-RoCE. To configure initial CSN and MSN, while remaining compatible with existing code, the `IBV_QP_SQ_PSN` and `IBV_QP_RQ_PSN` options carry the initial values. Since the `sq_psn` and `rq_psn` variables are 32 bits long, the 16 MSBs represent the starting CSN and the 16 LSBs represent the starting MSN. As used in the verbs interface, `sq_psn` configures the values the local NIC uses when sending; `rq_psn` configures the values it expects when receiving. Setting the starting RQ MSN configures the CDMSN to be the starting RQ MSN minus 1.

Receive sequence information is configured before the queue pair enters RTR, and send sequence information before it enters RTS. A state transition moves every Tag on the queue pair, while sequence numbers belong to the Tag whose `ibv_qp` took the call, so each Tag takes its own `ibv_modify_qp`.

It is typical to make a single `ibv_modify_qp` call that both configures the fields a state needs and moves to that state. One such call here configures Tag 0 and moves every Tag, leaving the rest unconfigured. Configure each other Tag first, with a call that omits `IBV_QP_STATE`, and let the last call make the transition.

In practice, the verbs PSN fields may be restricted to a valid 24-bit RoCE PSN; attempting to configure an initial CSN greater than 255 may not be supported.

10.4 Retransmission Timeout and Persistent Retry

The `IBV_QP_TIMEOUT` and `IBV_QP_RETRY_CNT` options of `ibv_modify_qp()` give two values that bound retransmission. Both keep the encodings described in `ibv_modify_qp(3)`: `attr.timeout` is a five-bit value denoting a duration of $4.096\,\mu\text{s} \times 2^{\texttt{timeout}}$, and `attr.retry_cnt` is a three-bit count of retries following the first attempt.

Before any round trip time has been measured, the initial RTO must not exceed the configured timeout, and may be set equal to it. It affects only that first interval. The configured timeout is the encoded duration, which the driver may treat as a hint or replace with its own configuration. Once measurement begins the RTO follows Section 4.5.2, within the bounds in the congestion control profile.

The two values together bound how long a queue pair keeps trying. A queue pair with data to send that makes no forward progress for approximately the configured timeout multiplied by one more than `retry_cnt` reports a local timeout error and enters the error state (Section 5.4). Meta-RoCE does not stop after `retry_cnt` retransmissions: retransmission is per flow and driven by both selective acknowledgment and timeout, so the number of attempts within that interval depends on the flow count and on the loss pattern. The product is read as a duration, not as a count. While an HRNR holds transmission the bound is suspended, and the HRNR retry limit applies instead.

An implementation must accept a `timeout` of at least 20, which encodes approximately 4.3 seconds. A device may define a minimum duration and raise a smaller configured value to it. The bound on the initial RTO applies after any such minimum.

With a `timeout` of zero, the initial RTO comes from the congestion control profile. `retry_cnt` is a multiplier and has no distinguished values. A `retry_cnt` of zero gives the shortest bound, one interval.

10.5 QP MTU

The QP MTU is set with `ibv_modify_qp()`, as `attr.path_mtu` under `IBV_QP_PATH_MTU`, in the transition to RTR, as for RoCE. It is the payload size by which POSN indexes a message (Section 2.3); both endpoints configure the same value, and it does not change once the first work request is posted (Section 2.3.1). It is QP-wide, so a queue pair's Tags share it (Table 10.1).

The `ibv_mtu` enumeration ends at `IBV_MTU_4096`, value 5. Meta-RoCE continues its doubling with value 6 for an 8192-byte QP MTU.

10.6 Tags

Tags identify "sub" queue pairs, which act as additional work request and completion queues associated with the same pair of endpoints, using the same Meta-RoCE flows and retaining the same congestion state and configuration. To create a Tag, `ibv_create_qp_ex()` is used, referring to the original QP number. The number of Tags a queue pair supports is implementation defined.

What configures the reliability, congestion control, and ordering of the queue pair is shared by its Tags; a Tag has its own queues and its own sequence numbers. Table 10.1 divides the configuration.

QP-wide configuration	**Tag-specific configuration**
Congestion control profile	Work and completion queues
MTU	Inline data cap
Protection domain	Initial MSN and CSN (Section 10.3)
Retransmission timeout, with `retry_cnt`	
`max_rd_atomic`	

Table 10.1: Configuration of a queue pair with Tags.

10.6.1 Creating a Tag

When a QP is first created using `ibv_create_qp()` or `ibv_create_qp_ex()`, it gets one Tag by default, which always has to be Tag number 0. The QP number for the newly created QP is stored in the `qp_num` field of the returned `ibv_qp` struct. A QP number in RoCE is 24 bits but it is stored as a 32-bit field (*note*: QP numbers in Meta-RoCE are 32 bits, but for this discussion, they can be restricted to 24 bits as well). To support Tags, the 8 MSBs of `ibv_qp->qp_num` as the Tag number, which is 0 for the first Tag on a QP.

To create an additional Tag within an existing QP, use `ibv_create_qp_ex`() to take advantage of the `create_flags` and `source_qpn` fields in the `ibv_qp_init_attr_ex` struct. If the flag `IBV_QP_CREATE_SOURCE_QPN` is set in `init_attr_ex->create_flags` and a QP with the 24 LSBs of this number already exists, then create a new Tag within the existing QP. The number of the newly created Tag will be the 8 MSBs of `init_attr_ex->source_qpn`. Associate the parameters (e.g., the completion queues) in `init_attr_ex` with this new Tag. Multiple Tags may share completion queues but they have separate work queues. If `IBV_QP_CREATE_SOURCE_QPN` is set but a QP with number `init_attr_ex->source_qpn` does not already exist, then return an error. Tags share the protection domain of their queue pair: if `IBV_QP_INIT_ATTR_PD` is set in `init_attr_ex->comp_mask` and `init_attr_ex->pd` is not the protection domain of the existing QP, then return an error.

10.6.2 Use of the returned QP structure

The `ibv_qp` that is returned by `ibv_create_qp_ex`() will represent the new Tag, and will be associated with the specified queues. All fields in the returned `ibv_qp` are valid. The `ibv_qp` struct can be used as that of a normal QP, even though it represents a Tag.

10.6.3 Timing restrictions

All Tags must be created while the QP is in the RESET state, before it is moved to INIT. Bring-up is queue pair wide; a Tag has no RESET, INIT, RTR, or RTS transition of its own. A Tag does enter the error state by itself (Section 5.3.3). An `ibv_modify_qp` on a Tag's own `ibv_qp` configures that Tag and causes no state transition. Any attempt to create a Tag after the QP has moved to INIT should fail. Multiple QPs (each with multiple Tags) may be created in parallel, as long as the preceding INIT requirement is respected for each QP. Once the QP is moved to INIT, then the only way to create new Tags is to reset the QP. In application terms, the CommID (communicator ID) is fixed, and the only way to change it is to tear down the QP and re-init.

A queue pair persists while any of its Tags does, and is destroyed with the last of them.

10.6.4 Tags on both endpoints

Tags must be created on both endpoints. On the responder, the Tag number received in Meta-RoCE packets will be directly used to determine which Tag to associate the packet with. Therefore, the Tag number must be the same on both endpoints (even if the QP number is different). If a Tag number does not exist on the responder, then it will return a BRNR indicating that the Tag is not ready. (See Section 5.2.2.)

10.6.5 Miscellaneous

Destination GID is a property of the QP: all Tags within a QP share the same destination GID. It is expected that the implementation will internally store a data structure tracking the Tags associated with each QP. The

Tags may reside as an extension to each QP state, or they may be stored in a new structure entirely.

10.7 Read with Immediate

A read with immediate has no opcode in libibverbs, so the work request and the responder's completion use provider-defined values. An application posts one with `ibv_post_send()` and `wr->opcode` set to `IBV_WR_DRIVER1`, filling `wr->wr.rdma` and the scatter list as for a plain read, and `wr->imm_data` with the immediate, which is sent without byte swap (Section 8.4). On the requester it counts against `max_rd_atomic` and completes exactly as a plain read does, reporting no immediate.

At the responder the request carries no payload and consumes a receive WQE, which serves as a completion notification rather than a scatter destination; where none is posted the responder returns a BRNR. The receive completes with opcode `IBV_WC_DRIVER1`, `IBV_WC_WITH_IMM` set, `imm_data` holding the immediate, and `byte_len` the length of the read. The opcode differs from write with immediate's `IBV_WC_RECV_RDMA_WITH_IMM` so the application can tell the two apart, and keeps the `IBV_WC_RECV` bit, so code that identifies receive completions by testing `opcode & IBV_WC_RECV` still recognizes it. The receive completes only once the initiator has acknowledged the read response (Table 3.1), so the responder's application may reuse the buffer that was read as soon as it sees the completion.

10.8 Multi-port API

The configuration of the local and remote GIDs in the ibverbs library is done by invoking `ibv_modify_qp()` and providing the appropriate local network-level address, using the `ah_attr` fields `grh.sgid_index` and `port_num`, and remote network-level endpoint address, using `grh.dgid`.

To support configuring multiple sets of these pairs, at least one of the following two methods should be supported:

- Repeated invocation of `ibv_modify_qp()` using the `IBV_QP_AV` flag will append, rather than overwrite.
- An alternate `ibv_modify_qp_multi_port()` function that accepts a single structure to represent all pairs.

All port/GID configuration must occur while the QP is in the INIT state; dynamically modifying the set of addresses need not be supported. The receiver should not use the configured address set for filtering; it is expected that devices accept traffic from addresses that were not included as a dgid in this call. If the number of entries exceeds what the hardware can support, an error should be returned.

10.8.1 Extending the semantics of `ibv_modify_qp()`

To support the configuration of multiple network address pairs for the same QP, each successive call to `ibv_modify_qp()` (when configuring the address vector) will no longer replace the existing GID settings. Instead, it will add a new remote and local GID pair to the QP.

Key aspects of this new behavior include:

- Appending GIDs: When `ibv_modify_qp()` is called, the provided `attr->ah_attr` fields `grh.dgid`, `grh.sgid_index`, and `port_num` will be used to add a GID pair to the QP.

- Uniqueness Enforcement: Each GID pair will be added only once. If a subsequent call attempts to add a pair that already exists for the QP, it will be ignored, preventing redundant entries.
- GID pools: An SGID or a DGID can be added to the SGID or the DGID pool. All GIDs of the SGID pool will communicate with all GIDs in the DGID pool. A DGID can be added to the DGID pool if both `port_num` and `sgid_index` are set to `UINT8_MAX`. Similarly, an SGID can be added to the SGID pool when `attr->ah_attr.grh.dgid` is set to all ones.
- Endpoint combinations: The QP will use either the list of GID pairs supplied by successive calls to `ibv_modify_qp()`, or the product of the entries of the SGID and the DGID pools.